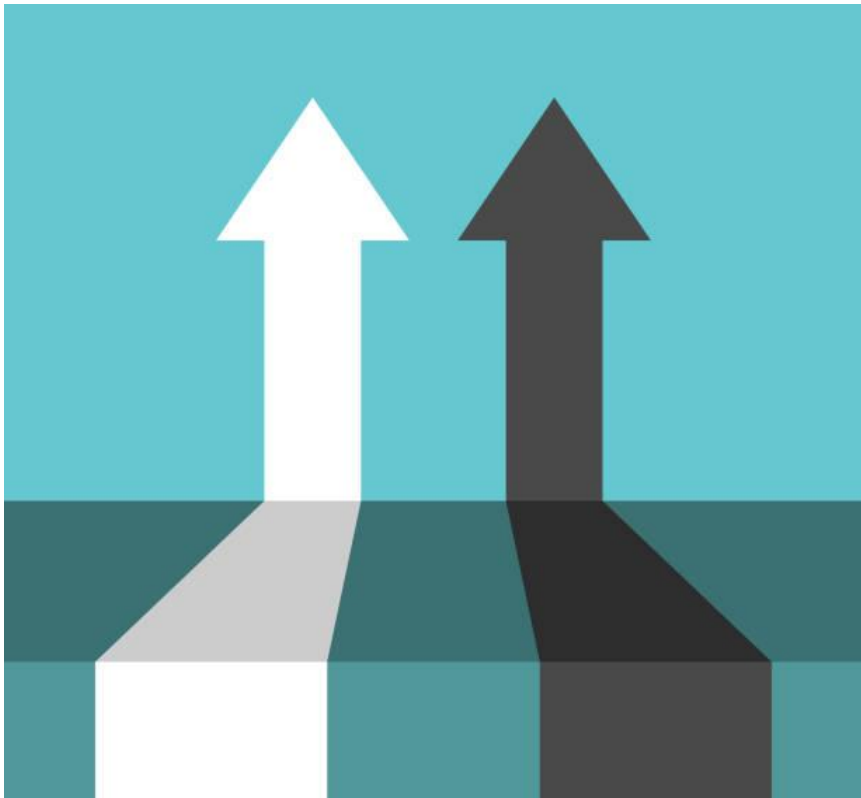




Cómputo Paralelo

y en la Nube



Profesor:

Dr. Octavio Gutiérrez

octavio.gutierrez@itam.mx



Escenarios de Paralelización Manual

Ordenar números



141592653589793238462643383279502
419716939937510582097494459230781
062862089986280348253421170679821
086513282306647093844609550582231
535940812848111745028410270193852



Ordenar números

274	988	583	34	872
562	790	872	389	832
675	876	915	193	753
175	782	615	458	962
202	554	263	650	361
396	795	282	792	640
552	151	611	455	777
501	697	406	953	338
850	98	61	280	932
910	864	357	238	702
763	811	24	248	1
844	530	28	800	748
831	373	393	978	809
957	924	920	168	78
645	513	751	188	99
632	780	182	662	595
831	896	190	99	971
57	872	617	256	528
260	470	851	457	929
260	522	461	869	427

100

[illegible]

**¿Por qué es
fácil / complicado
trabajar
en paralelo?**





Objetivos

- Desarrollar habilidades de **programación paralela** (en C++, Julia y Python).
- Introducir a los **diferentes modelos** de programación paralela.



Objetivos

- **Comparar** cualitativamente y cuantitativamente programas paralelos en una arquitectura dada.
- Proveer conocimiento sobre **arquitecturas modernas de procesadores** capaces de ejecutar código en paralelo.

Memory Controller

Core

L1 cache

Core

Queue

Core

Core

Larger L2
cache

Shared L3 Cache

Misc

Op11



Temario

- ***Introducción al paralelismo***
 - Justificación y necesidad del paralelismo
 - Retos de la programación paralela



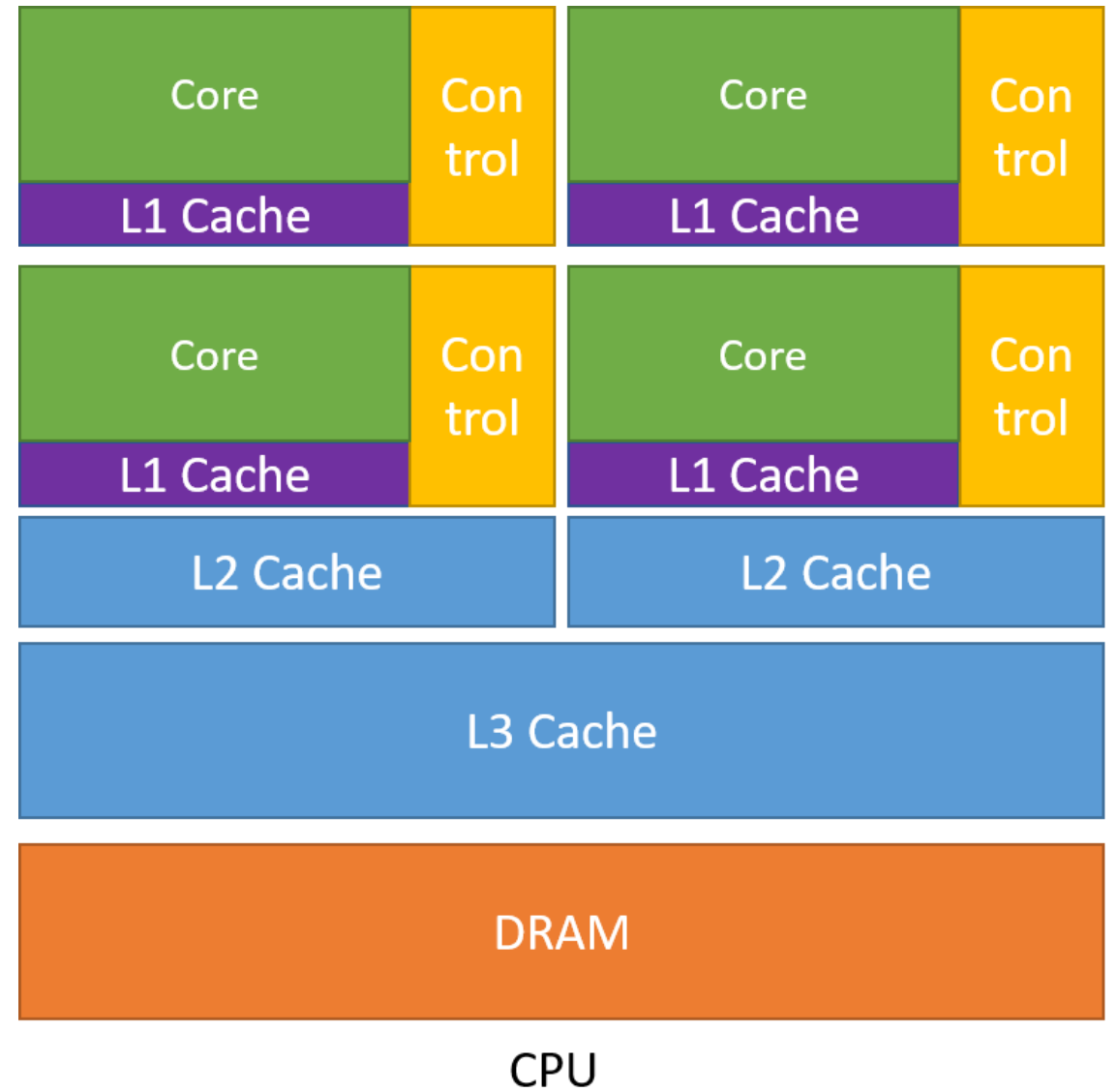
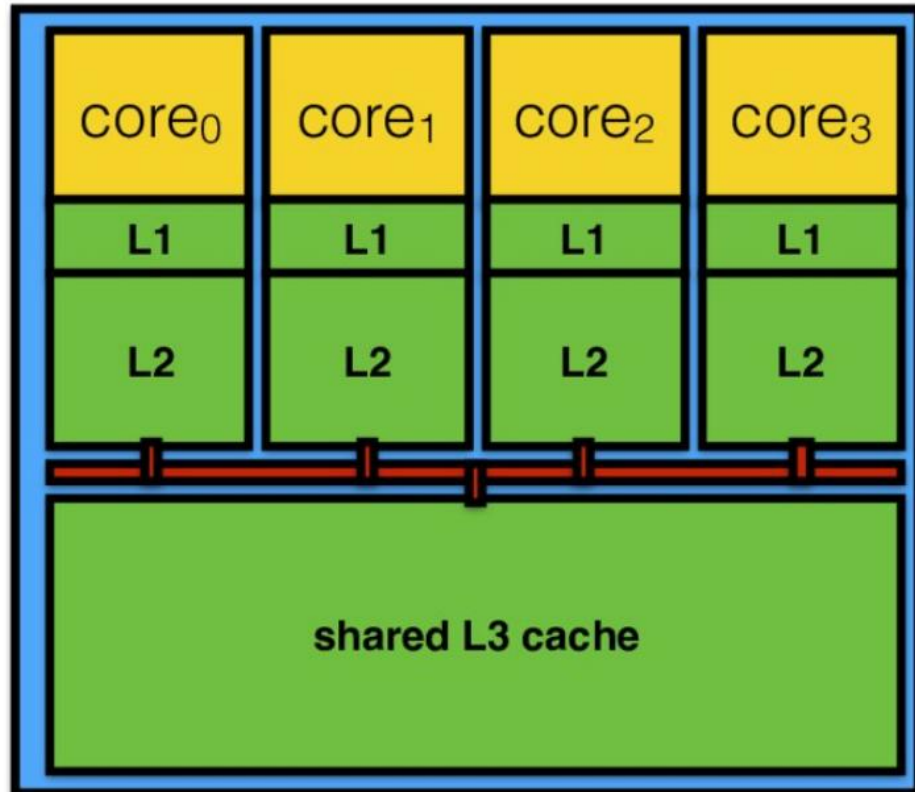


Temario

- ***Arquitecturas modernas de procesadores***
 - Taxonomía de Flynn
 - Arquitecturas multinúcleo y sus niveles de cache
 - Arquitecturas GPU



- *Arquitecturas modernas*





Temario

- ***Modelos de Programación Paralela***

- Modelo de hilos
- Modelo de memoria compartida
- Modelo de paso de mensajes
- Modelo paralelo de datos





Temario

- ***Diseño de Programas Paralelos***

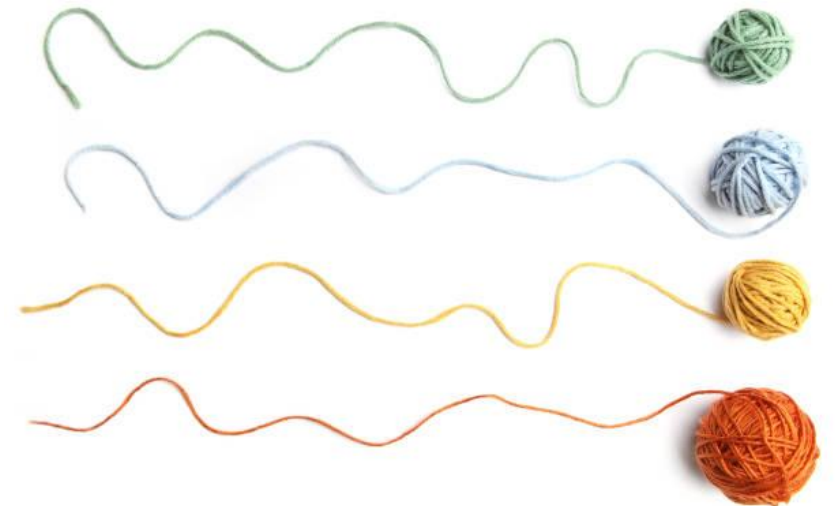
- Relación entre el problema, los recursos y la estrategia de paralelismo
- Granularidad y particionamiento
- Comunicación y sincronización





Temario

- ***Modelo de Hilos***
 - Problemas comunes: deadlock y condiciones de carrera.
 - Sincronización, regiones críticas y operaciones atómicas.





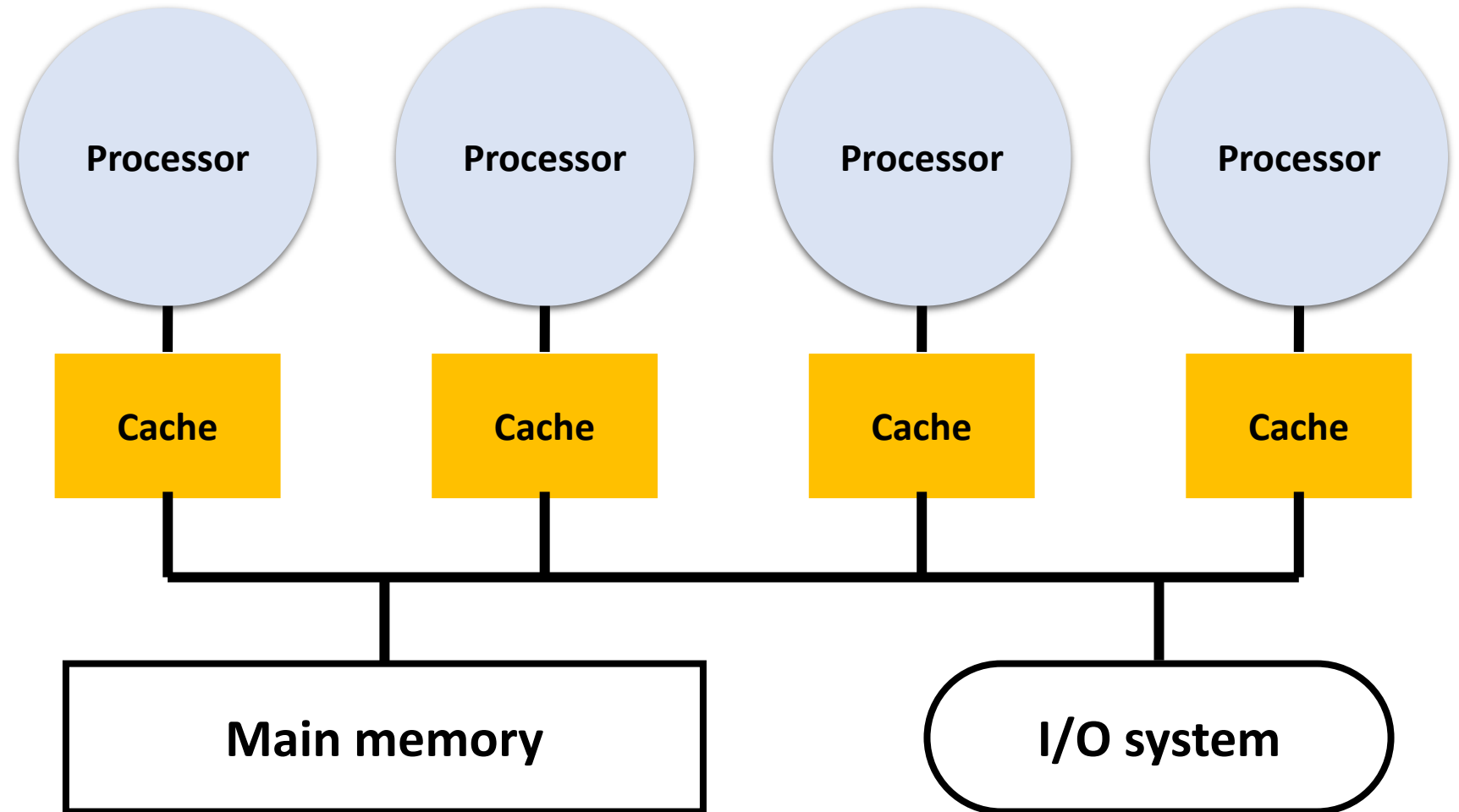
Temario

- *Modelo de Memoria compartida (C++ OpenMP)*
 - Hilos y sincronización
 - For paralelos
 - Directivas

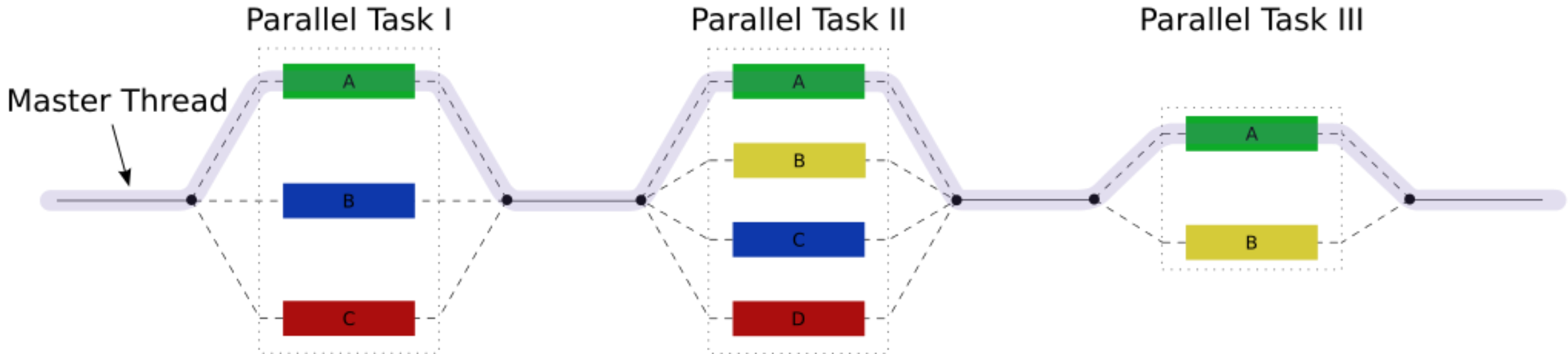
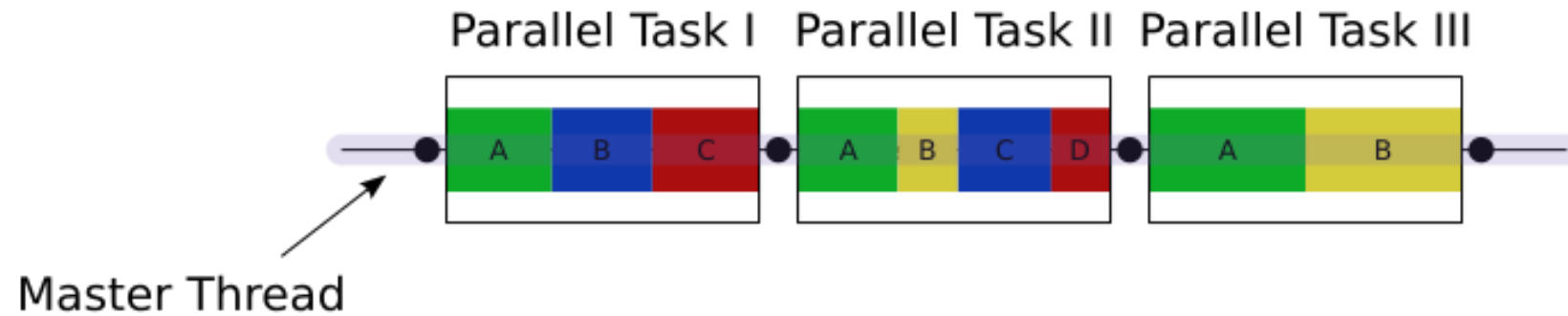
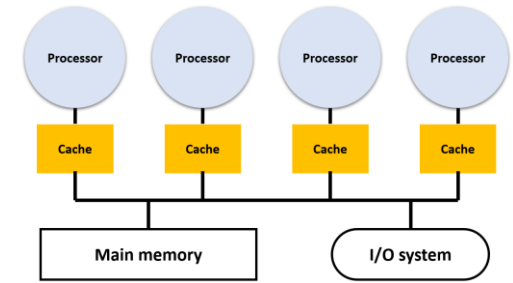


Open Multi-Processing

Configuración
más común



Open Multi-Processing





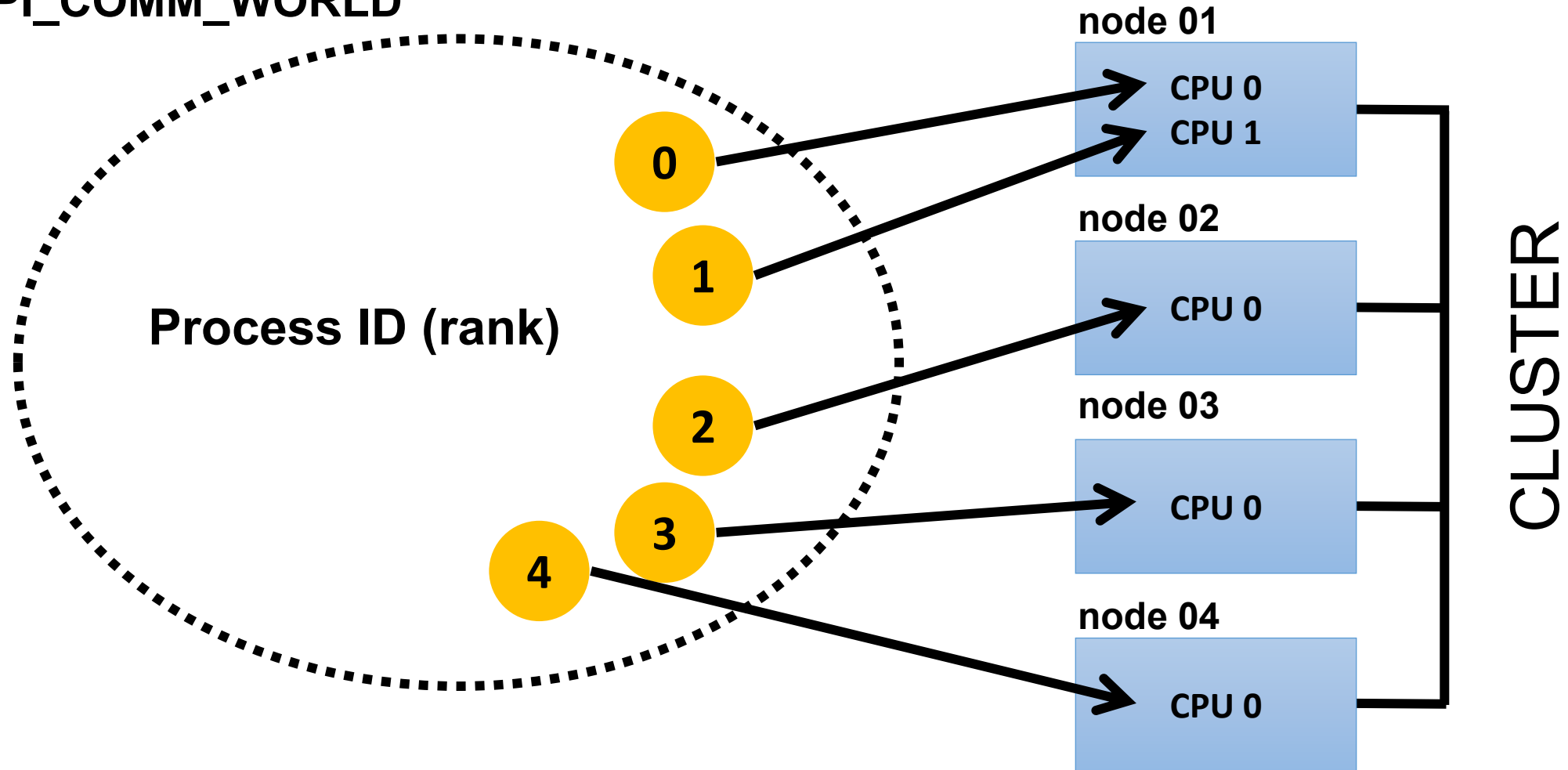
Temario

- ***Modelo Paso de Mensajes (C++ MPI)***
 - Comunicación punto a punto
 - Comunicación colectiva
 - Directivas

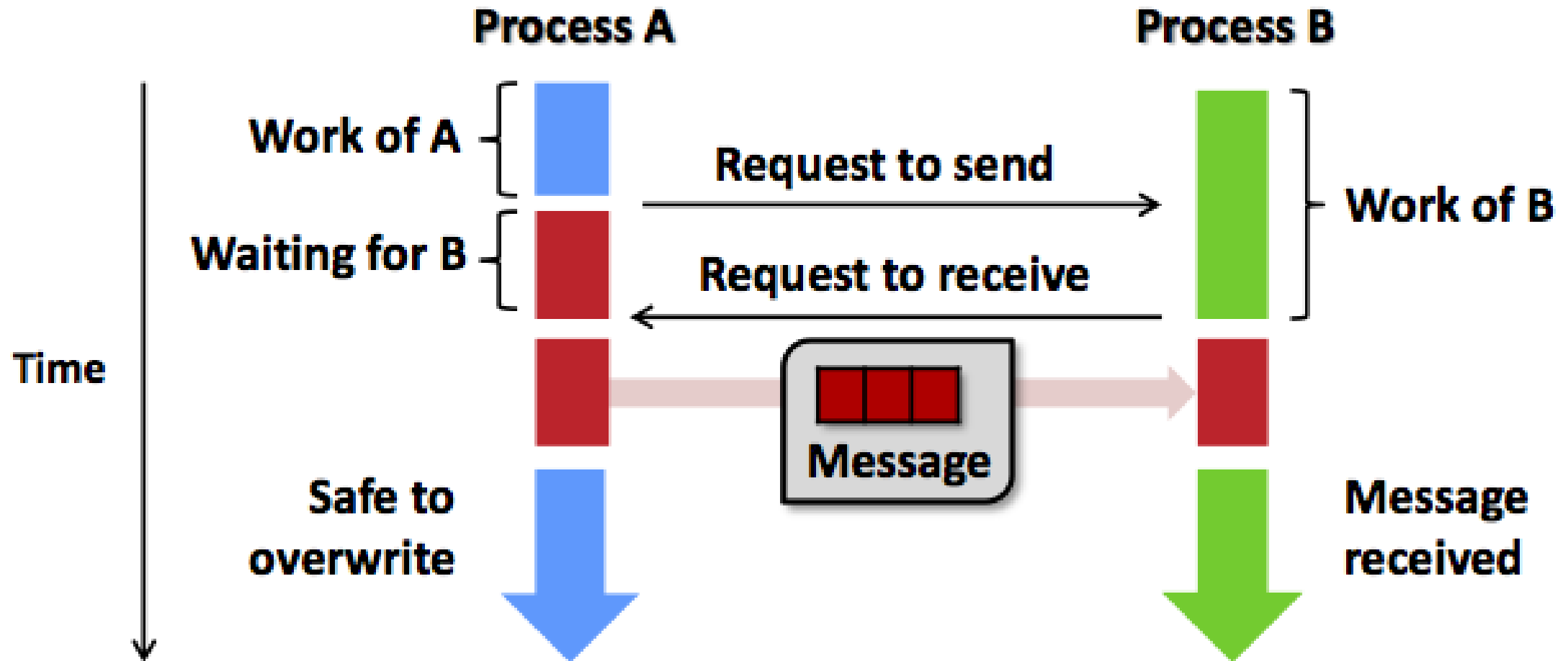


MPI – Message Passing Interface

MPI_COMM_WORLD



MPI – Message Passing Interface





Temario

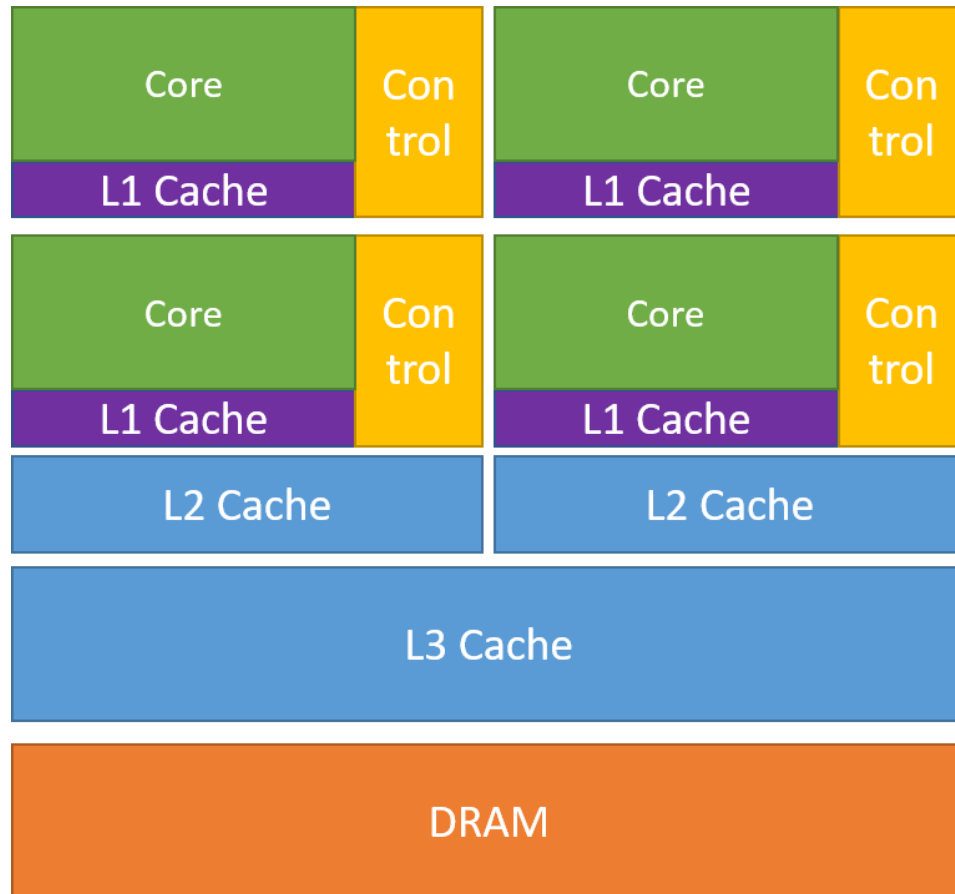
- ***Programación con CUDA para GPU (Python)***
 - Introducción a CUDA
 - Modelo e Interfaz de programación



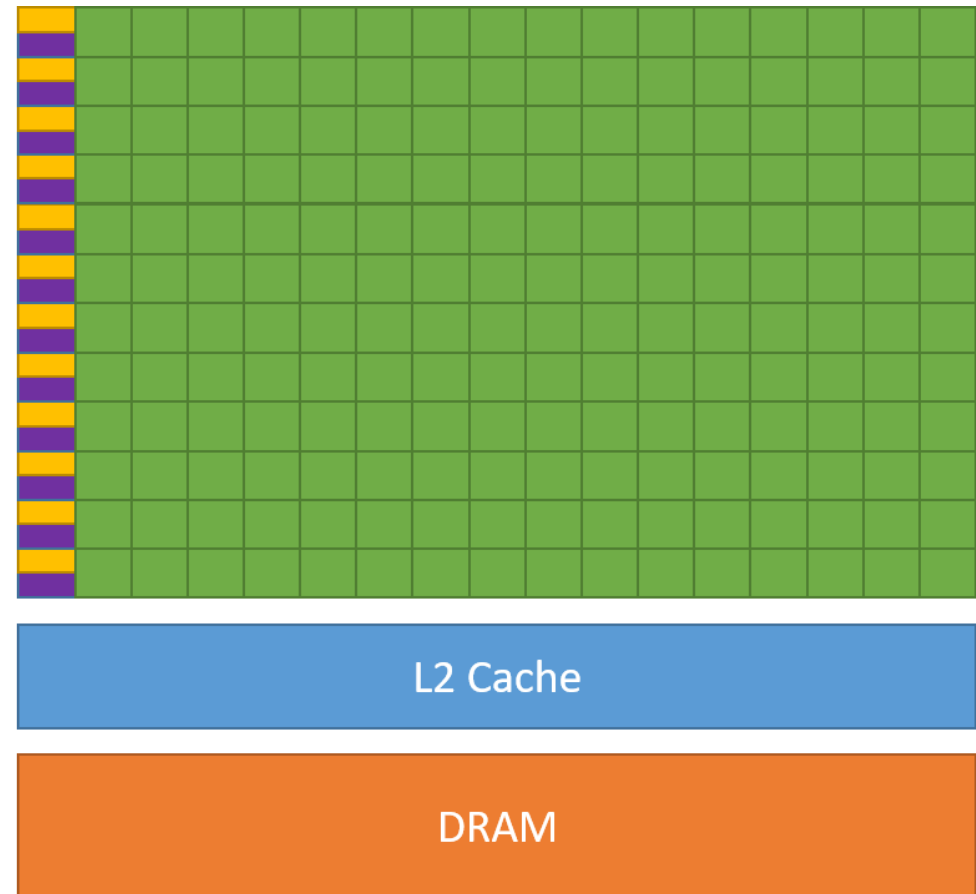
NVIDIA®

CUDA®

GPUs & CUDA

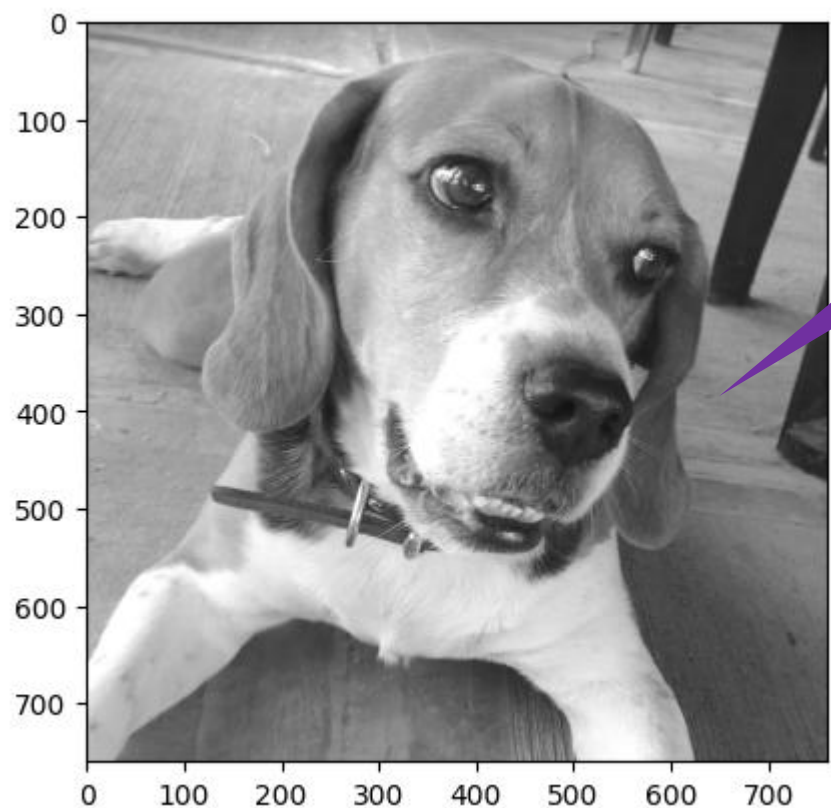


CPU



GPU

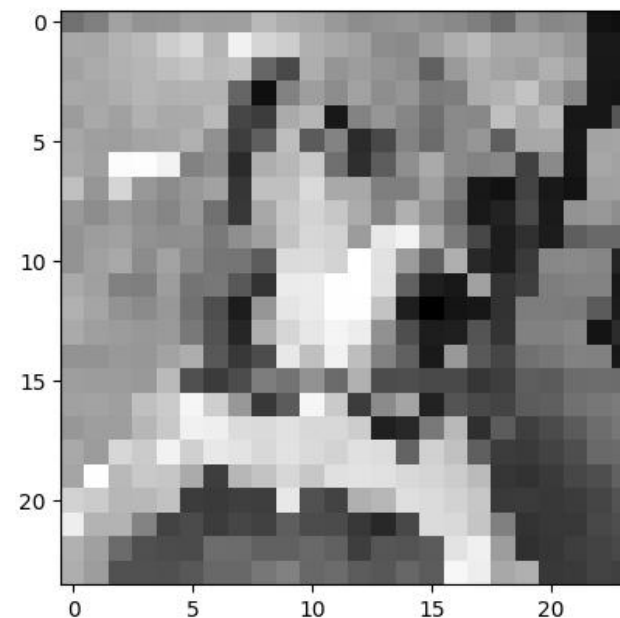
Ejercicio de paralelización con GPUs



760 x 760

Me llamo
Dante.png

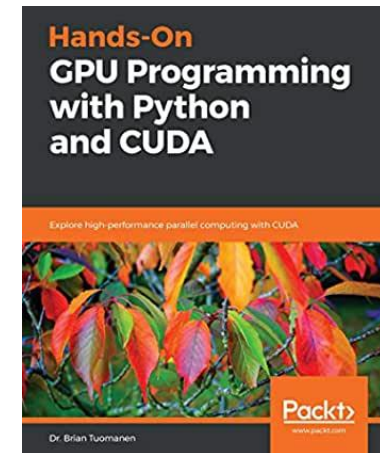
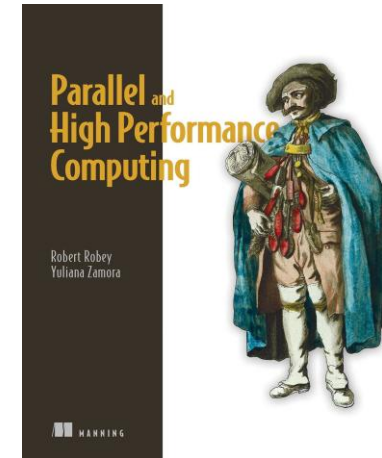
Reducción



24 x 24

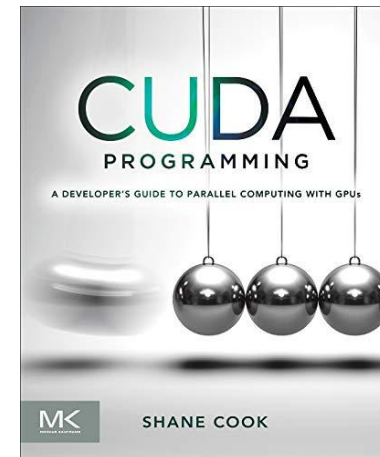
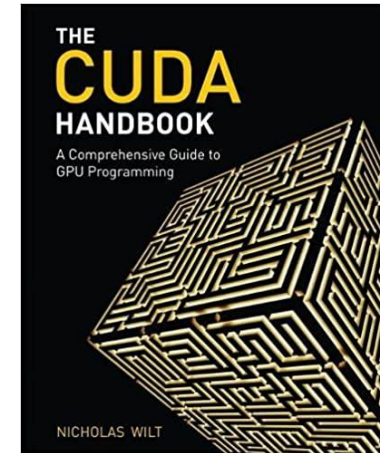
Bibliografía

- Robey, Robert, and Yuliana Zamora. Parallel and High Performance Computing. Simon and Schuster, 2021.
- Tuomanen, Brian. Hands-On GPU Programming with Python and CUDA: Explore high-performance parallel computing with CUDA. Packt Publishing Ltd, 2018.



Bibliografía

- Wilt, Nicholas. The CUDA handbook: A comprehensive guide to gpu programming. Pearson Education, 2013.
- Cook, Shane. CUDA programming: a developer's guide to parallel computing with GPUs. Newnes, 2012.



Atención a Alumnos



Horario Flexible

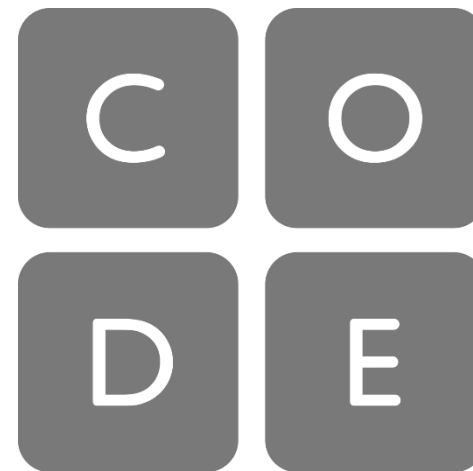
- Favor de agendar en:
calendly.com/octaviogtz/30min

La reunión puede ser
presencial o en zoom.
(aunque por defecto
genera vínculo de zoom)



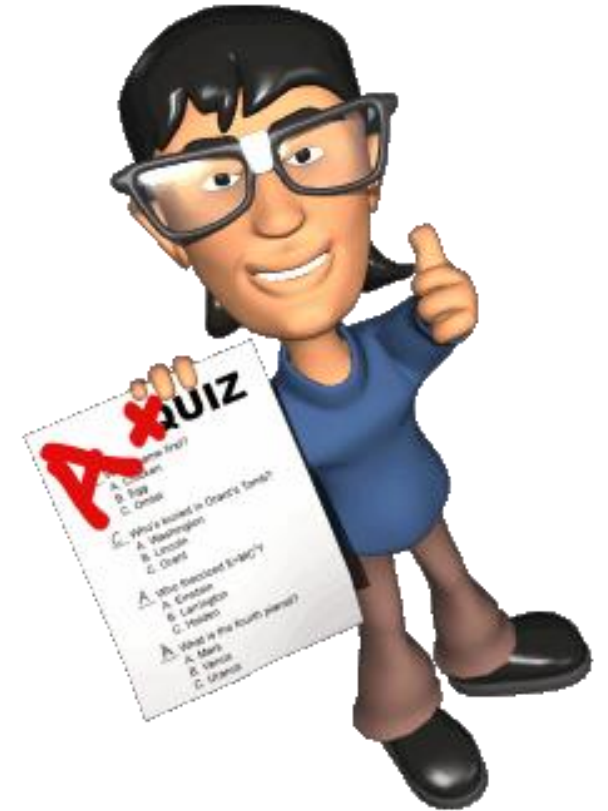
Código para prácticas

<https://github.com/octavio-gutierrez/computoparalelo2026b>



Evaluación

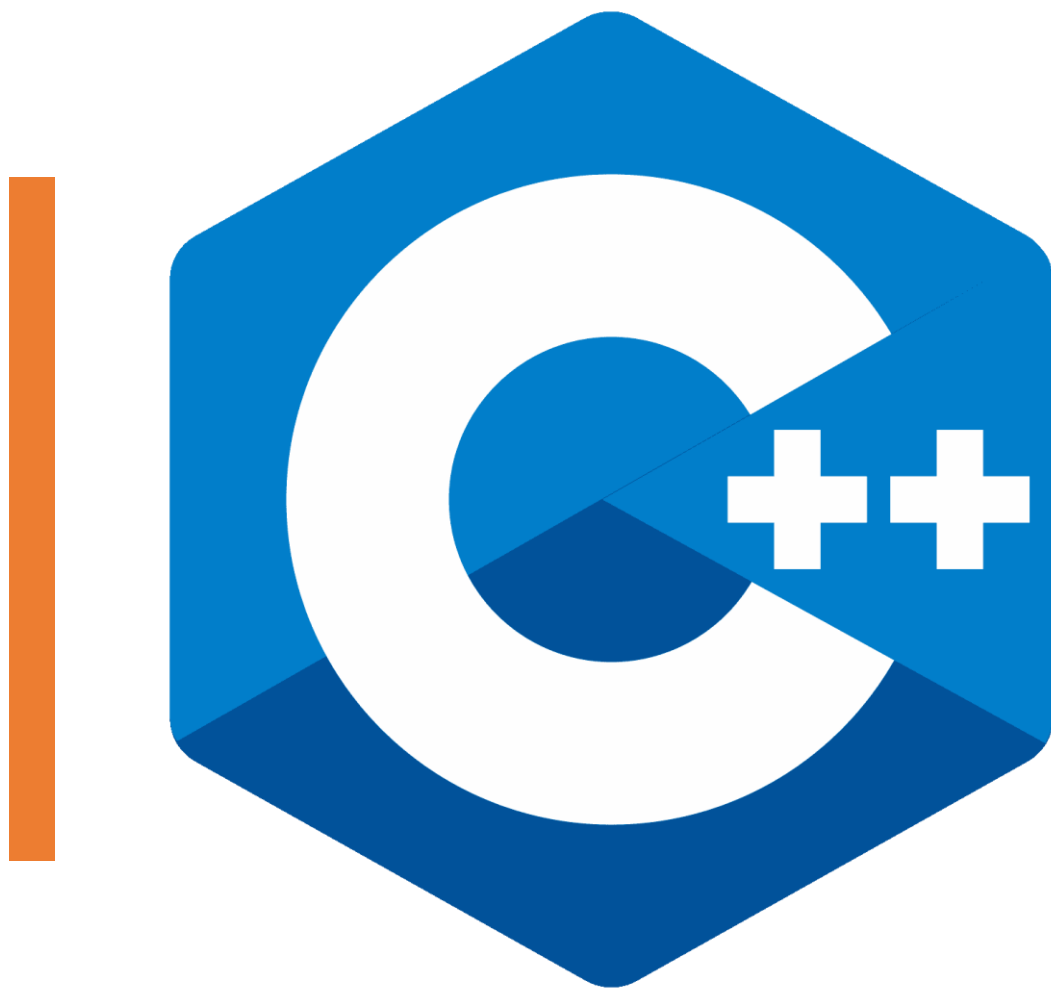
- Trabajo en clase (ejercicios selectos) **10%**
- Examen Parcial **20%**
Octubre de 2026
- Proyecto Apertura **20%**
Octubre de 2026
- Proyecto Clausura **20%**
1° de Diciembre de 2026
- Examen Final **30%**
A definirse por control escolar



Lenguajes de programación y librerías

- C++, Julia y Python.
- MPI y OpenMP para C++ y CUDA para Python.



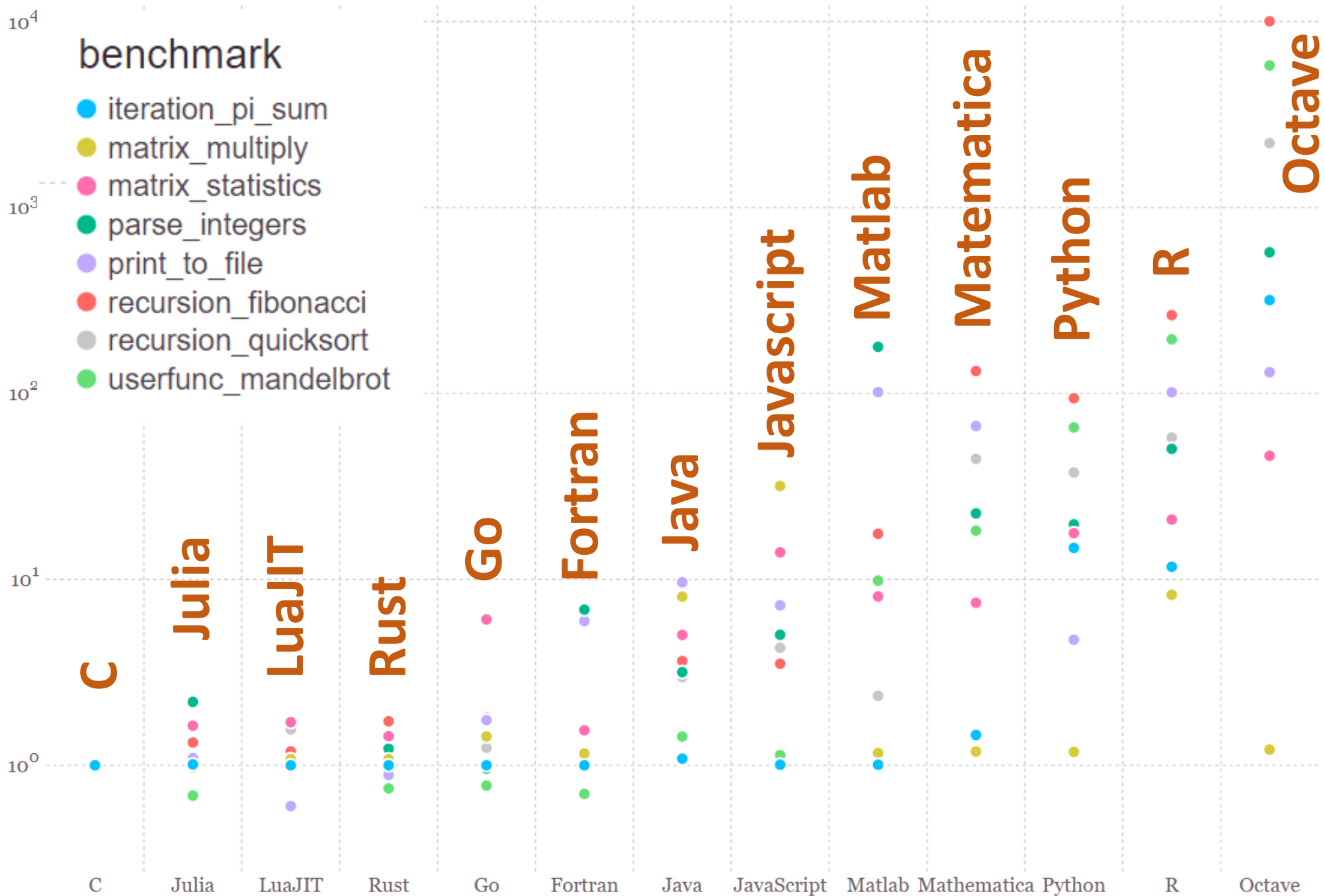


- Compilado (ahead of time)
- Portabilidad
- Manipulación a bajo nivel
- Manejo de Memoria
- Escalable
- Tipado estático



- Compilación Just-In-Time
- Sintaxis simple
- Tipado dinámico aunque soporta estático
- Manejo de memoria automatizado
- Diseñado para paralelizar
- Puede llamar código de C/C++ & python

<https://julialang.org/>



Instalación Julia & Visual Studio Code

- <https://julialang.org/downloads/>
- <https://code.visualstudio.com/download>
 - File>Preferences>Extensions> JuliaLanguageSupport – JuliaLang
- Path – Windows & Mac

C:\Users\Octavio\AppData\Local\Programs\Julia-1.10.0\bin

ln -s /Applications/Julia-1.10.app/Contents/Resources/julia/bin/julia /usr/local/bin/julia

Configuración Julia & Visual Studio Code

Antes de Iniciar

```
CPU Information
Intel(R) Core(TM) i7-8750H CPU @ 2.20GHz
Intel VT-x Technology           Supported
CPU Speed                      2200 MHz
CPU Stepping                   U0
Microcode Patch                84
CPU Cores                     6Core(s) / 12Thread(s)
Intel(R) Graphics Controller
Intel(R) GOP Driver [9.0.1074]
```

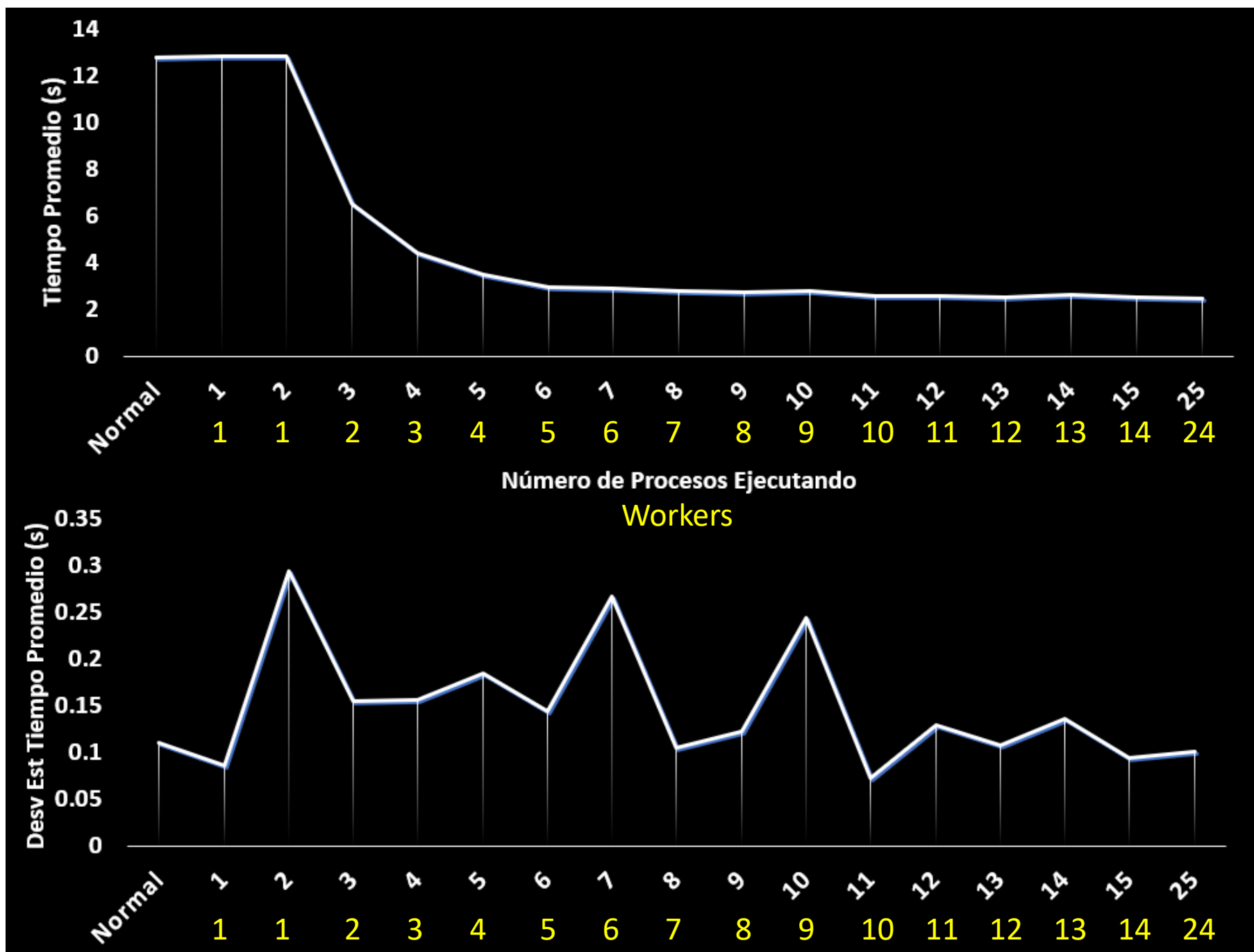
- Número de Procesos **vs** Número de Workers **vs** Hilos de CPU **vs** Cores
 - Sys.CPU_THREADS
 - nprocs() / nworkers() / procs()
 - addprocs(1) / rmprocs(p:q)
 - Configuración de la Extensión: "julia.NumThreads": "auto"
 - Variable de ambiente: JULIA_NUM_THREADS
 - include("helloworld.jl")

Mi primer programa Paralelo con Julia

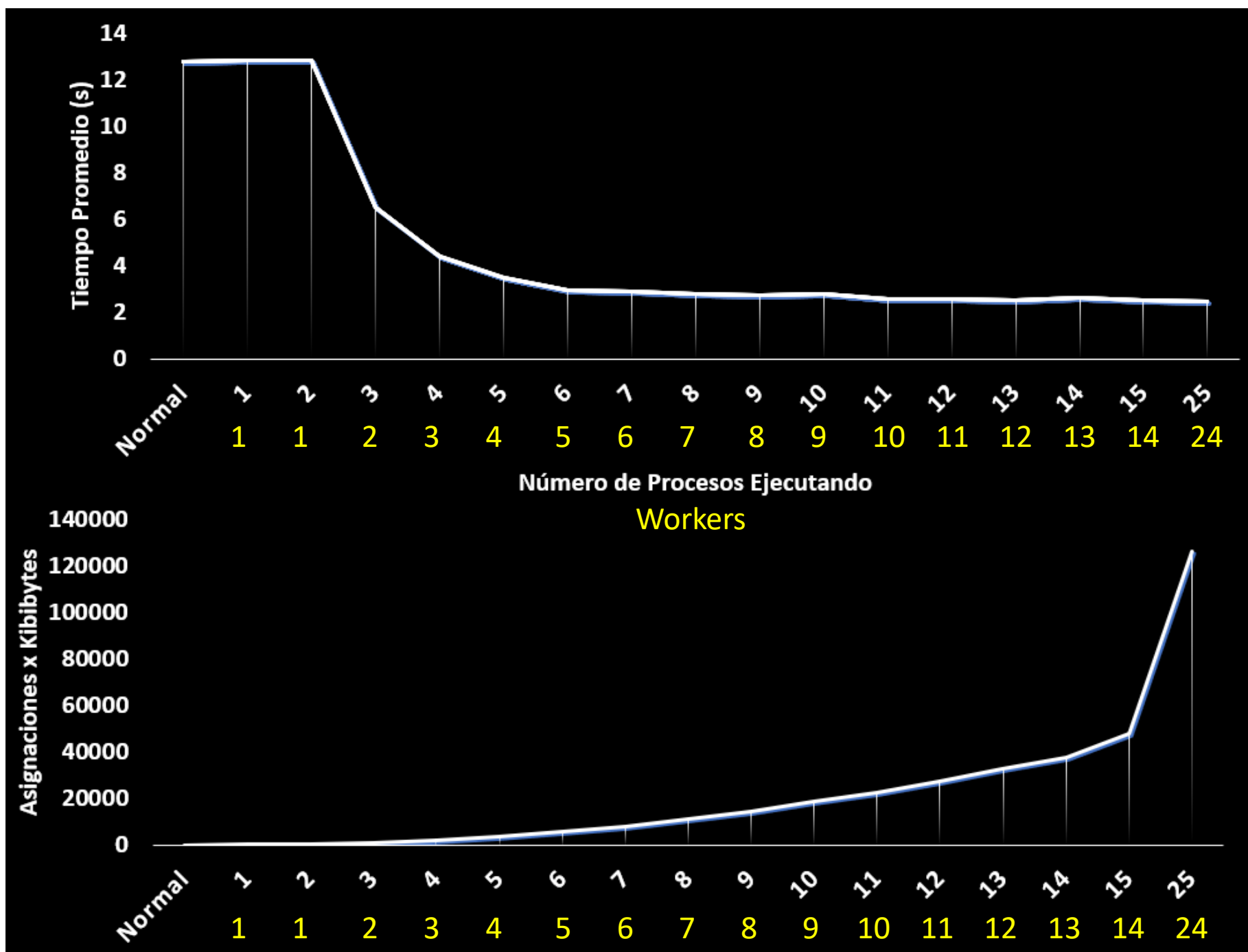
```
function azarparalela(lanzamientos)
    numcaras = @distributed (+) for i = 1:lanzamientos
        Int(rand{Bool})
    end
    return numcaras
end
```



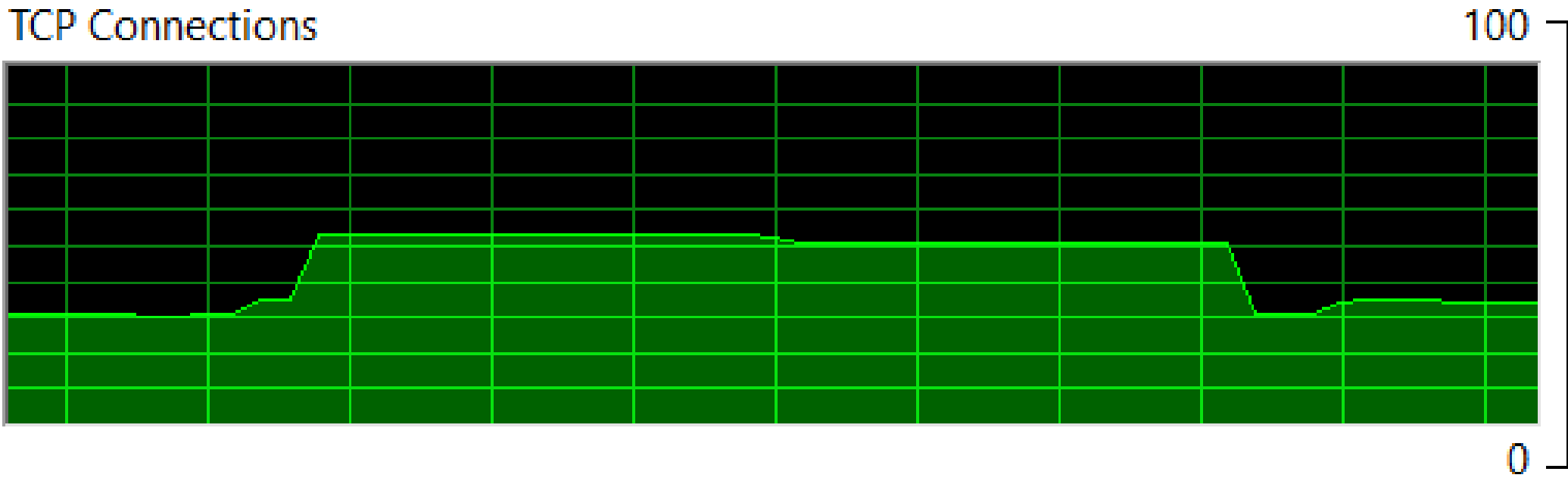
Resultados



Resultados



Resultados





Cómpu

to

Para

lelo

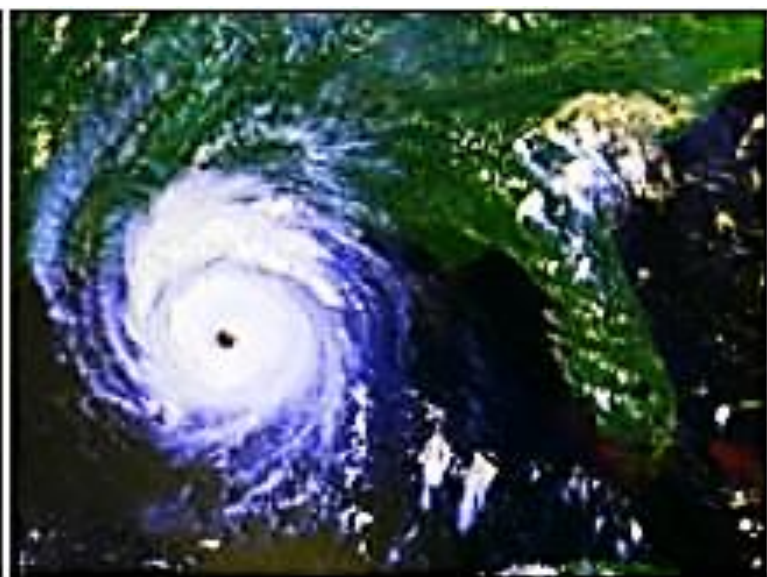




Galaxy Formation



Planetary Movments



Climate Change



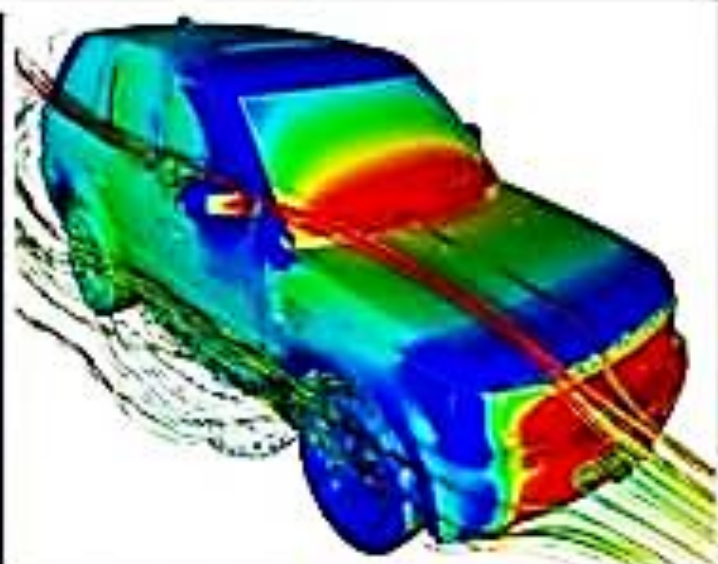
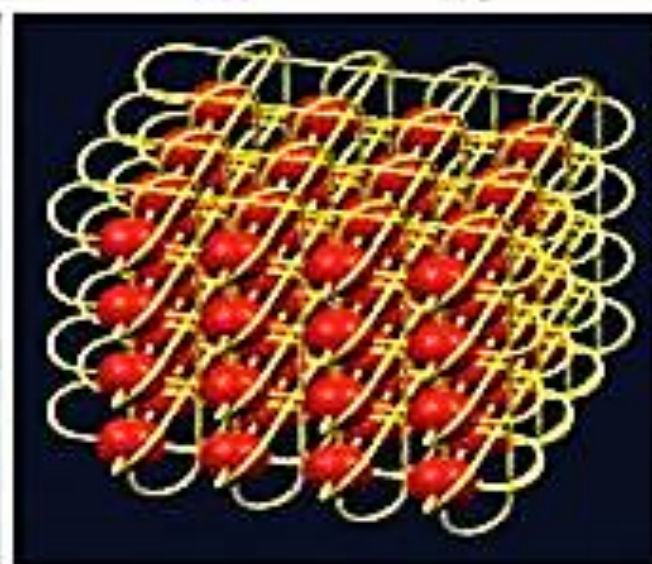
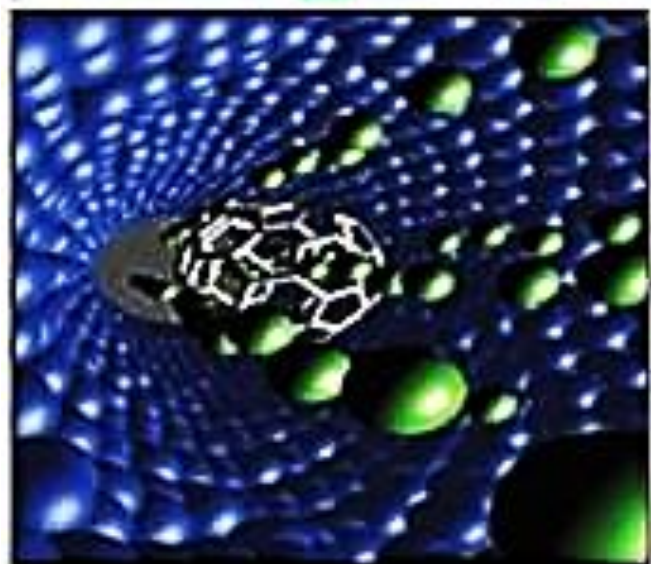
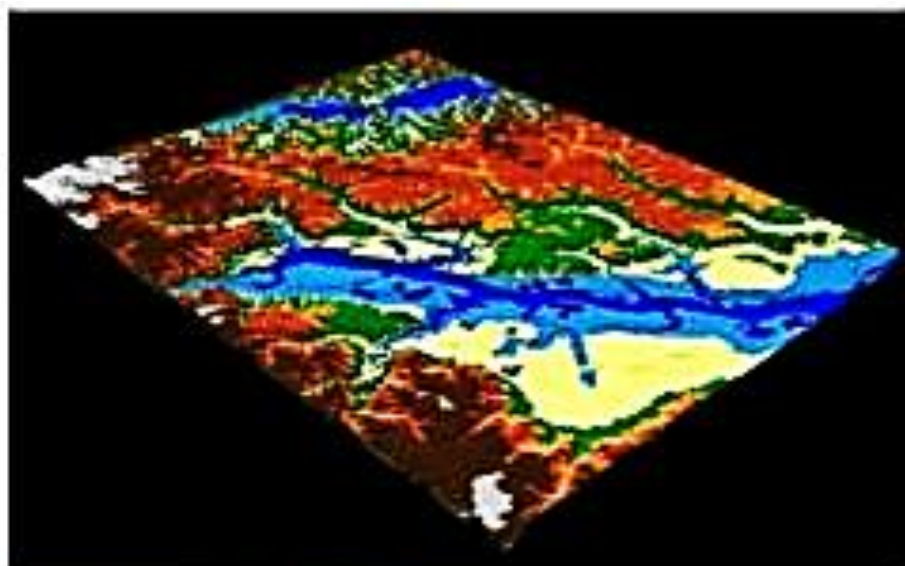
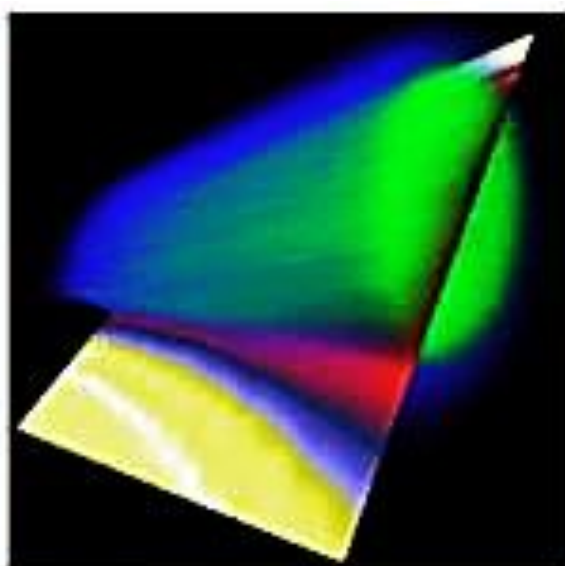
Rush Hour Traffic

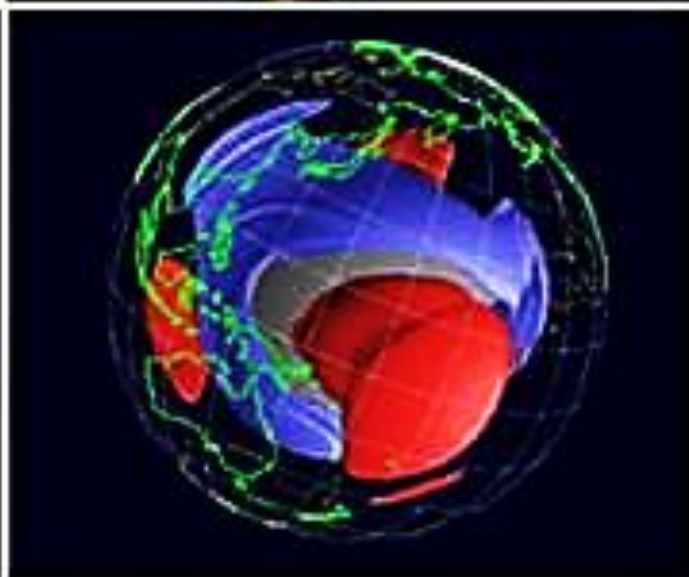
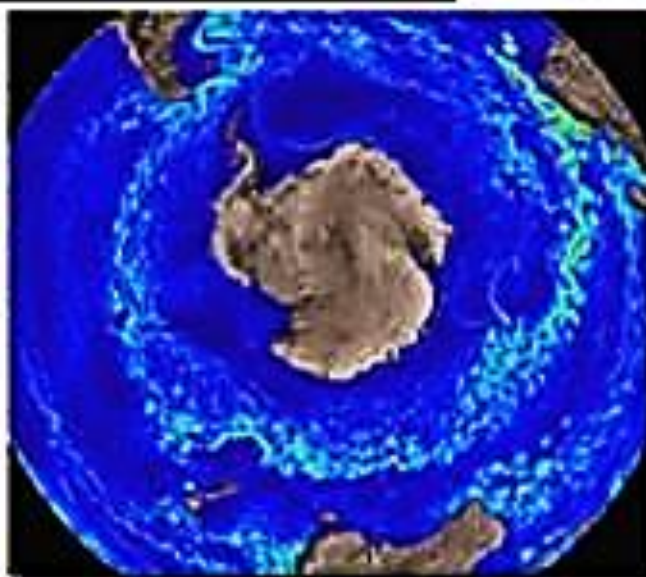
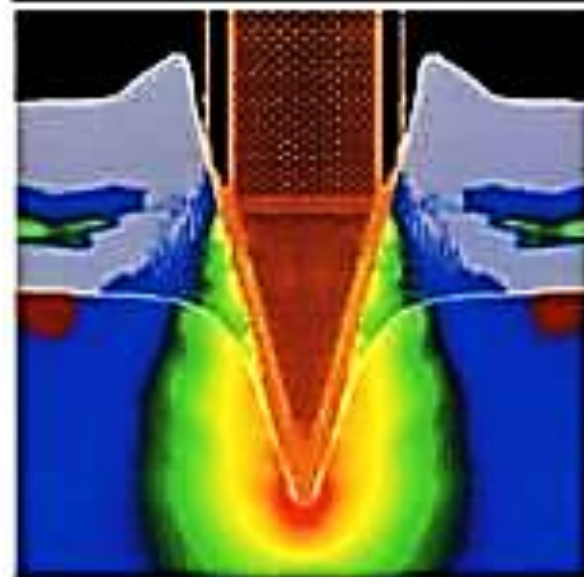
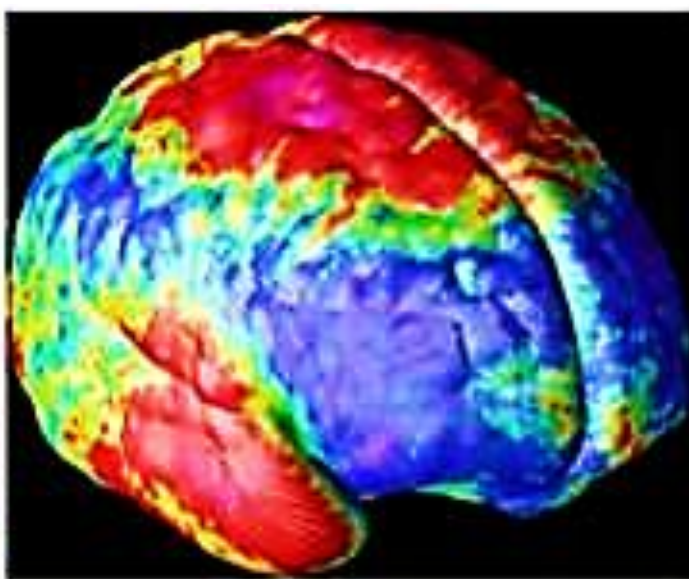
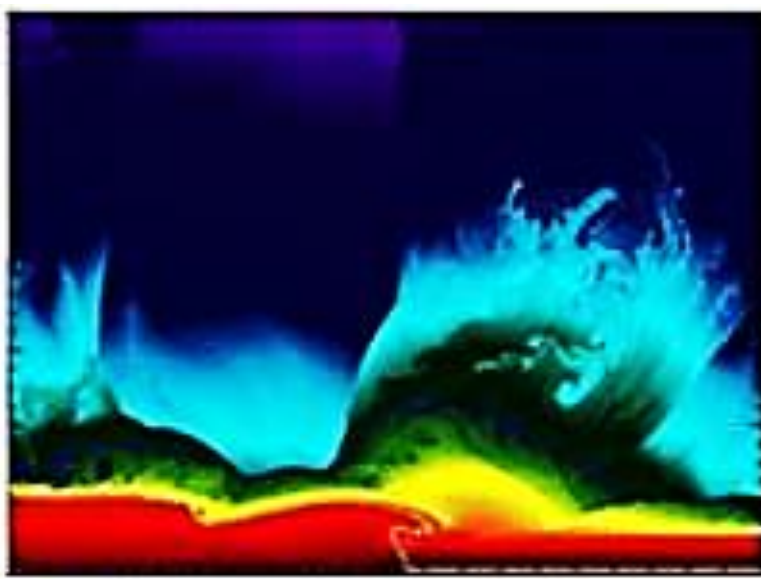
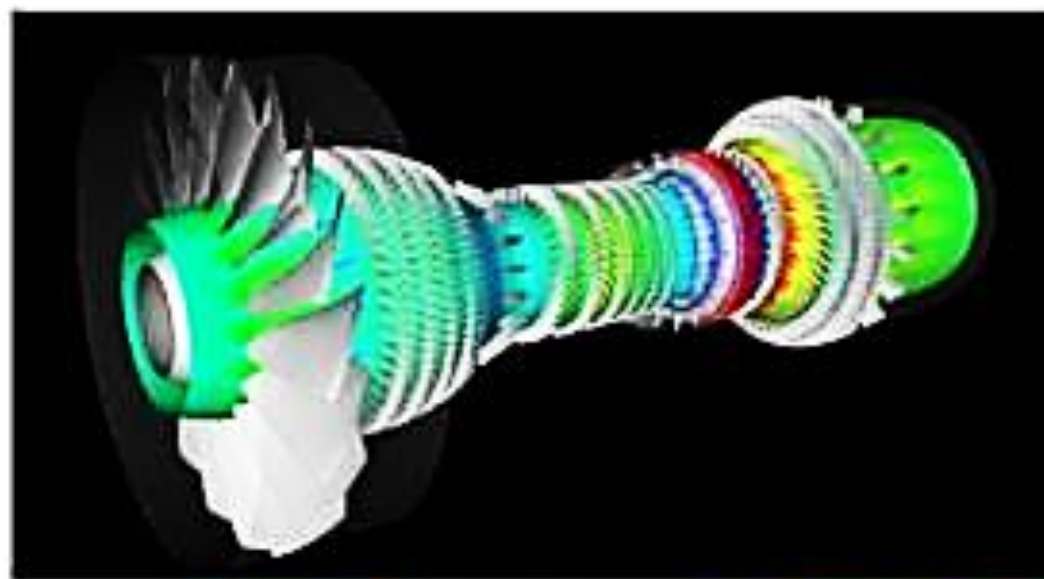


Plate Tectonics



Weather



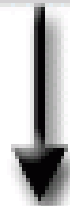
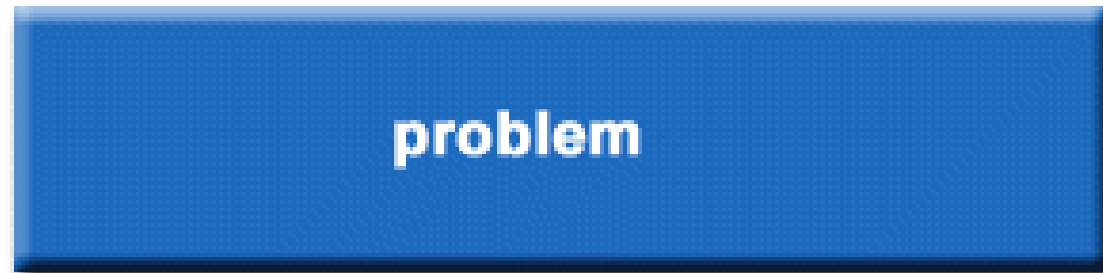


Cómputo serial

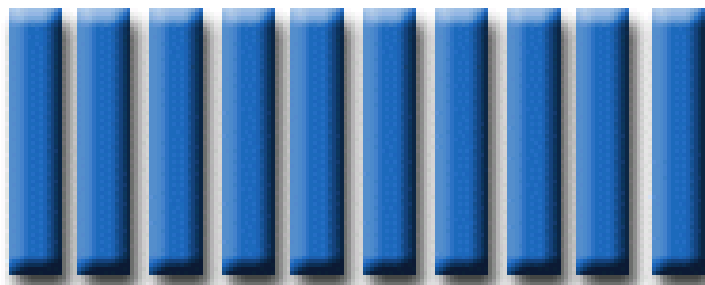
- Programas se han diseñado de forma serial para correr sobre un solo CPU
- La solución a un problema se plantea como una serie de instrucciones discretas que se ejecutan una después de la otra



Cómputo serial



instructions



t_N



t_3



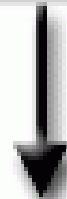
t_2



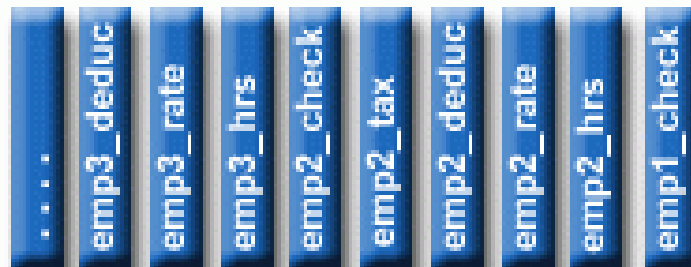
t_1



C mputo serial



instructions



t_N



t_3



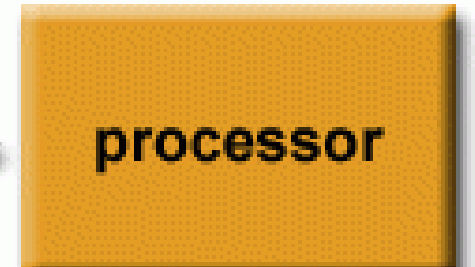
t_2



t_1



processor



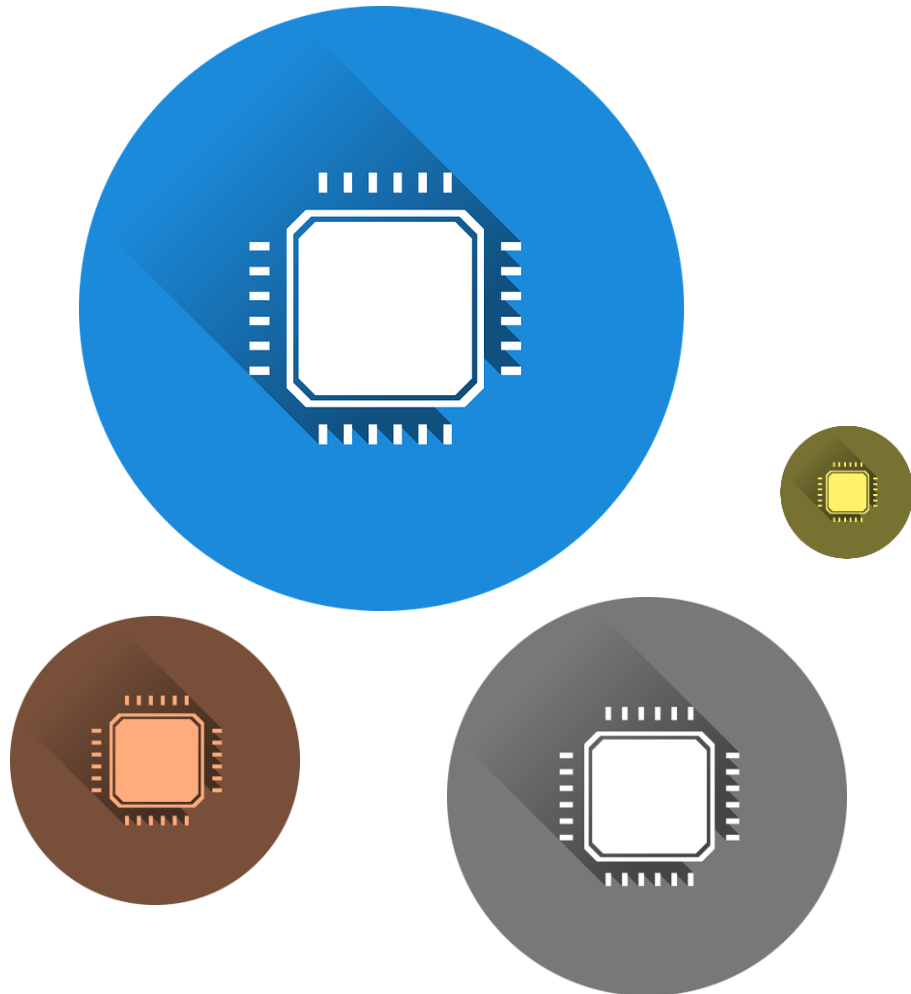
Cómputo Paralelo

1) Uso de una **computadora paralela** con el fin de **reducir el tiempo** necesario para resolver un problema computacional.

2) El cómputo paralelo es el uso simultáneo de **múltiples recursos de cómputo** para resolver un problema



Programación Paralela



Programar en un lenguaje que permita indicar explícitamente **cómo** diferentes porciones de un cálculo pueden ser **ejecutados concurrentemente** por diferentes procesadores.

Concurrencia

vs

Paralelismo

vs

Serial



Concurrencia vs Paralelismo

Tarea A

Tarea B

A.1

B.1

A.2

B.2

A.3

B.3

A.4

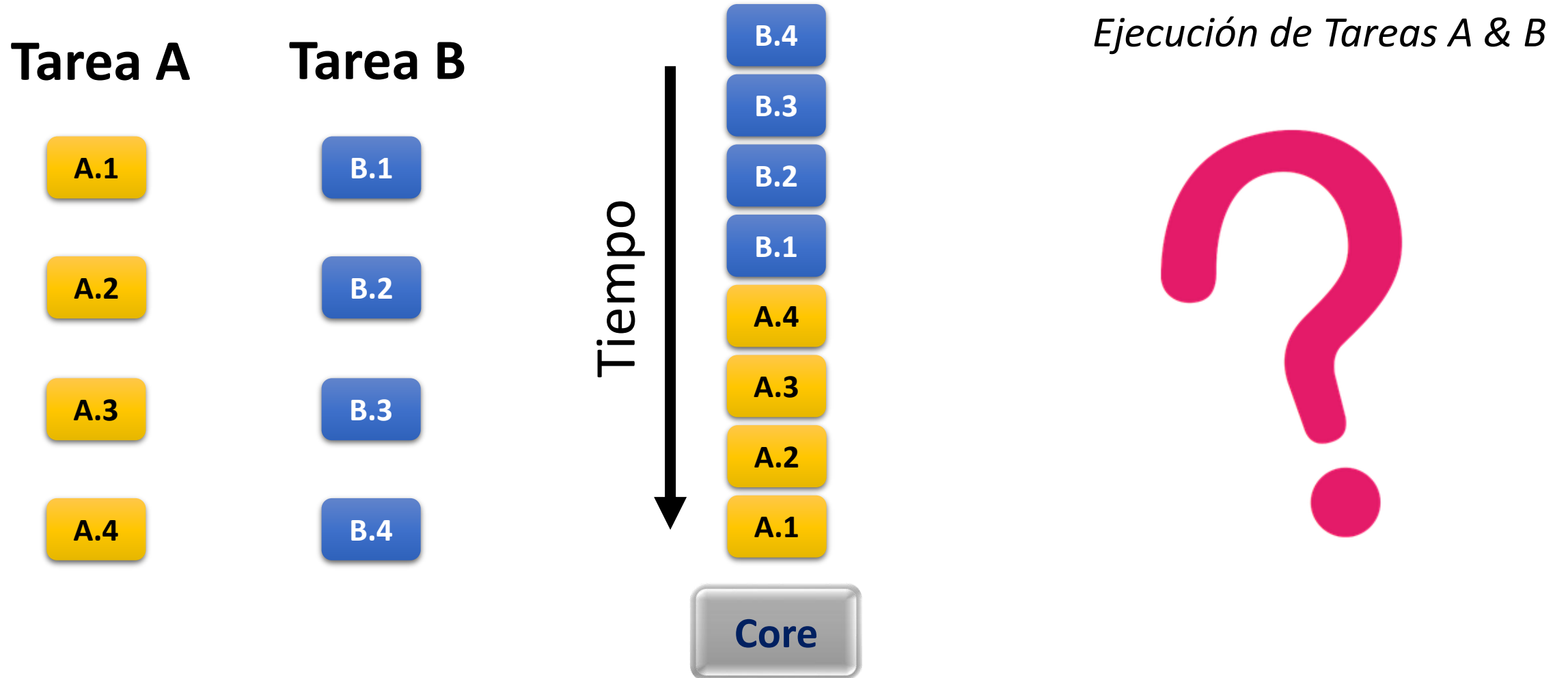
B.4

Tiempo

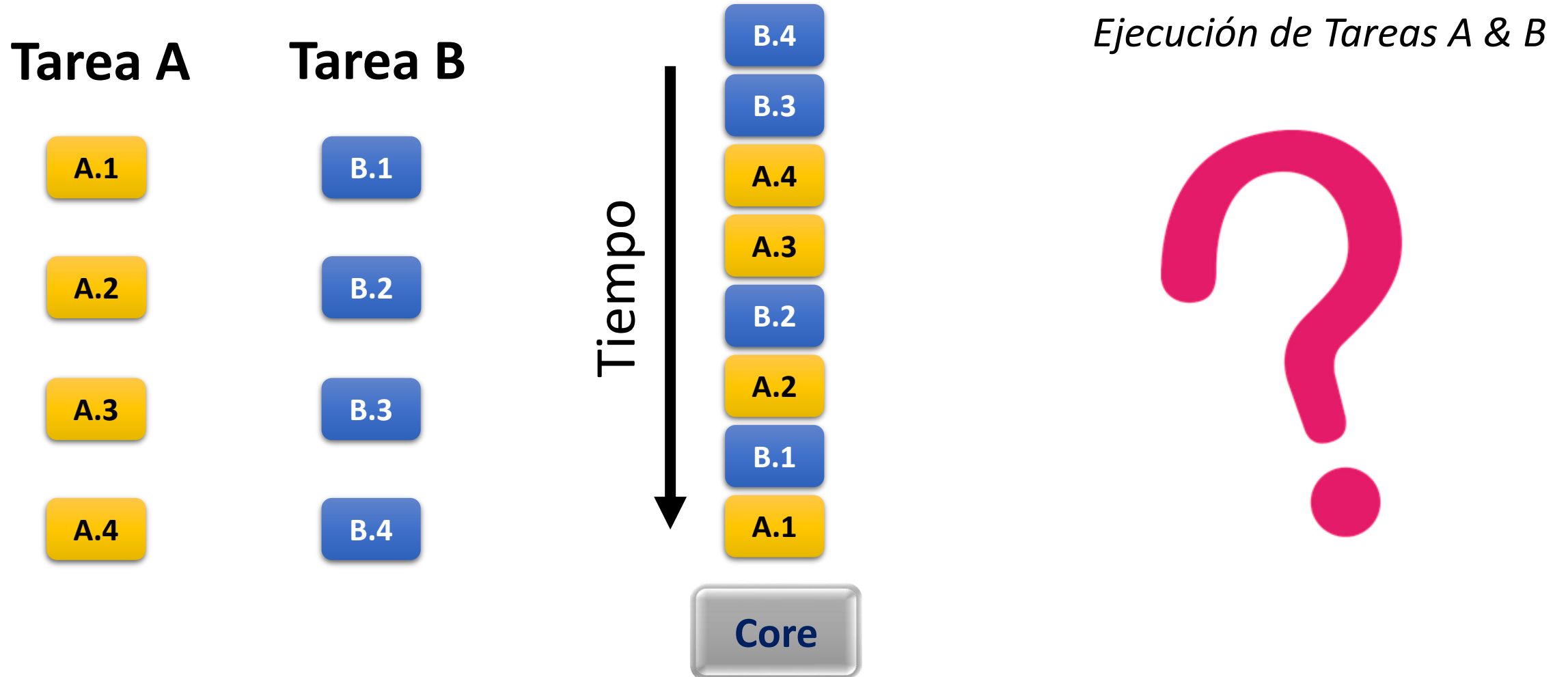
Core



Concurrencia vs Paralelismo 1



Concurrencia vs Paralelismo 2



Concurrencia vs Paralelismo 3

Tarea A

Tarea B

A.1

B.1

A.2

B.2

A.3

B.3

A.4

B.4

Tiempo



A.4

B.4

A.3

B.3

A.2

B.2

A.1

B.1

Core

Core

Ejecución de Tareas A & B



Concurrencia vs Paralelismo 4

Tarea A

Tarea B

A.1

B.1

A.2

B.2

A.3

B.3

A.4

B.4

Tiempo

A.4

B.3

B.2

A.1

Core

B.4

A.3

A.2

B.1

Core

Ejecución de Tareas A & B



Concurrencia vs Paralelismo 5

Tarea A

A.1

A.2

A.3

A.4

Tarea B

B.1

B.2

B.3

B.4

Tiempo



A.4

B.4

A.3

A.2

B.3

B.2

A.1

Core

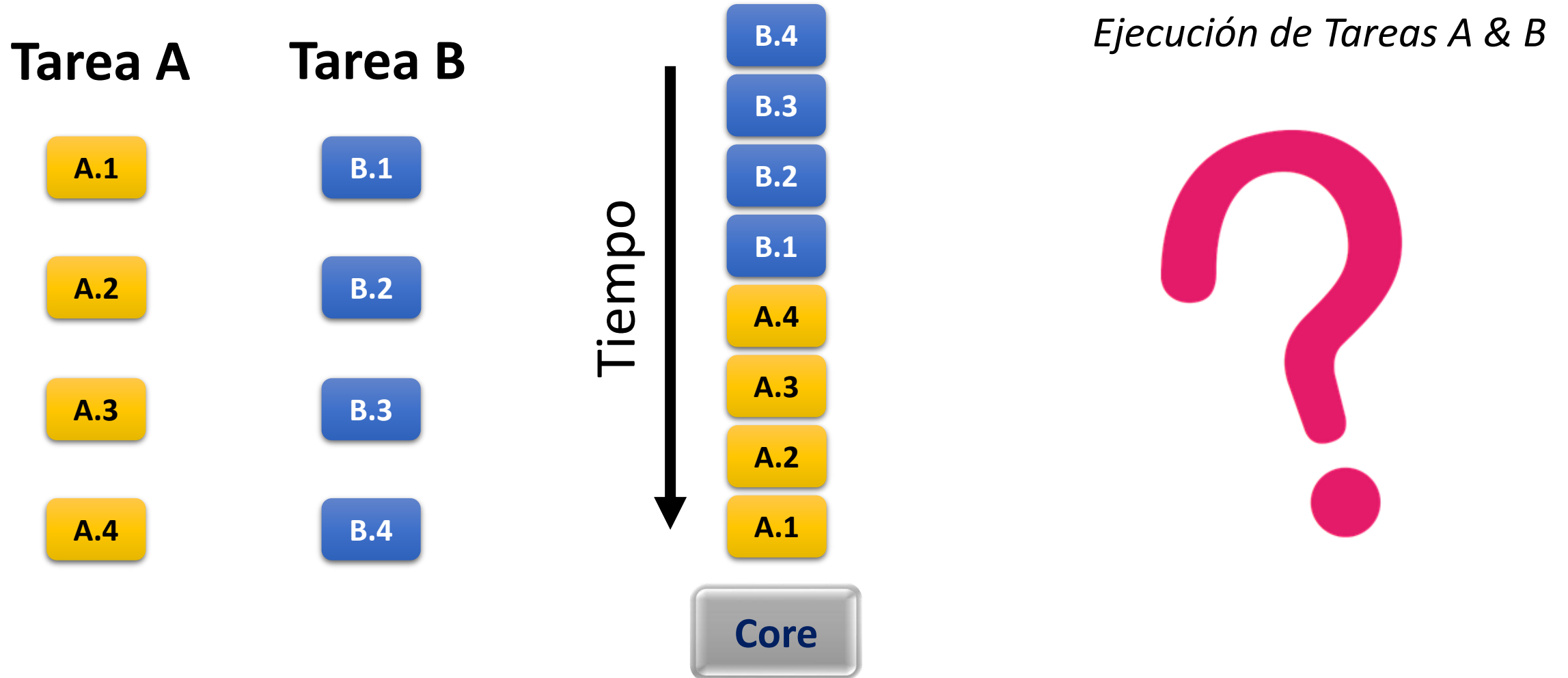
B.1

Core

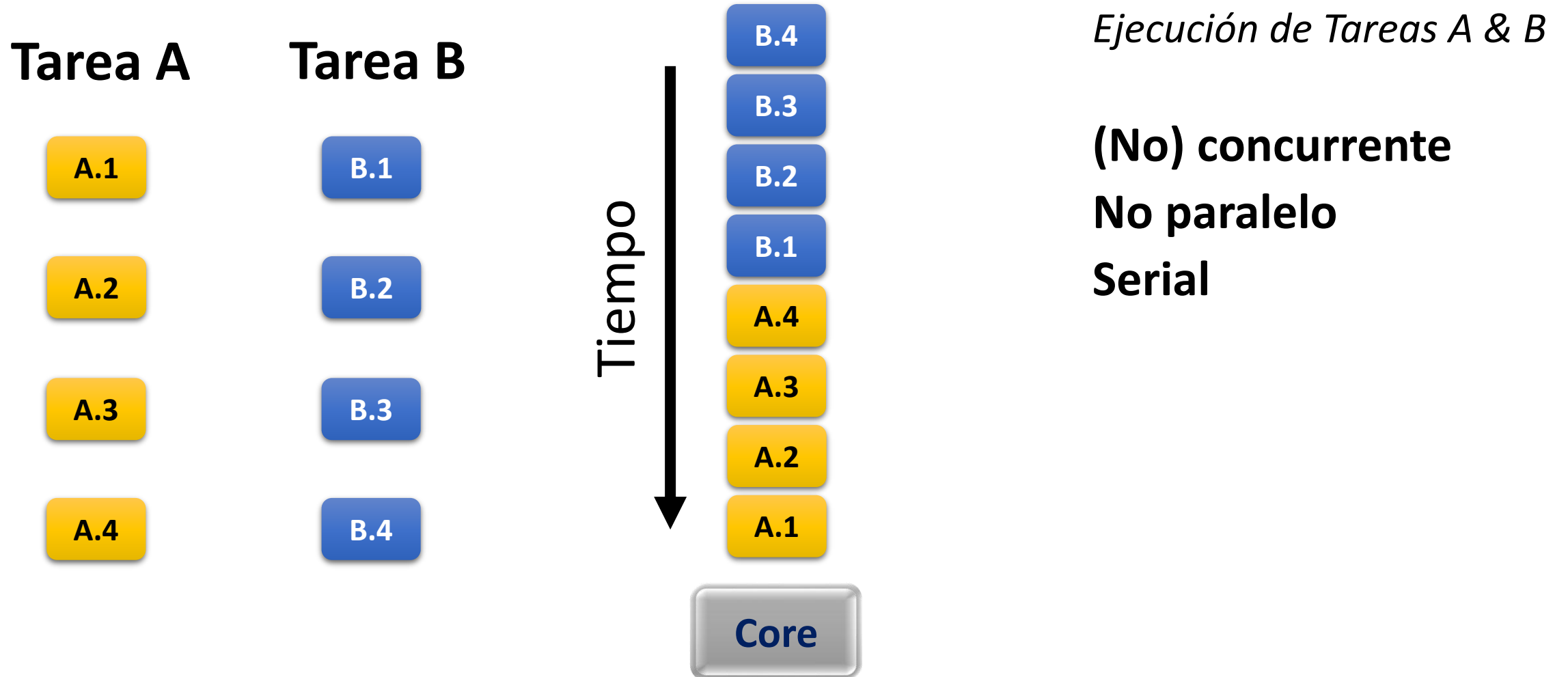
Ejecución de Tareas A & B



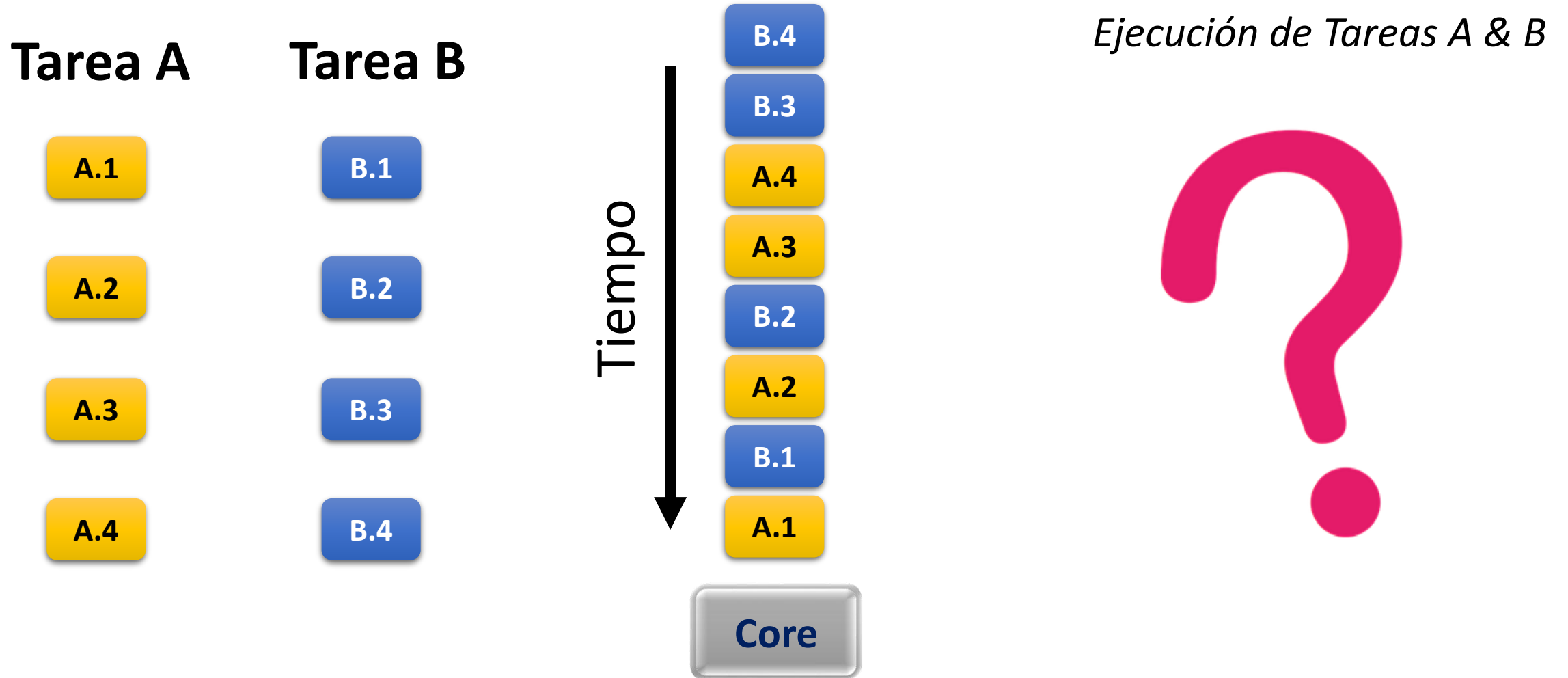
Concurrencia vs Paralelismo 1



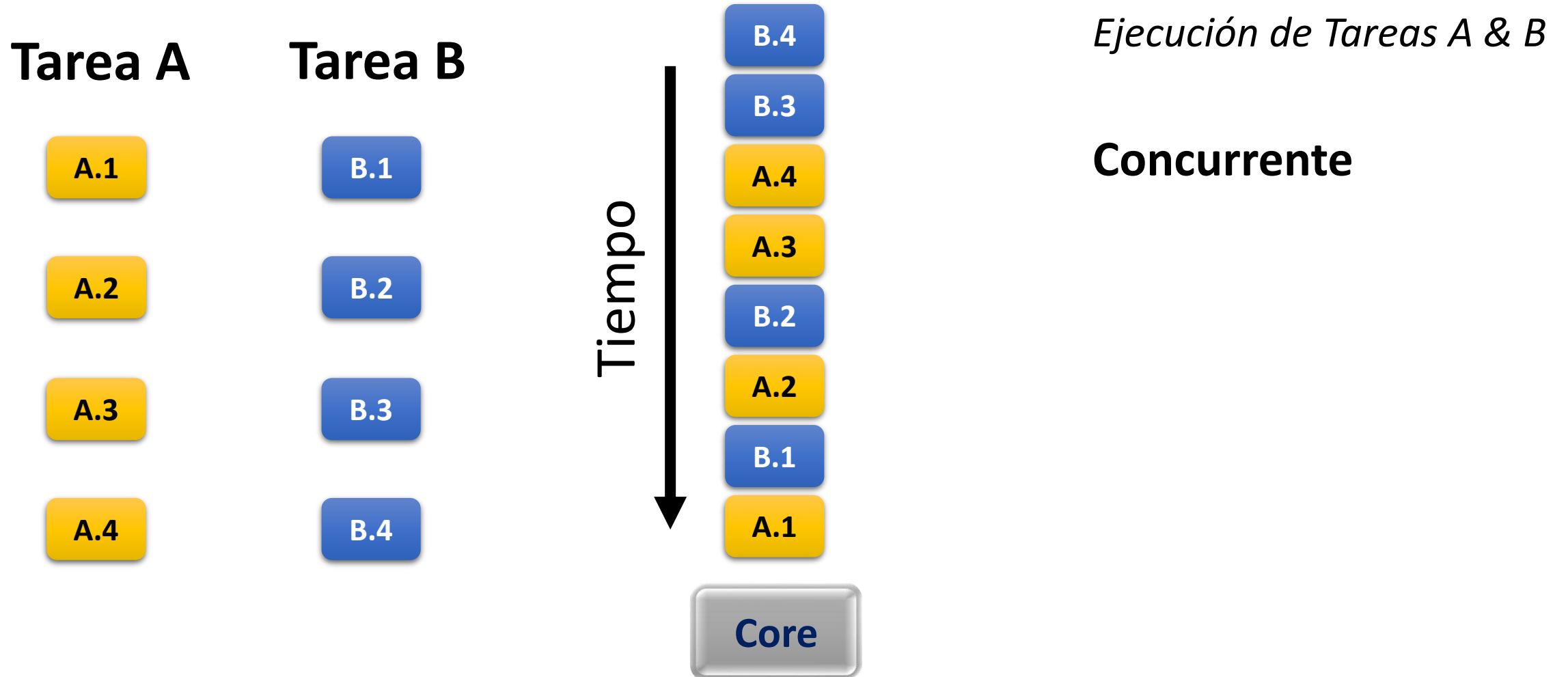
Concurrencia vs Paralelismo 1



Concurrencia vs Paralelismo 2



Concurrencia vs Paralelismo 2



Concurrencia vs Paralelismo 3

Tarea A

Tarea B

A.1

B.1

A.2

B.2

A.3

B.3

A.4

B.4

Tiempo



A.4

B.4

A.3

B.3

A.2

B.2

A.1

B.1

Core

Core

Ejecución de Tareas A & B



Concurrencia vs Paralelismo 3

Tarea A

Tarea B

Ejecución de Tareas A & B

A.1

B.1

A.2

B.2

A.3

B.3

A.4

B.4

Tiempo

A.4

B.4

A.3

B.3

A.2

B.2

A.1

B.1

Core

Core

Concurrente
Paralelo

Concurrencia vs Paralelismo 4

Tarea A

Tarea B

A.1

B.1

A.2

B.2

A.3

B.3

A.4

B.4

Tiempo



A.4

B.3

B.2

A.1

Core

B.4

A.3

A.2

B.1

Core

Ejecución de Tareas A & B



Concurrencia vs Paralelismo 4

Tarea A

Tarea B

A.1

B.1

A.2

B.2

A.3

B.3

A.4

B.4

Tiempo



A.4

B.4

B.3

A.3

B.2

A.2

A.1

B.1

Core

Core

Ejecución de Tareas A & B

Concurrente
Paralelo

Concurrencia vs Paralelismo 5

Tarea A

A.1

A.2

A.3

A.4

Tarea B

B.1

B.2

B.3

B.4

Tiempo



A.4

B.4

A.3

A.2

B.3

B.2

A.1

Core

B.1

Core

Ejecución de Tareas A & B



Concurrencia vs Paralelismo 5

Tarea A

Tarea B

A.1

B.1

A.2

B.2

A.3

B.3

A.4

B.4

Tiempo

A.4

B.4

A.3

A.2

B.3

B.2

A.1

B.1

Core

Core

Ejecución de Tareas A & B

Concurrente
Paralelo

¿Cómo se paraleliza?



- Un **problema se divide** en partes discretas que se pueden resolver concurrentemente
- Cada parte discreta se divide en una **serie de instrucciones**.
- Las instrucciones de cada parte se **ejecutan simultáneamente** en diferentes procesadores
- Se emplea un mecanismo general de control/**coordinación**.
- La **granularidad** es importante

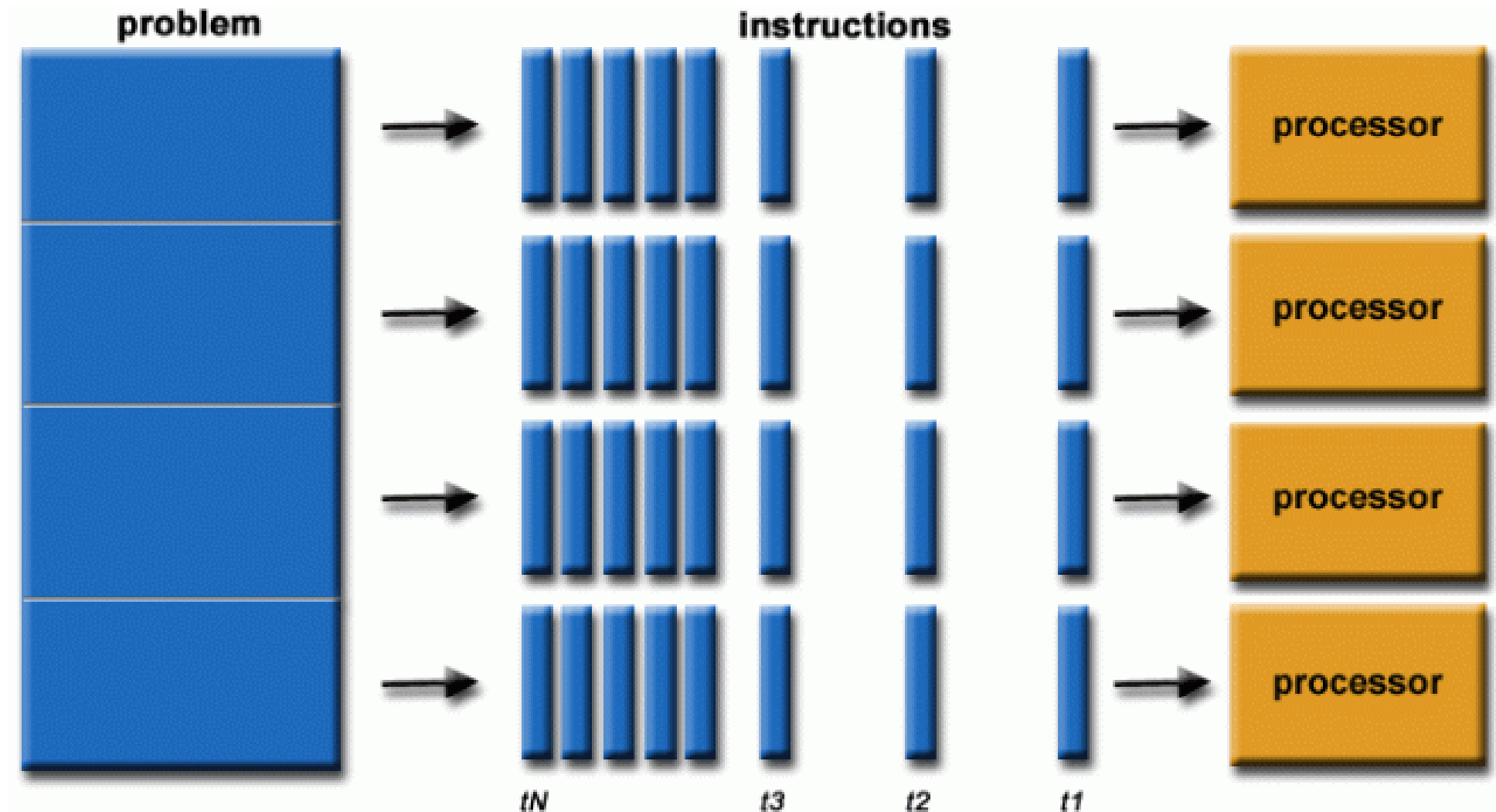
Ejecución paralela de procesos

- Ejecutar un proceso en un subconjunto de cores

`taskset -c 0,2,4,6 proceso`

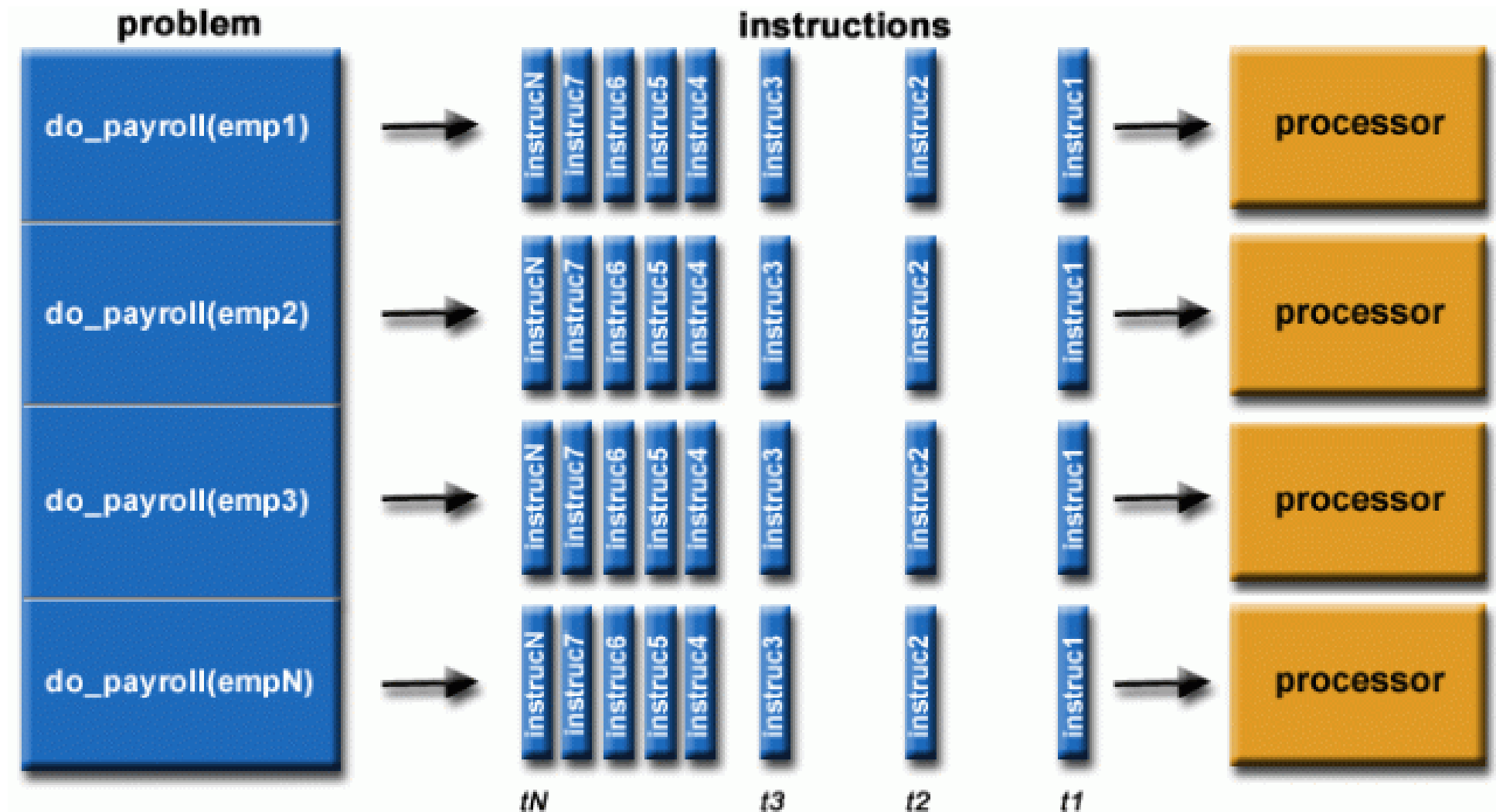
- Trade-off: rendimiento vs conveniencia

C  mputo paralelo





Cómputo paralelo



¿Qué se requiere para paralelizar?



- Descomposición en partes
- Resolver/determinar restricciones de orden
- Asignar trabajos a los procesadores

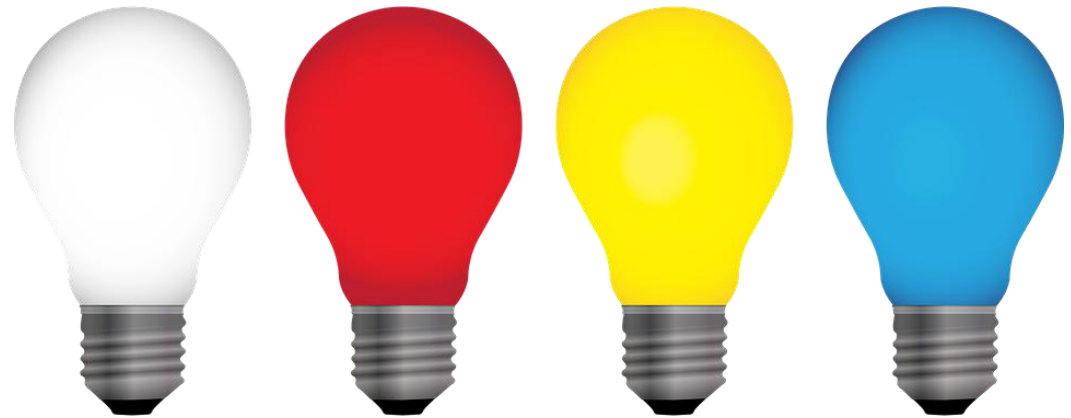
¿Qué se requiere para paralelizar?



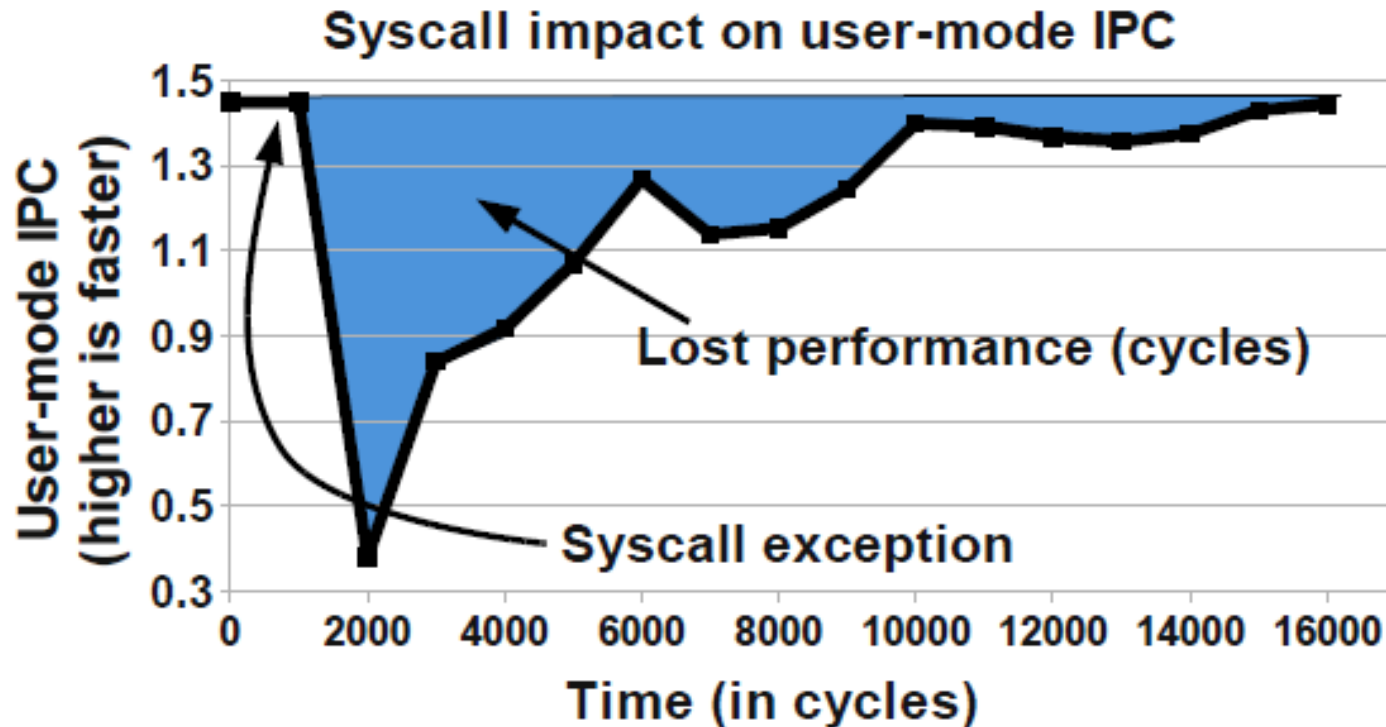
- Sincronización/comunicación entre los procesadores para no limitar su velocidad
- Lenguajes de programación para paralelismo
- Que se resuelva en menor tiempo en múltiples recursos:
 - Computadora multi-core
 - Computadoras en red

Algunos conceptos relevantes para entender el paralelismo

- Registros
- Procesos
- Hilos
 - Usuario
 - Kernel
- Contaminación de caches e Interrupciones

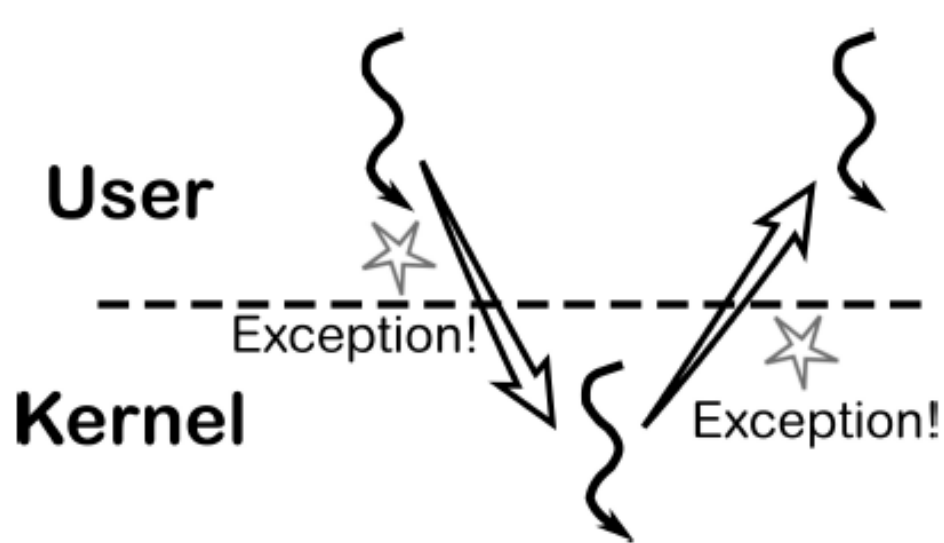


Interrupciones y sobrecarga [1/2]



User-mode instructions per cycles (IPC) of Xalan (from SPEC CPU 2006) in response to a system call exception event, as measured on an Intel Core i7 processor.

Interrupciones y sobrecarga [1/2]



Expensive! Costs are:

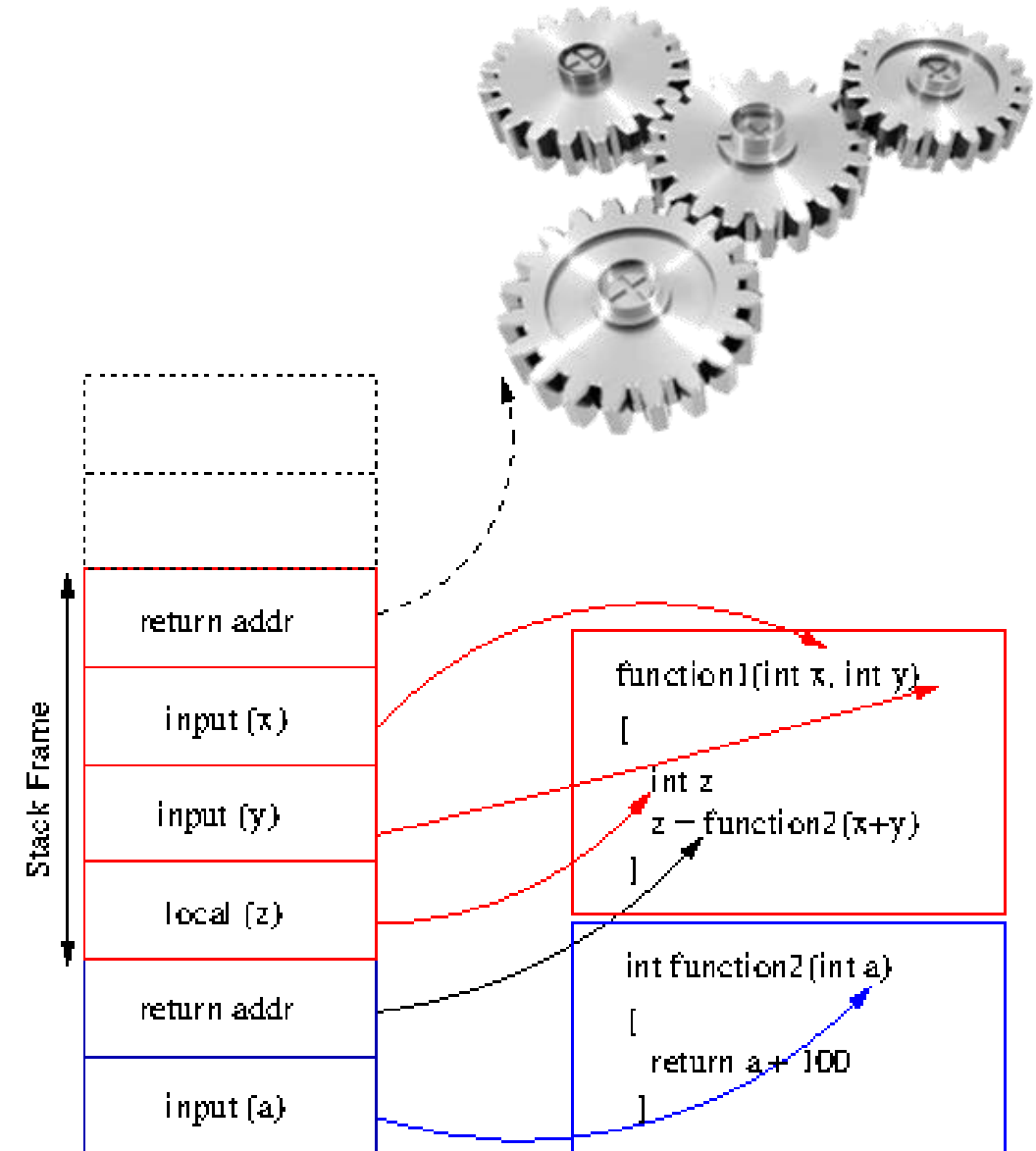
- **direct**: mode-switch
- **indirect**: processor structure pollution

Flushes the processor pipeline

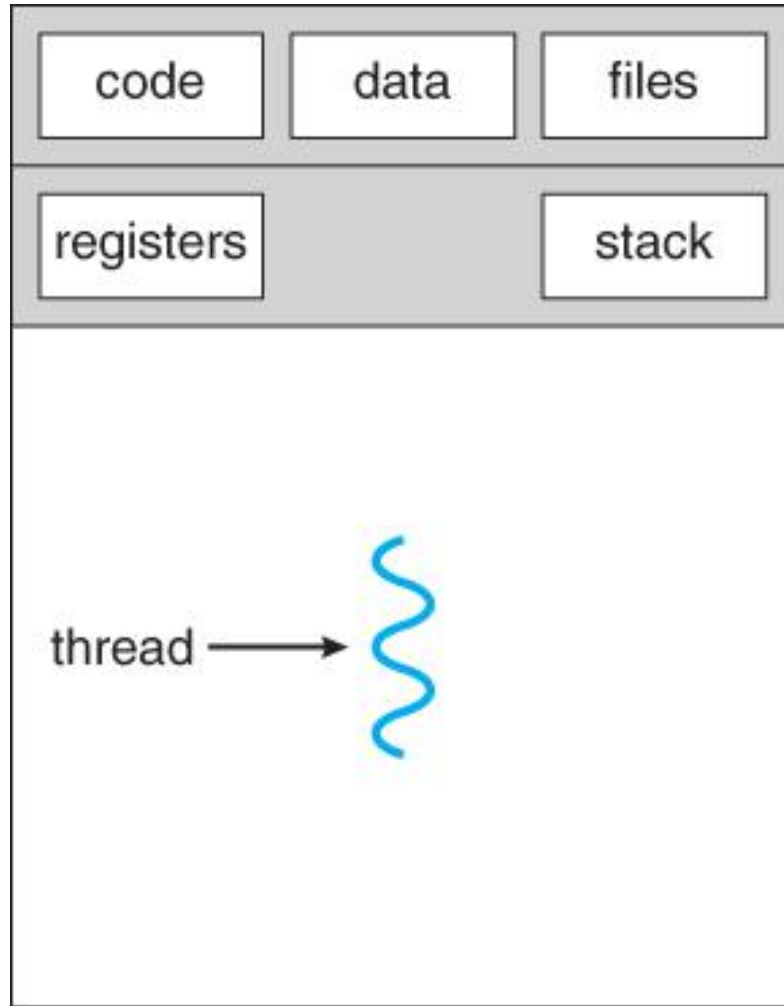
e.g., caches and TLBs

¿Qué es un proceso?

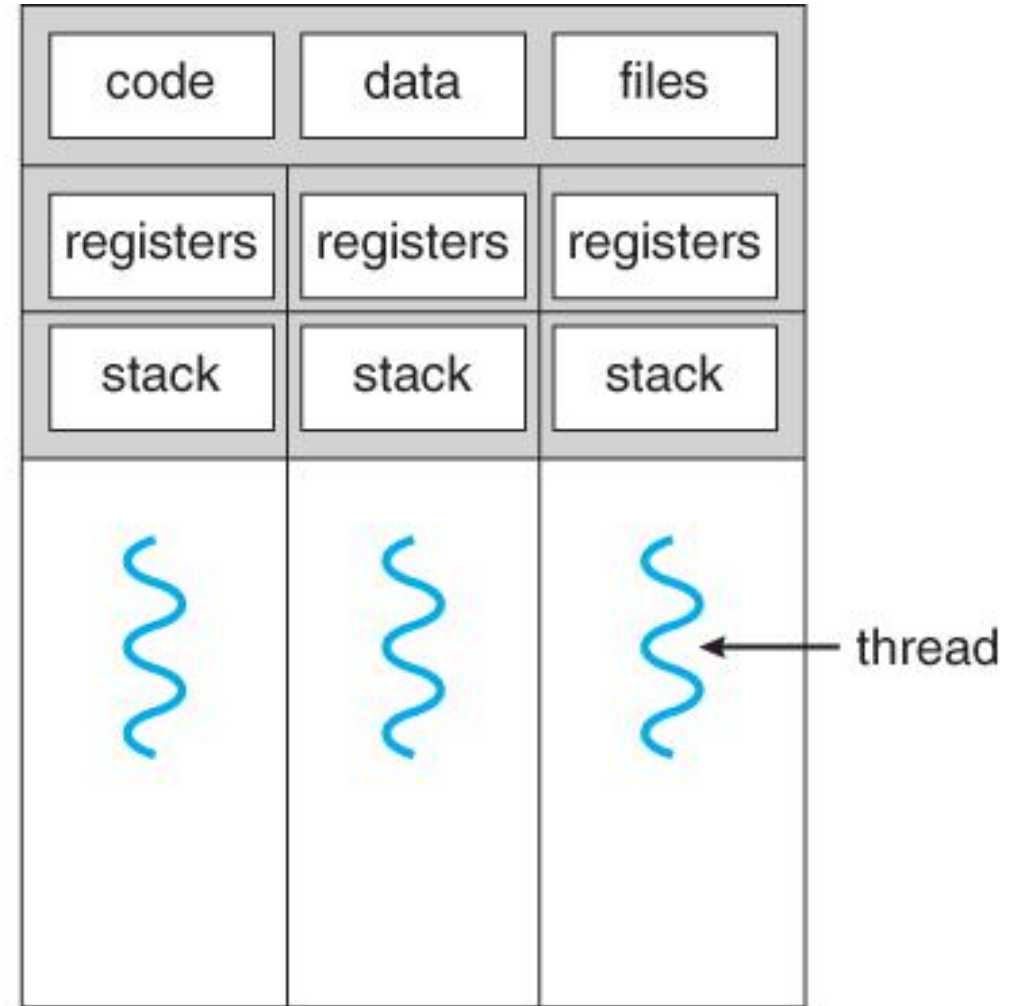
- Recipiente que contiene toda la información necesaria para ejecutar un **programa**
 - Espacio asociado de **direcciones en memoria** con programa ejecutable, datos y pila.
- También **tiene** asociado un conjunto de **recursos**:
 - Registros de sistema (e.g., contadores)
 - Archivos abiertos
 - Alarmas
 - etc.



Procesos e hilos

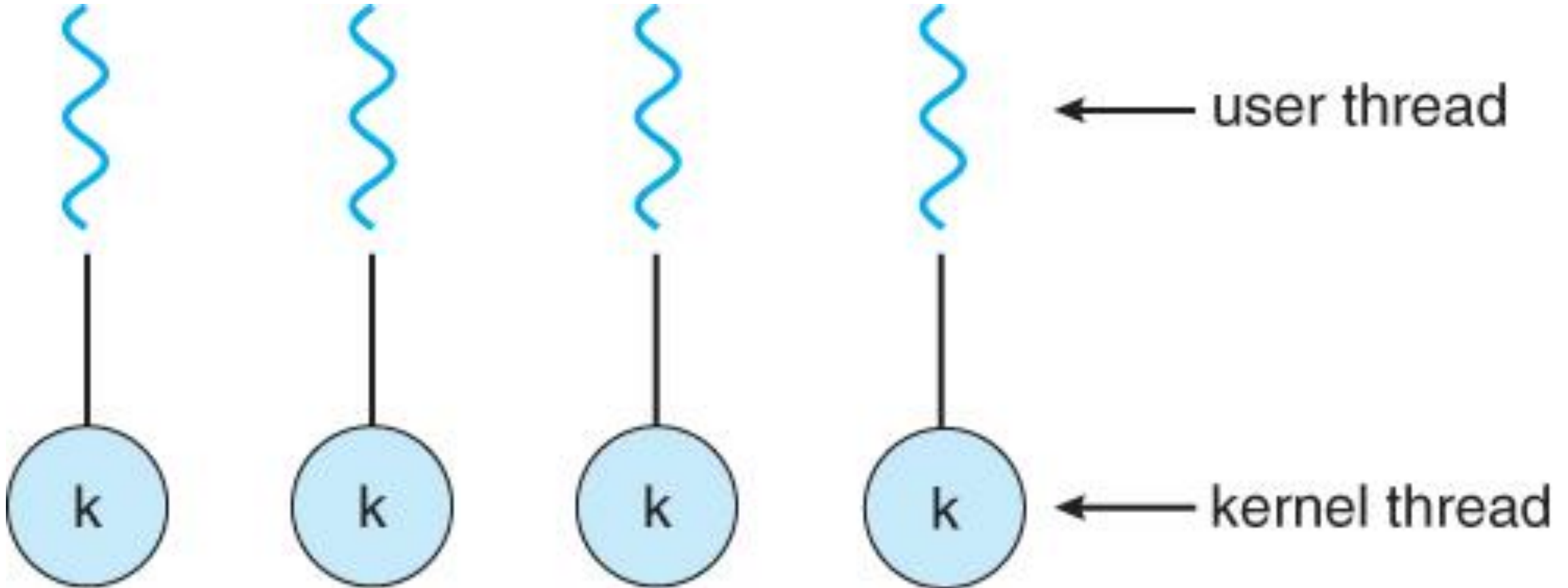


single-threaded process

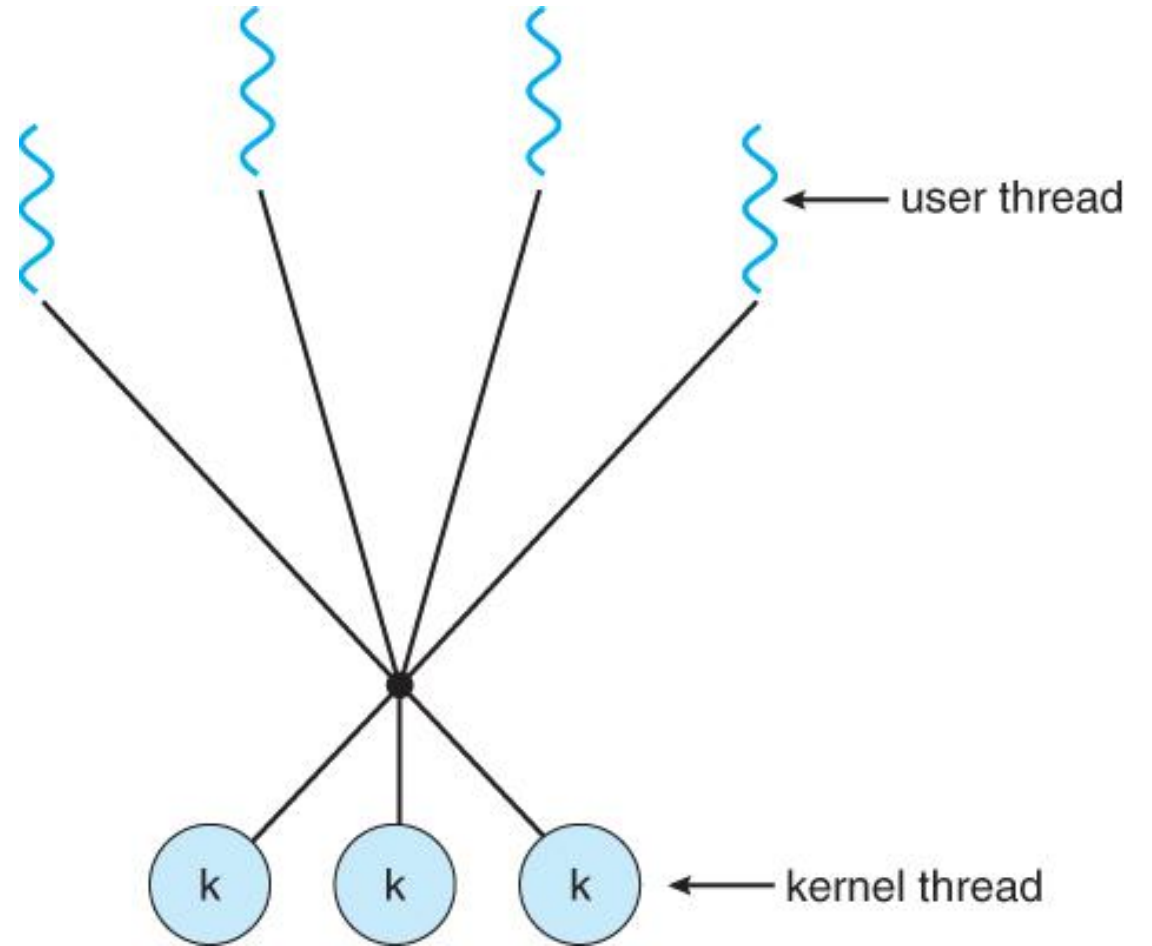
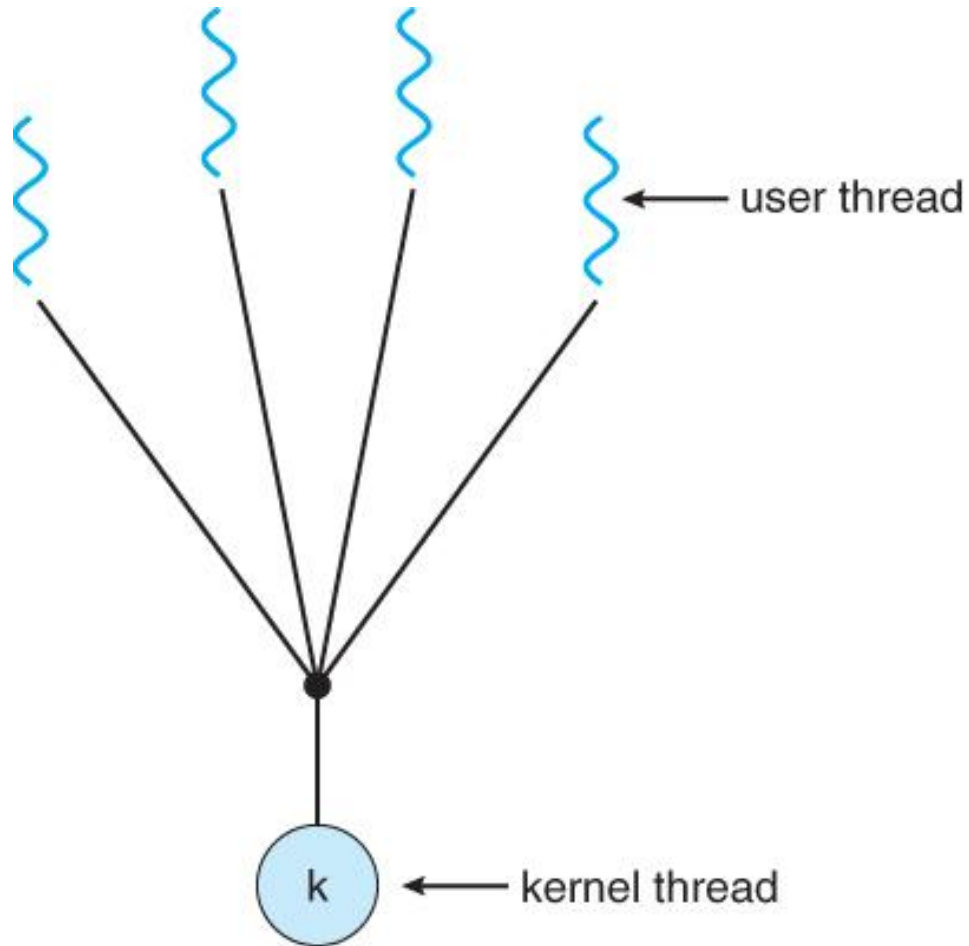


multithreaded process

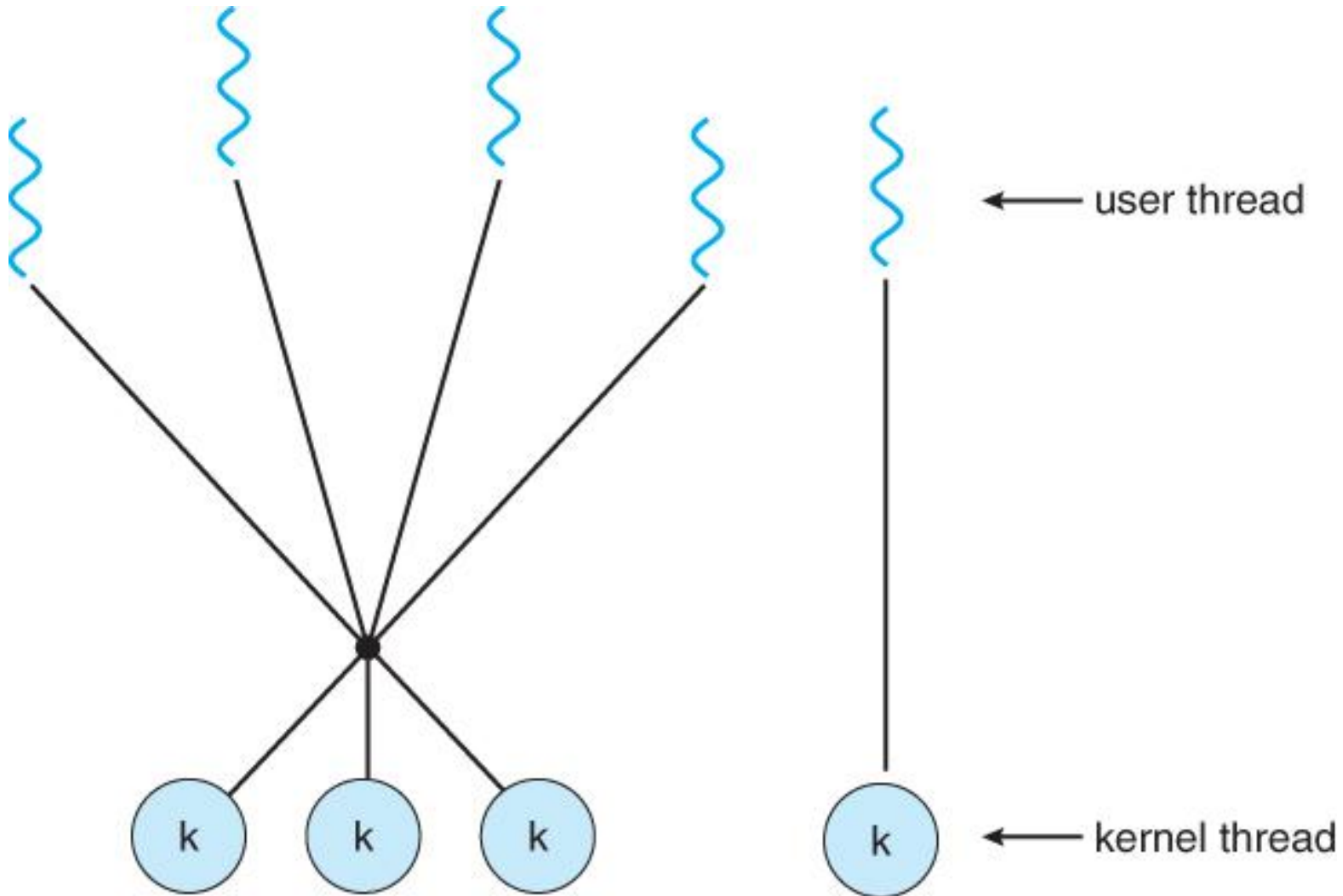
Hilos de Usuario & Hilos de Kernel



Hilos de Usuario & Hilos de Kernel



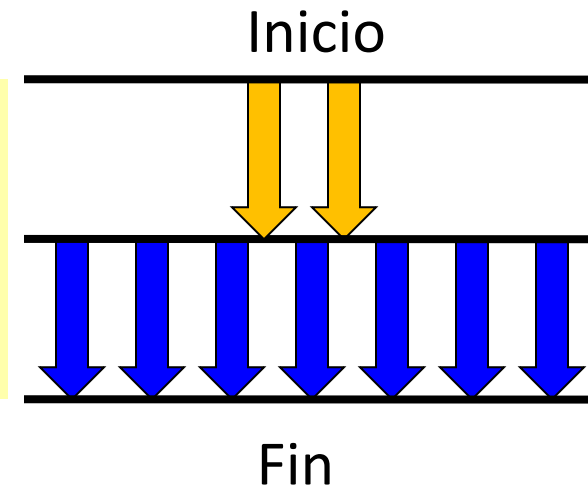
Hilos de Usuario & Hilos de Kernel



Ejemplo abstracto: Restricciones de orden

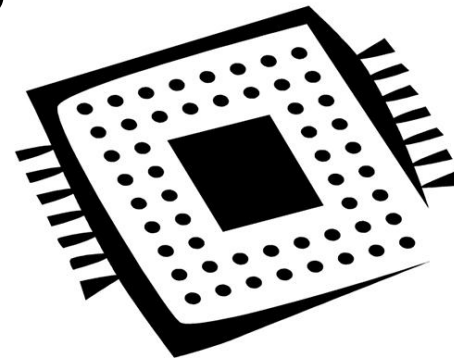
```
void parallel_task(float *a, float *b, float *c)
{
    int n=7; int i=0;

    parallel_for(; i< n; i++)
        c[i] = a[i] + b[i];
}
```

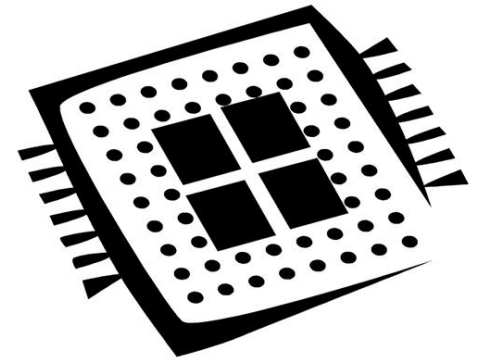


Terminología de Paralelismo

- Los **CPUs/Procesadores** contemporáneos constan de uno o más núcleos
- **Núcleo**: una unidad de ejecución distinta con su propio flujo de instrucciones.



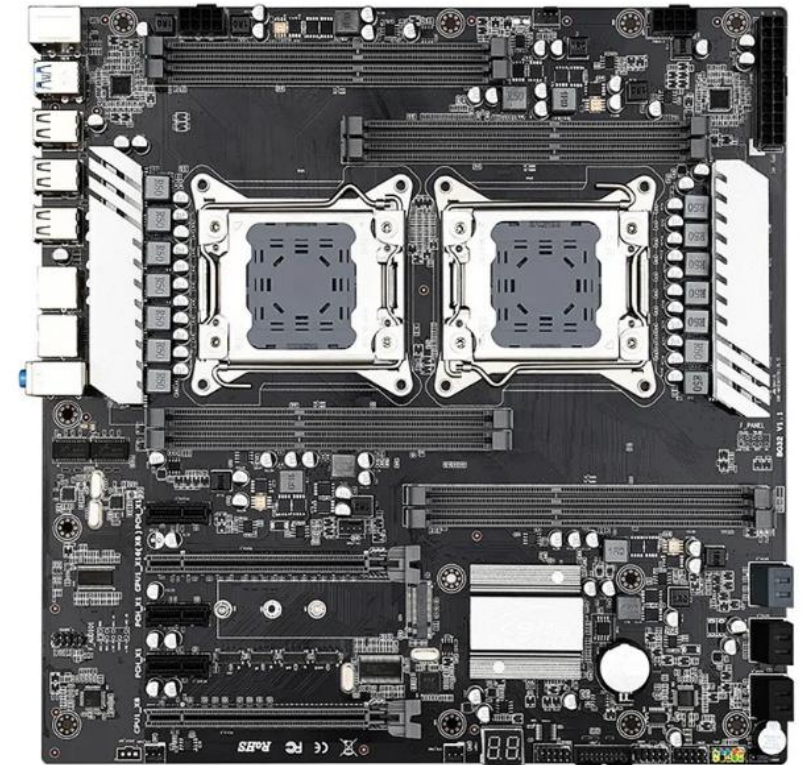
Single-core CPU



Multi-core CPU

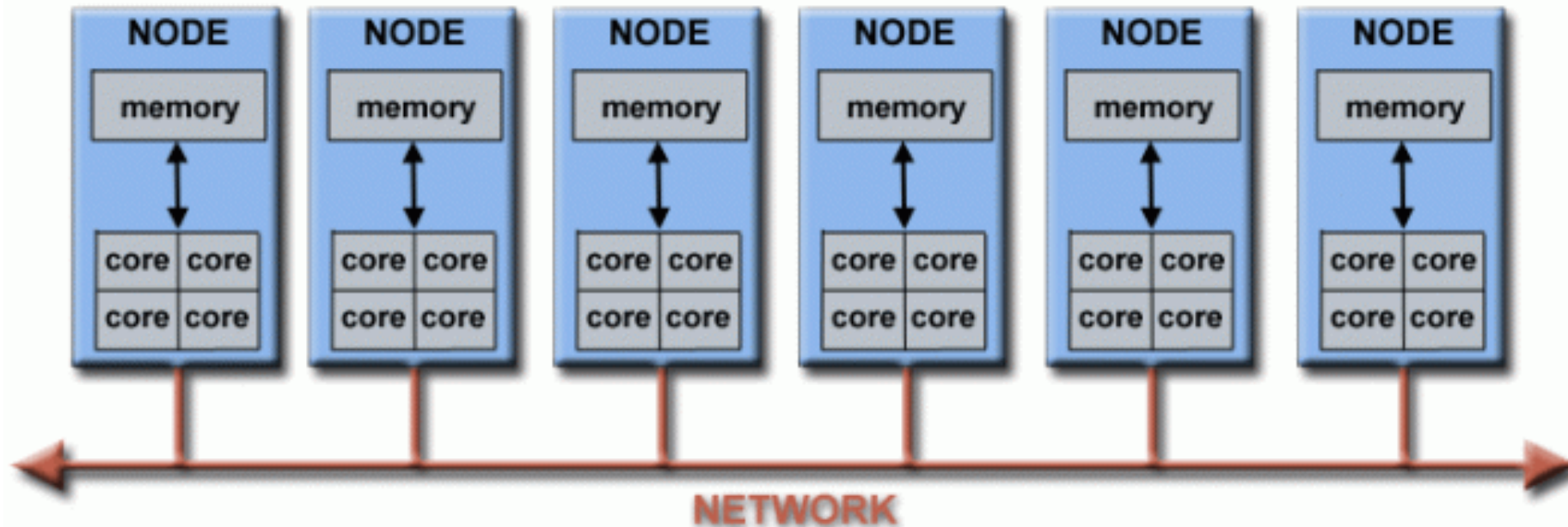
Terminología de Paralelismo

- Los núcleos de una CPU se pueden organizar en **uno o más zócalos**, cada zócalo con su propia memoria distinta.
- Cuando una CPU consta de dos o más zócalos, normalmente la infraestructura de hardware admite el **uso compartido de memoria** entre zócalos.



Terminología de Paralelismo

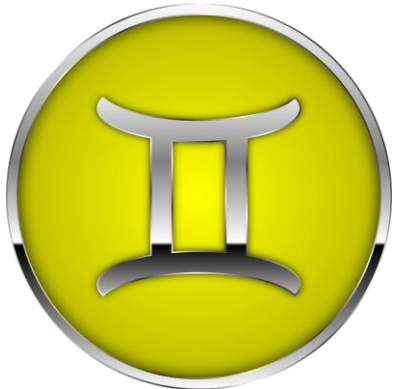
- **Nodo:** Una "computadora en una caja" independiente. Por lo general, se compone de múltiples CPU/núcleos, memoria, interfaces de red, etc.



Terminología de Paralelismo



- **Tarea:** Una sección lógicamente discreta de trabajo computacional. Una tarea suele ser un programa o un conjunto de instrucciones similar a un programa que ejecuta un procesador. Un programa paralelo consta de varias tareas que se ejecutan en varios procesadores.



- **Multiprocesador simétrico (SMP):** Arquitectura de hardware de memoria compartida en la que varios procesadores comparten un único espacio de direcciones y tienen el mismo acceso a todos los recursos: memoria, disco, etc.

Terminología de Paralelismo

- **Memoria compartida:** arquitectura de computadora donde todos los procesadores tienen acceso directo a la memoria física común.

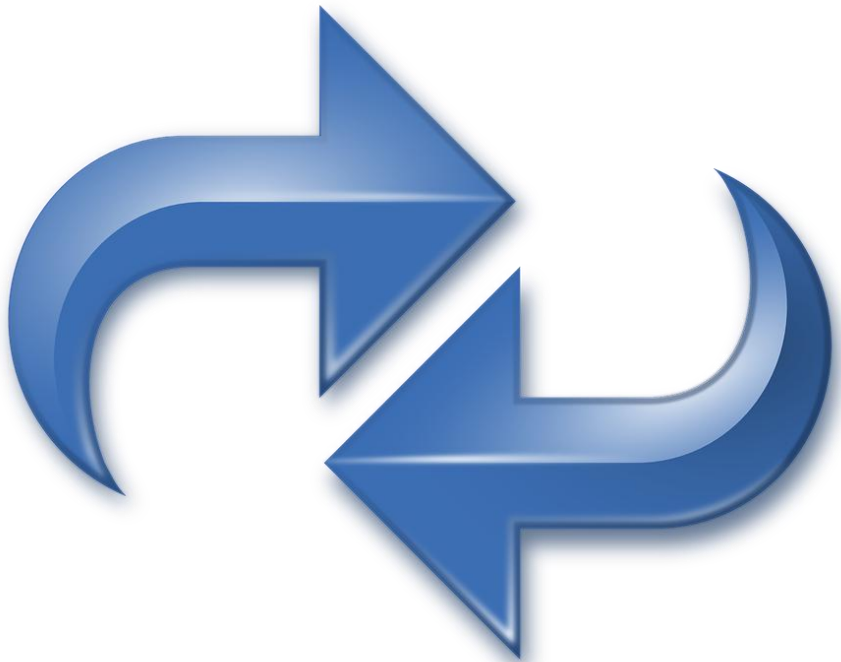
En un sentido de programación, describe un modelo en el que todas las tareas paralelas tienen la misma "imagen" de la memoria y pueden abordar y acceder directamente a las mismas ubicaciones de memoria lógica.

- **Memoria Distribuida:** acceso basado en red a memoria física no común.

Como modelo de programación, las tareas solo pueden "ver" lógicamente la memoria de la máquina local y deben usar las comunicaciones para acceder a la memoria en otras máquinas.



Terminología de Paralelismo



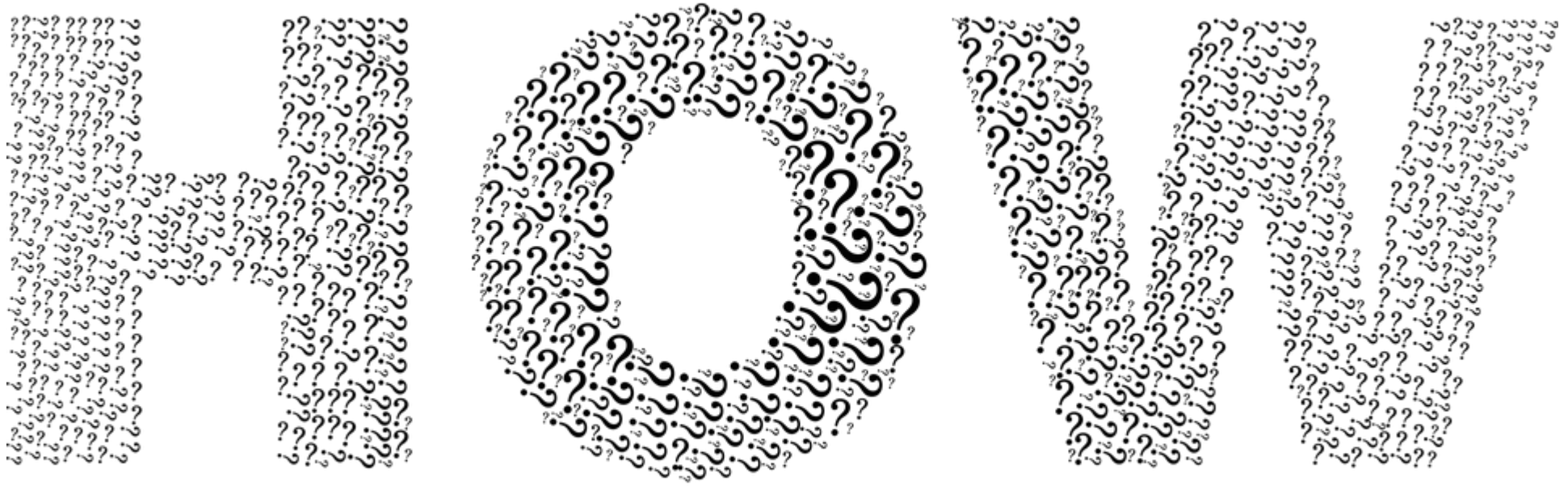
- **Comunicaciones:** Las tareas paralelas normalmente necesitan intercambiar datos, e.g., a través de un bus, de memoria compartida o a través de una red.
- **Sincronización:** Coordinación de tareas paralelas. La sincronización generalmente implica esperar al menos una tarea y, por lo tanto, puede hacer que aumente el tiempo de ejecución del reloj de pared de una aplicación paralela.

Terminología de Paralelismo

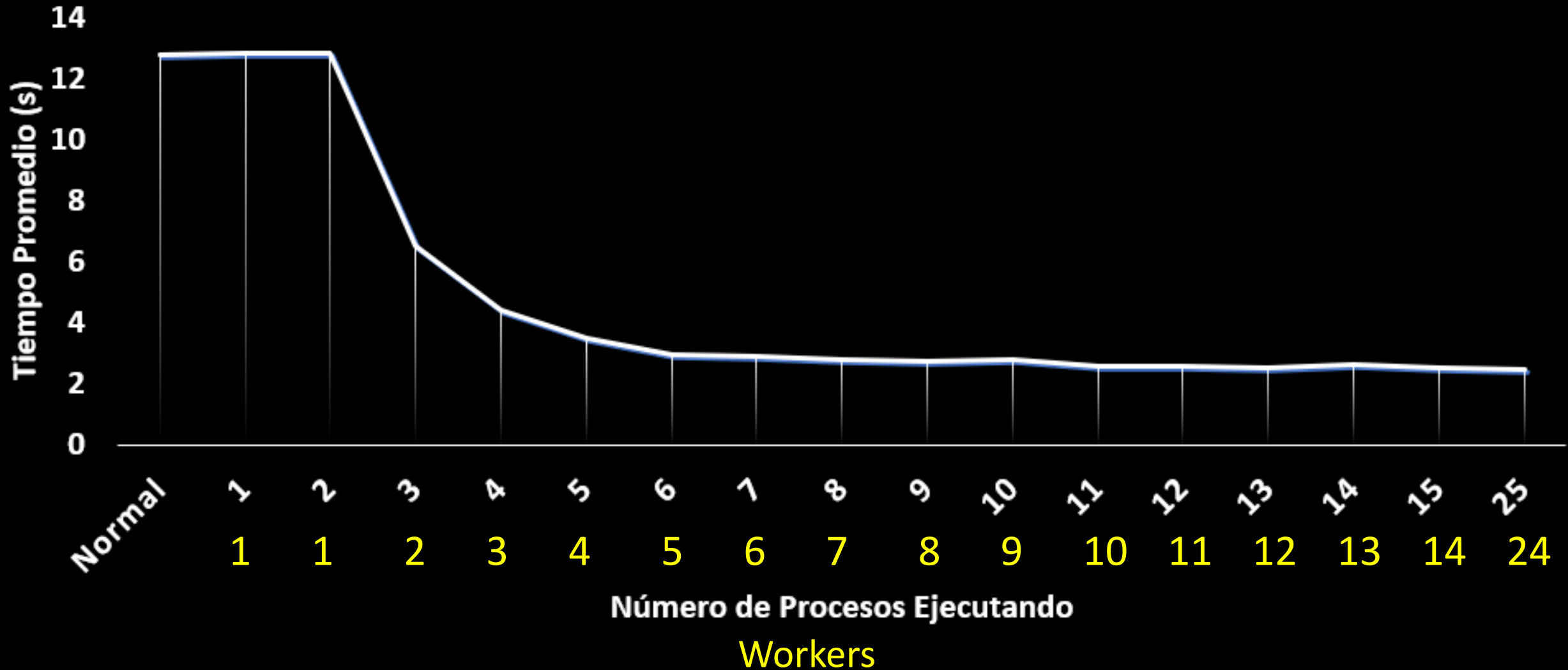
- **Granularidad computacional:** Es una medida cuantitativa o cualitativa de la relación entre cómputo y comunicación.
- **Gruesa:** se realizan cantidades relativamente grandes de trabajo computacional entre eventos de comunicación.
- **Fina:** se realizan cantidades relativamente pequeñas de trabajo computacional entre eventos de comunicación.



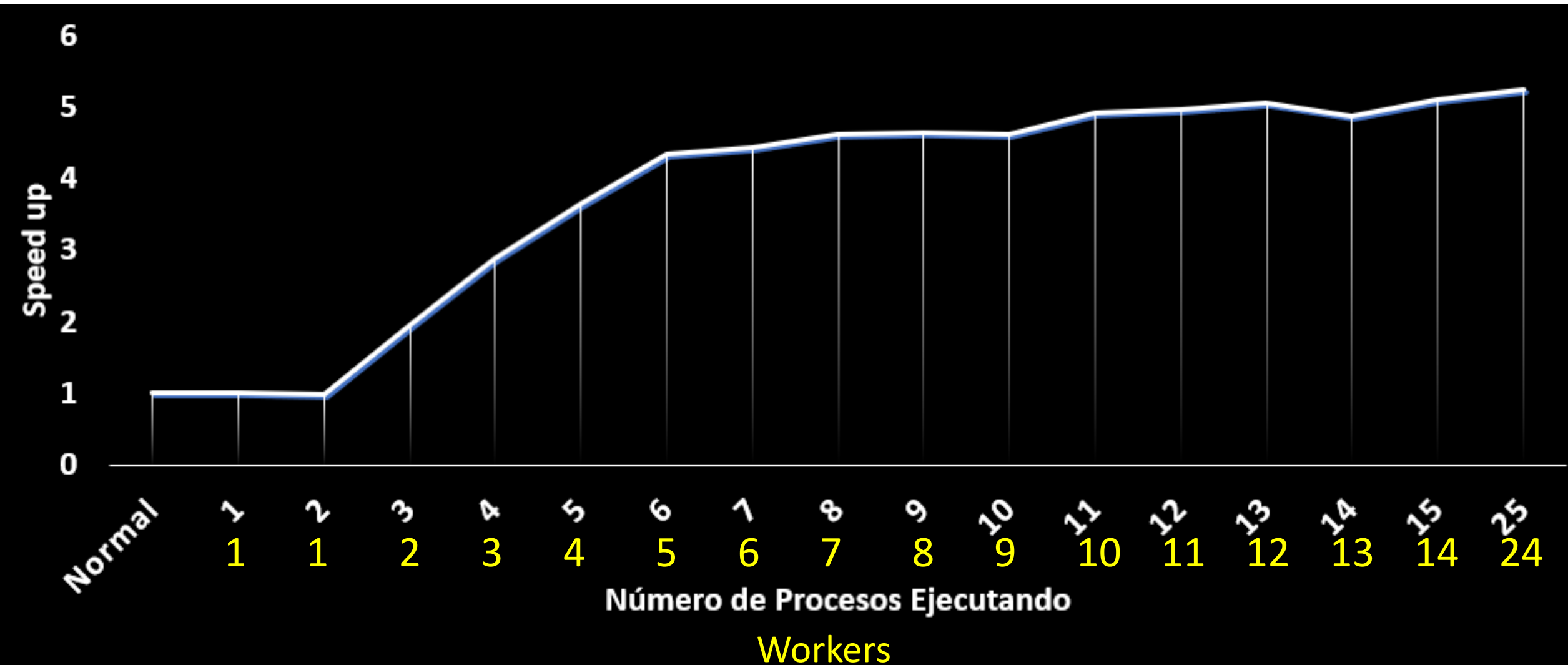
¿Cómo medir qué tan bien se paraleliza?



Tiempo ejecución



Aceleración de ejecución (speed up)



Aceleración de ejecución (speed up)

¿Un speed up de 5x es un buen resultado?



Speed up simplista – Línea base

Para un problema dado



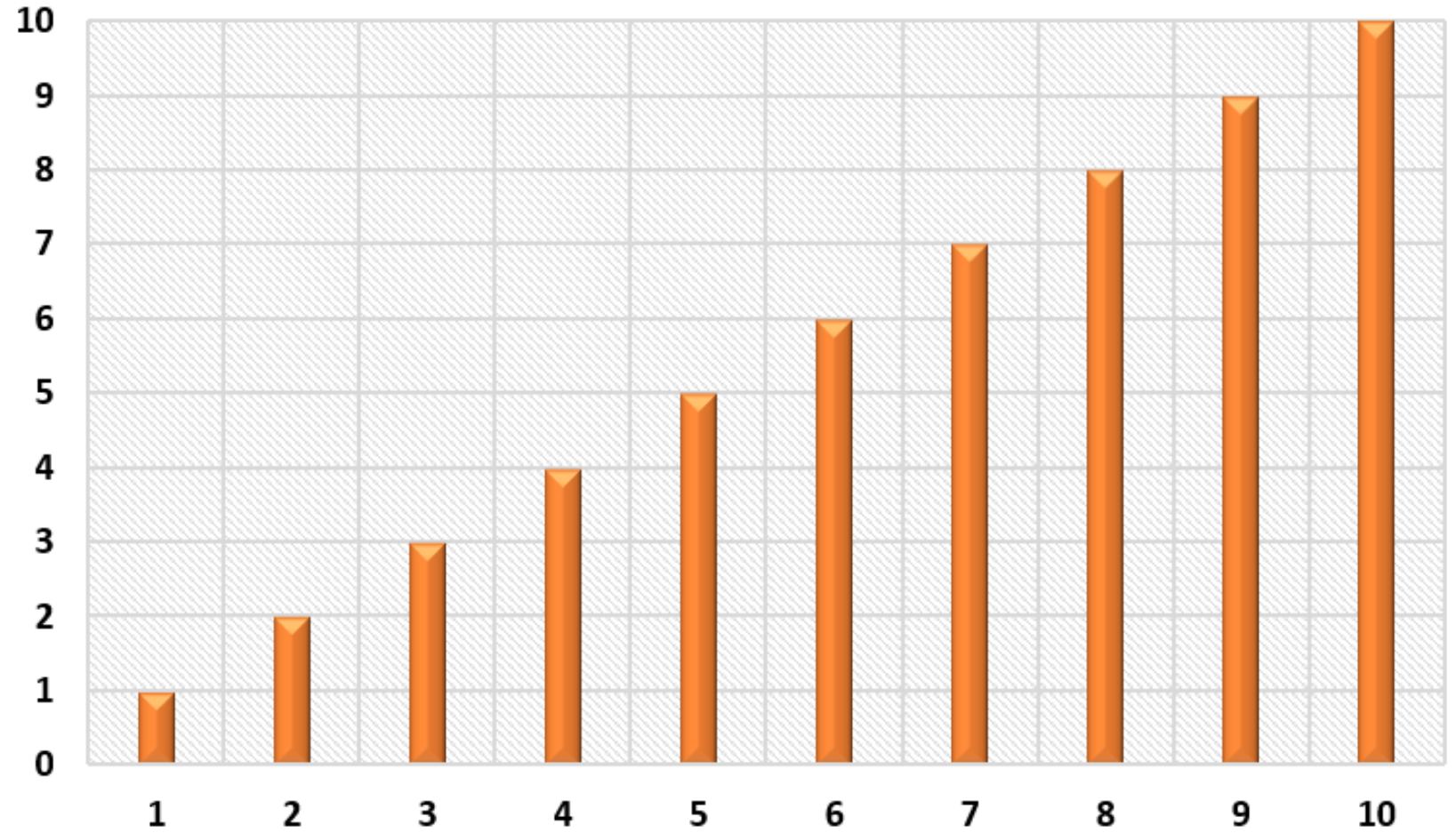
Speed up =

tiempo de ejecución usando **1** procesador
tiempo de ejecución usando ***p*** procesadores

Speed up simplista – Línea base



Speed up



Número de procesadores

Ley de Amdahl – Línea base

f: Fracción de código paralelizable de un programa

N: Número de procesadores

$$\text{Speed up} = \frac{1}{1 - f + \frac{f}{N}}$$

Aceleración máxima limitada por la porción serial: **Cuello de botella serial**

Ley de Amdahl, pero ...

$$\text{Speed up} = \frac{1}{1 - f + \frac{f}{N}}$$

- La porción paralela generalmente no es perfectamente paralelizable. Hay sobrecarga derivada de:
 - La sincronización
 - Desequilibrio de carga
 - Uso compartido de recursos (contención)

Ley de Amdahl

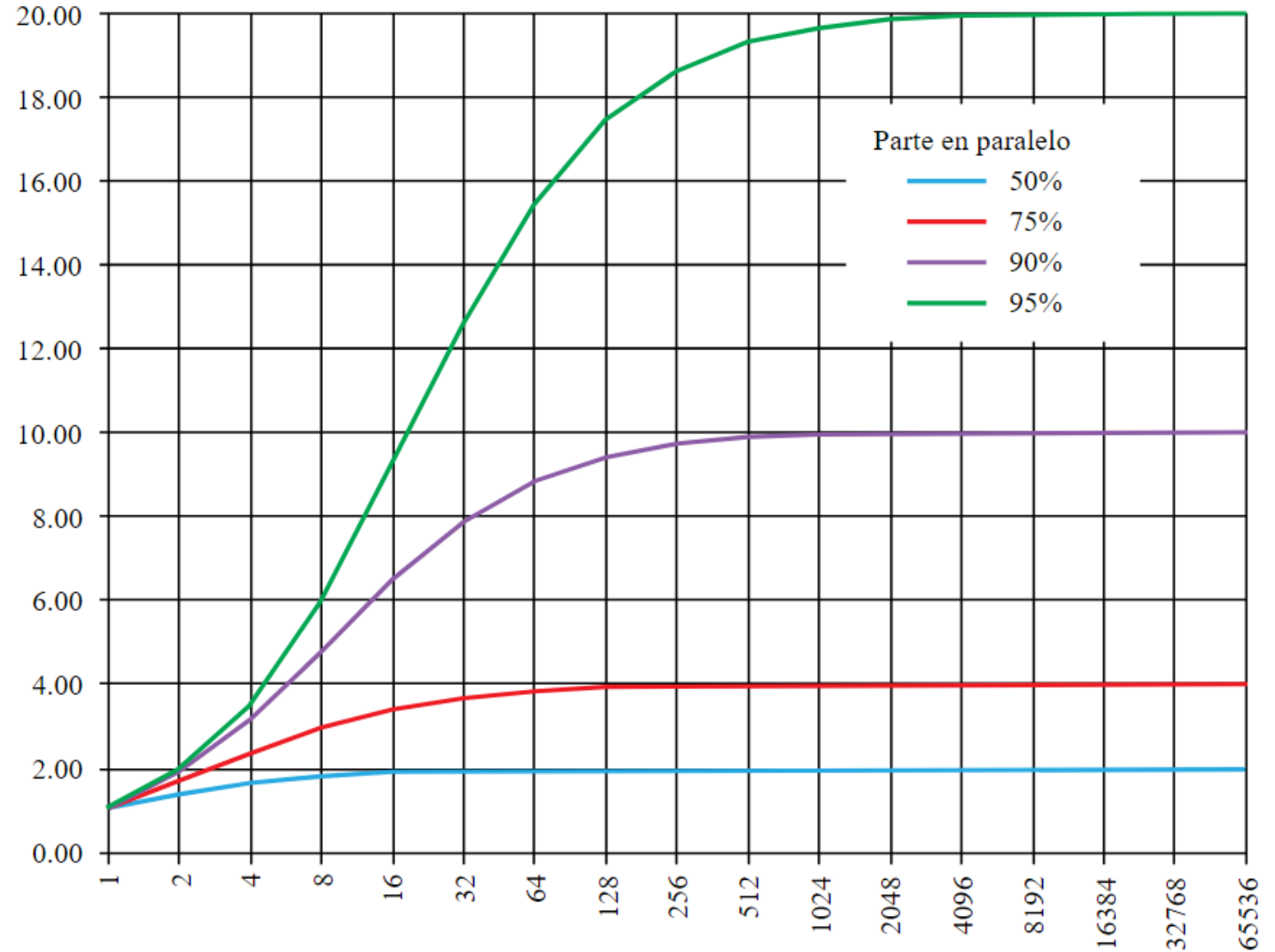
Speed up =

$1 - f$

+

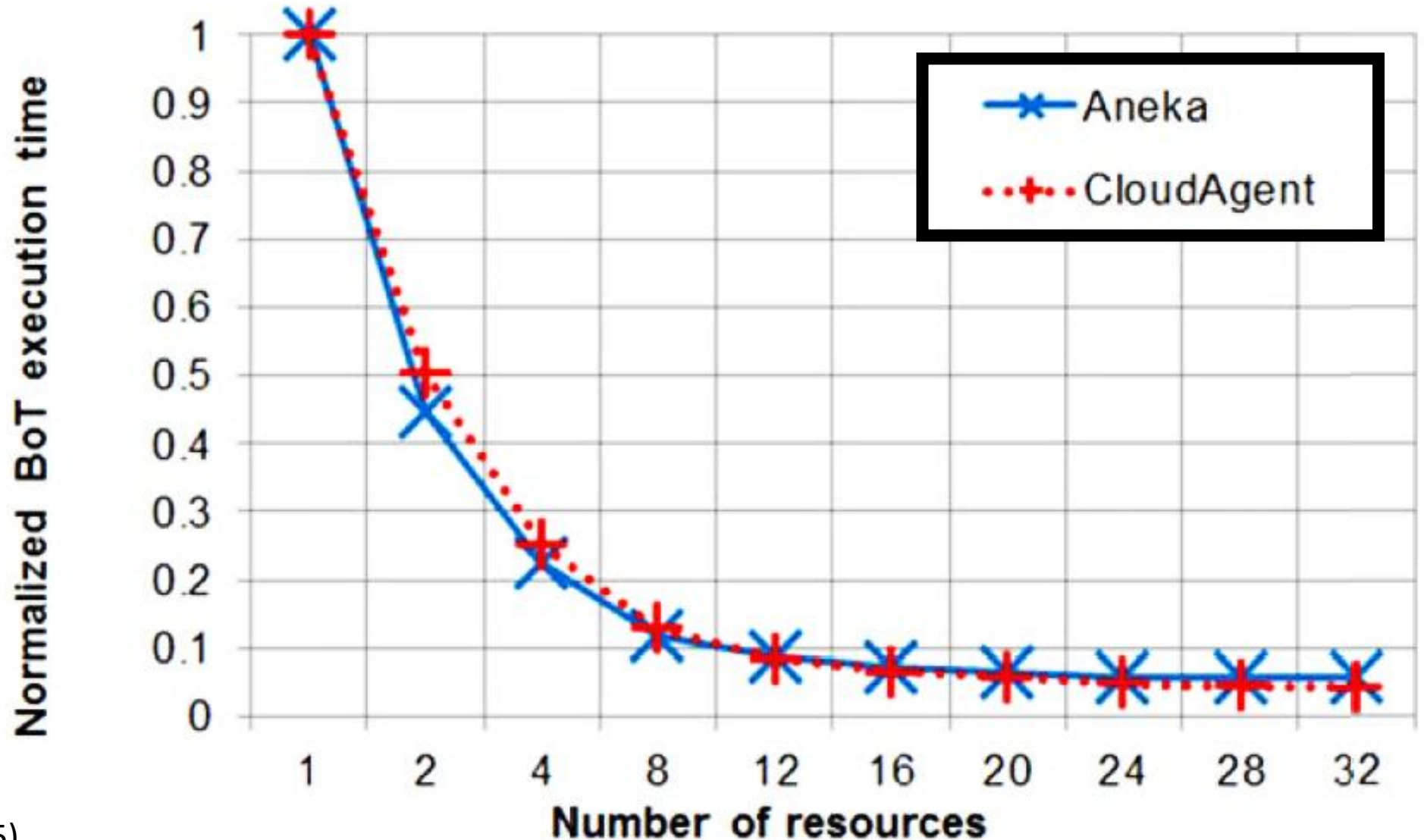
$\frac{f}{N}$

Speed up



Número de procesadores

Puntos de referencia – comparación



Ley de (Gordon) Moore

- El número de transistores en un circuito integrado se duplicará cada 2 años.

“So I changed my projection looking forward [...] Instead of doubling every year, it will be closer to doubling every two years.” (Moore)

60s

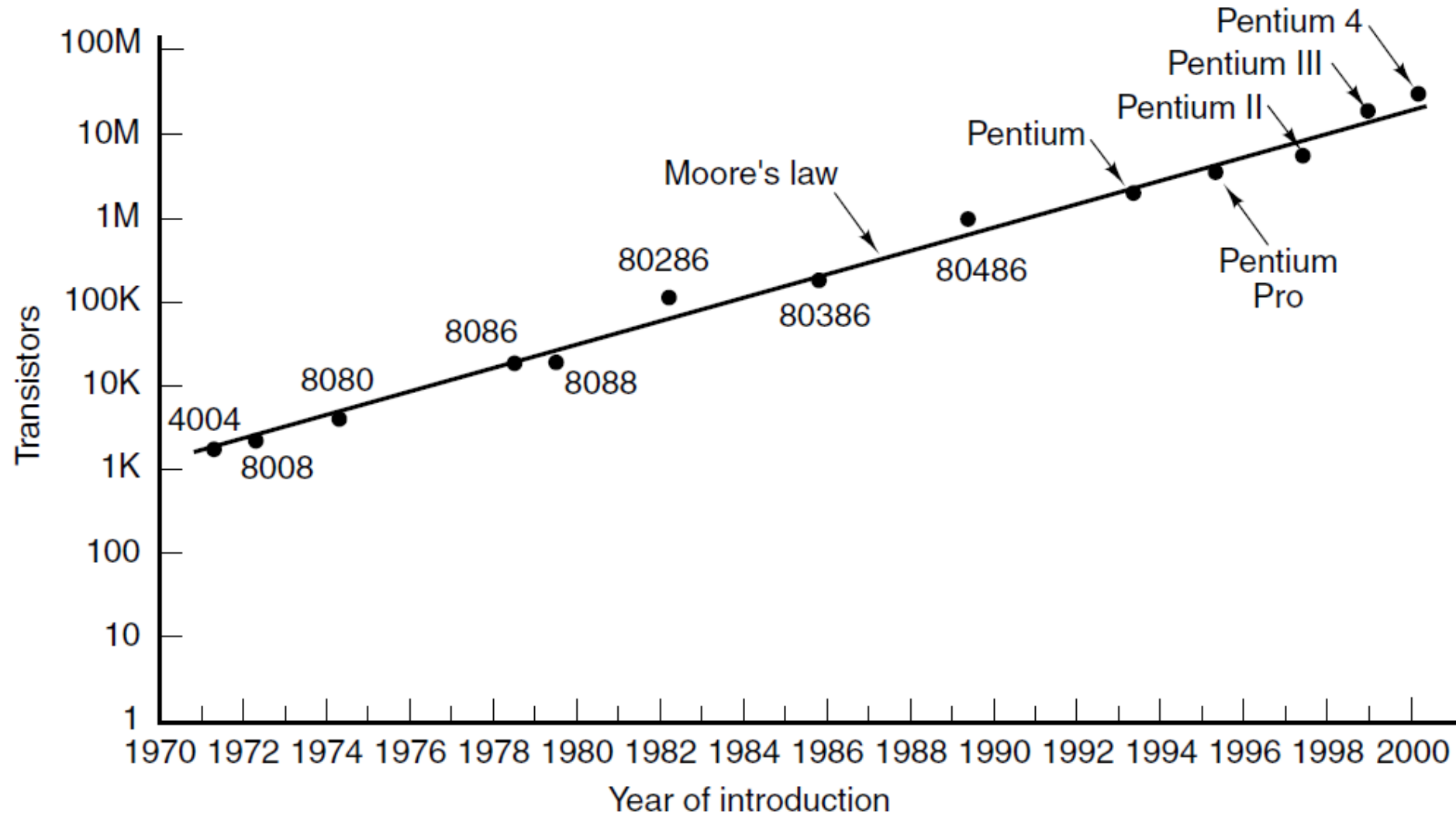


Moore, G. E. (1995, May). Lithography and the future of Moore's law. In *Integrated Circuit Metrology, Inspection, and Process Control IX* (Vol. 2439, pp. 2-17). SPIE.

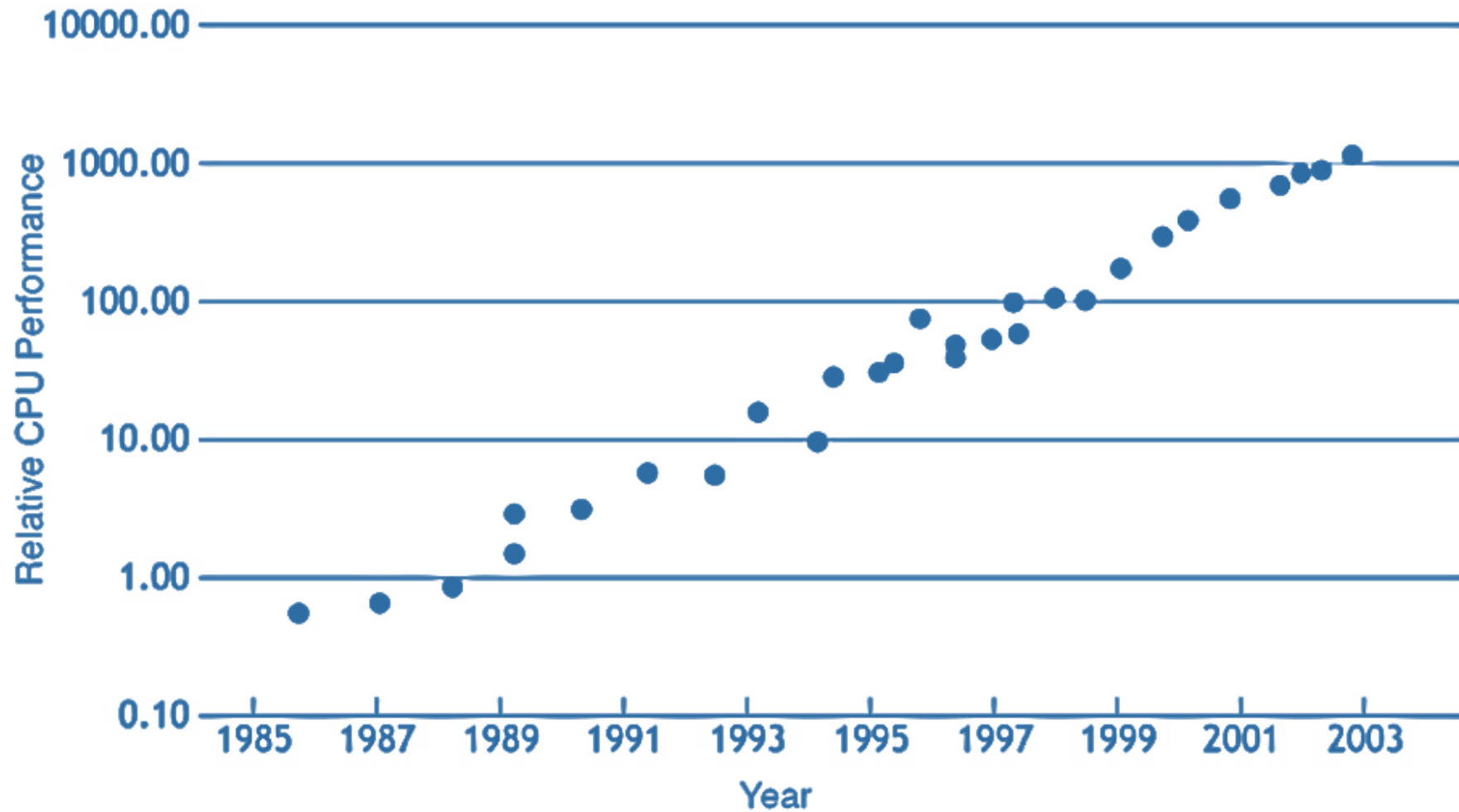
¿Realmente se necesita el Cómputo Paralelo?



Ley de (Gordon) Moore



Ley de (Gordon) Moore

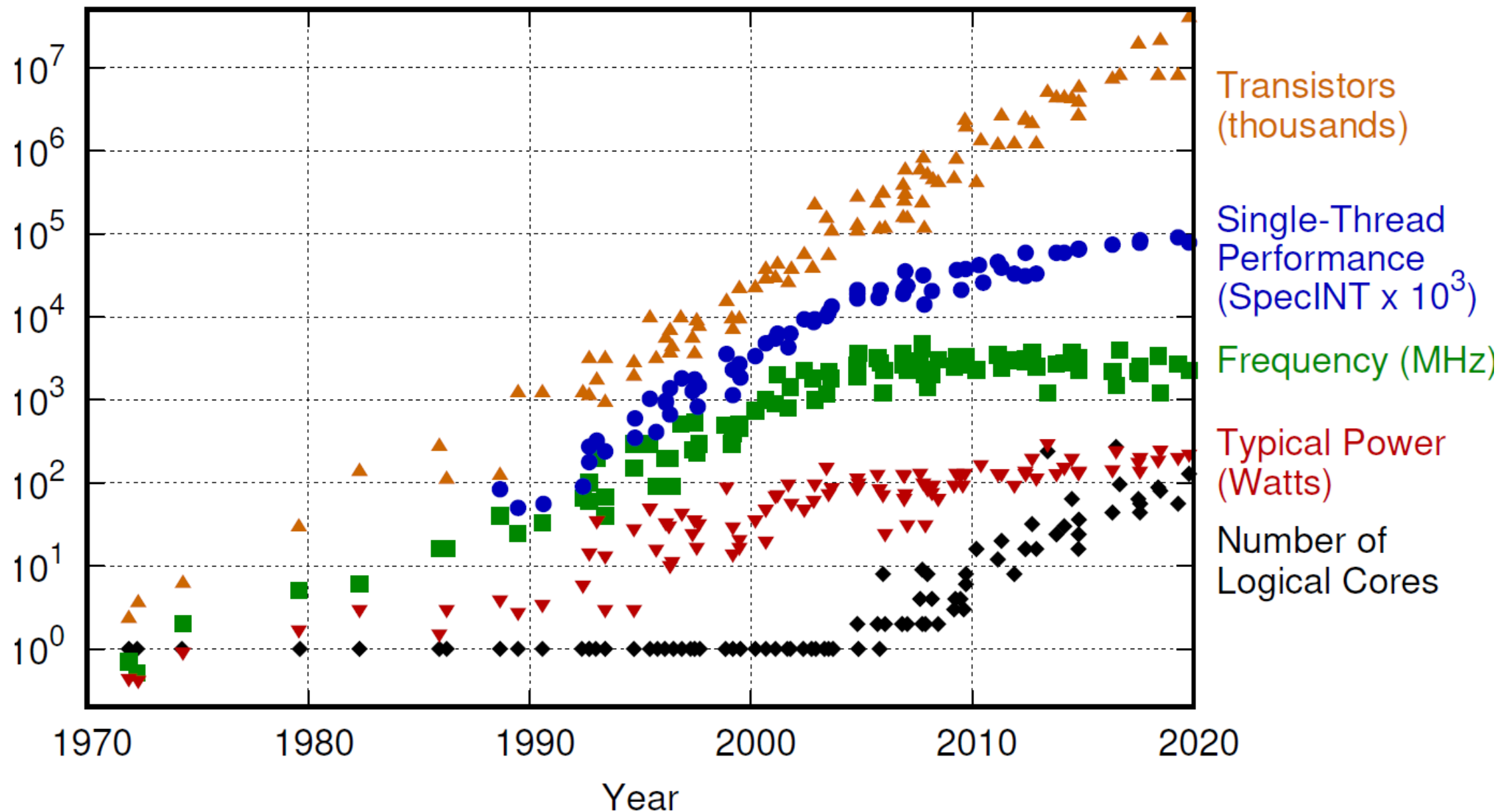


“As conventional **silicon-based transistors are fast approaching the physical limit**, it is essential to seek alternative candidates, which should be compatible with or even replace microelectronics in the future” (Meng, 2022)



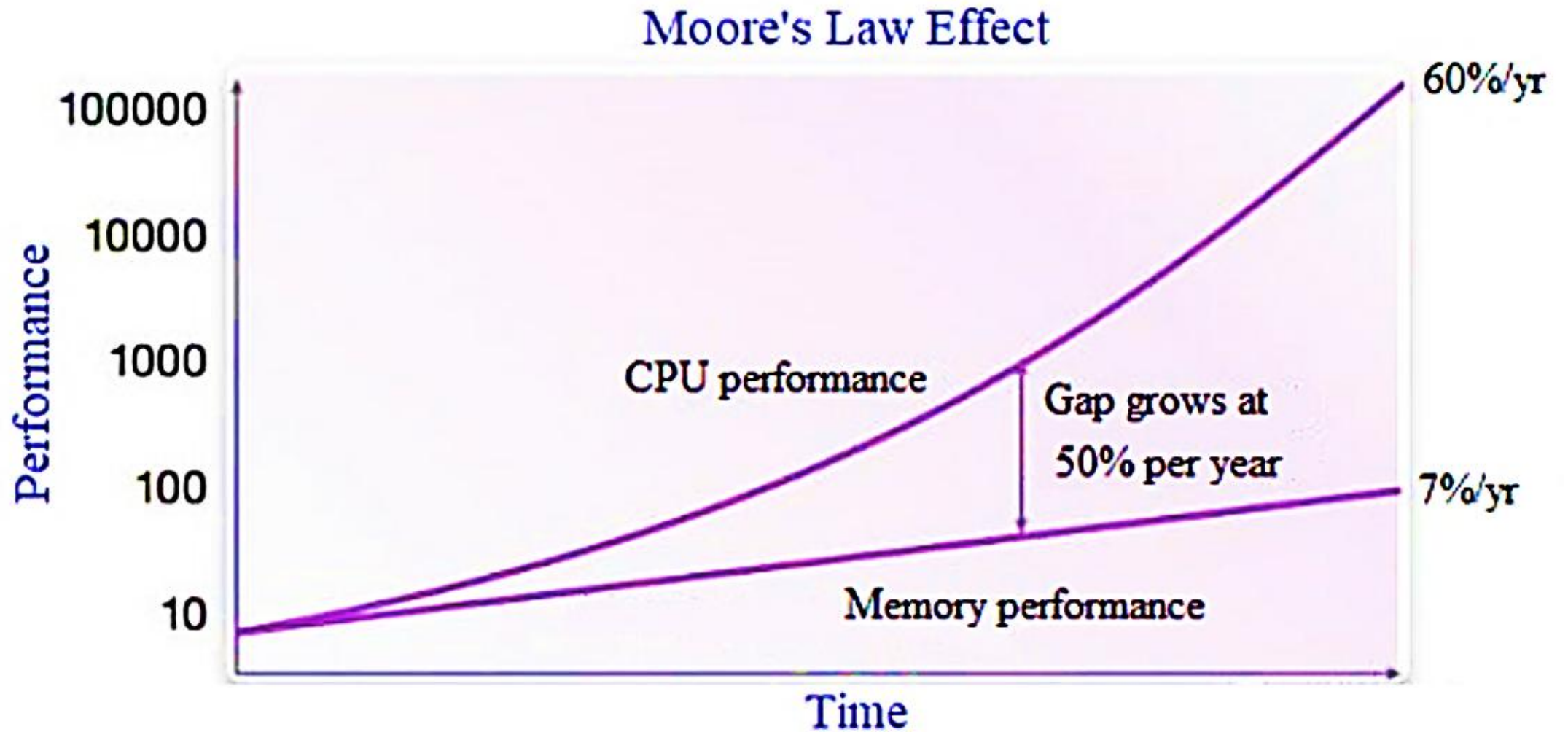
Meng, L., Xin, N., Hu, C., Sabea, H. A., Zhang, M., Jiang, H., ... & Guo, X. (2022). Dual-gated single-molecule field-effect transistors beyond Moore's law. *Nature communications*, 13(1), 1410.

48 Years of Microprocessor Trend Data



Original data up to the year 2010 collected and plotted by M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, and C. Batten
New plot and data collected for 2010-2019 by K. Rupp

The Memory Wall



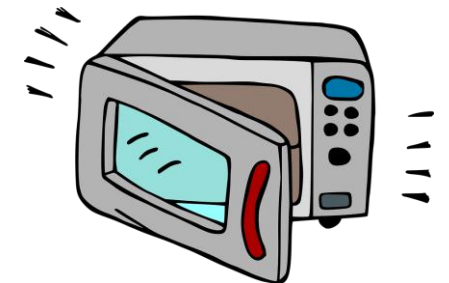
The Power Wall

- Mucho consumo energético = Altas temperaturas

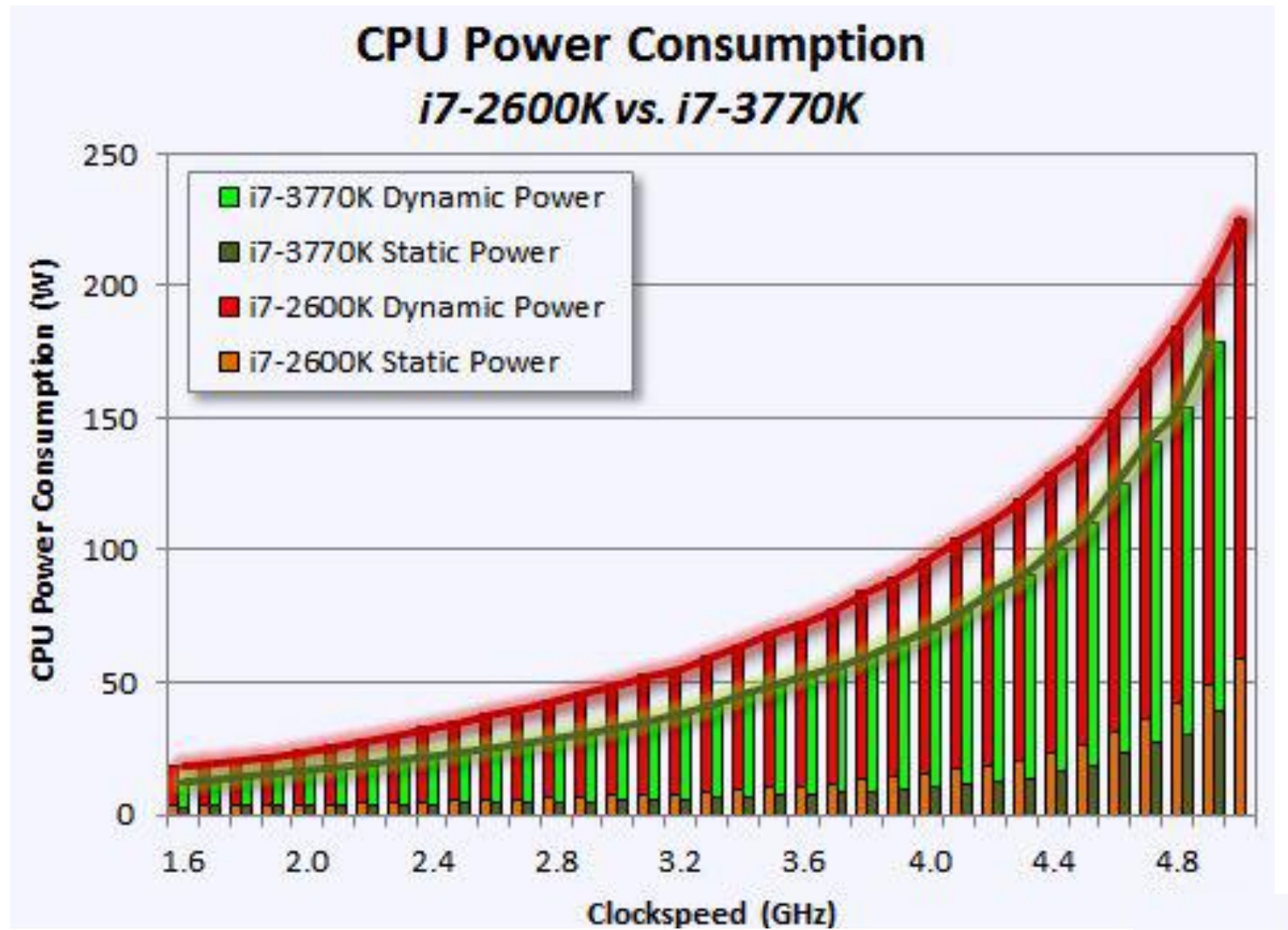


Apple M1 laptop:	13W
Intel Core i9 10900K (in desktop CPU):	95W
NVIDIA RTX 3080 GPU	320W
Mobile phone processor	1/2 - 2W
World's fastest supercomputer	megawatts

Standard microwave oven	700W
--------------------------------	-------------



The Power Wall



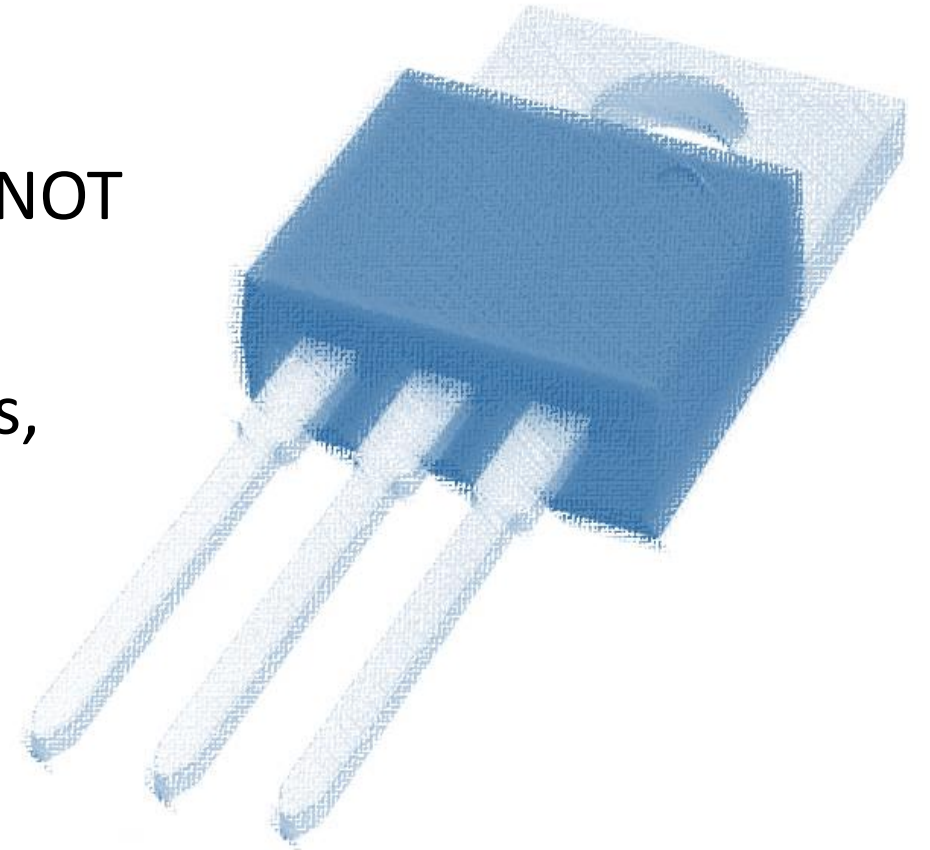
Una razón de la mejora en el rendimiento del procesador **(Hasta hace 15 años)**

- Incrementar la frecuencia del reloj del CPU

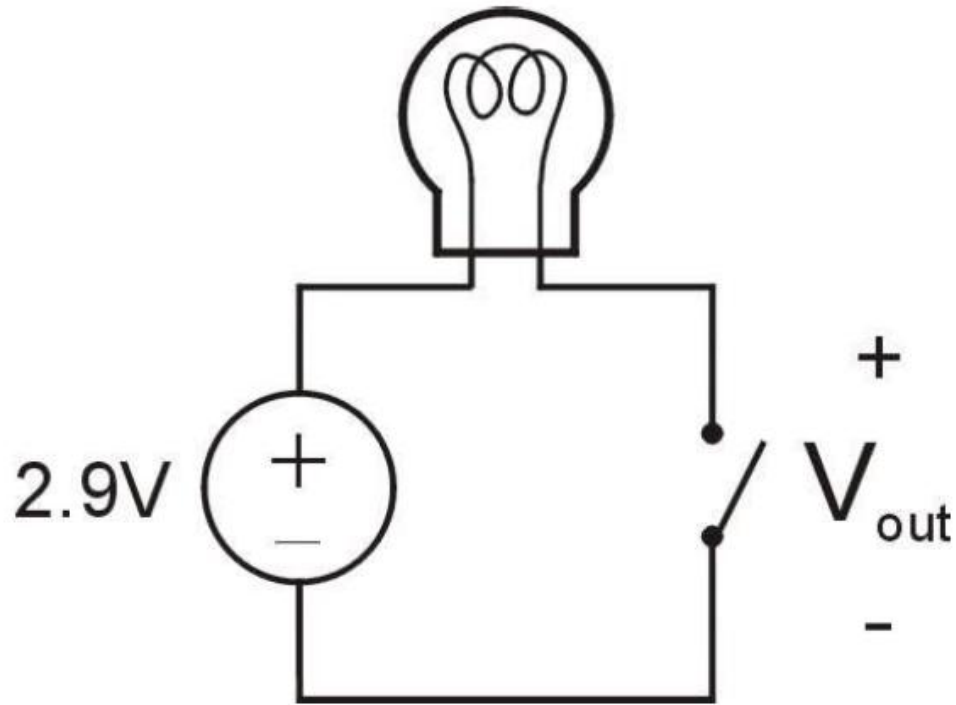


Transistores como Building Blocks

- Microprocesadores contienen millones de transistores
- Base de las **compuertas lógicas** AND, OR, NOT
- Base para hacer **sumadores**, multiplexores, decodificadores, ...
- Base de los **procesadores**



Circuito de un Switch



Puede fácilmente representar estados
on/off | abierto/cerrado | voltaje/no
voltaje

- **Switch abierto:**

- No hay corriente o la corriente no fluye
- La luz está apagada
- V_{out} es +2.9V

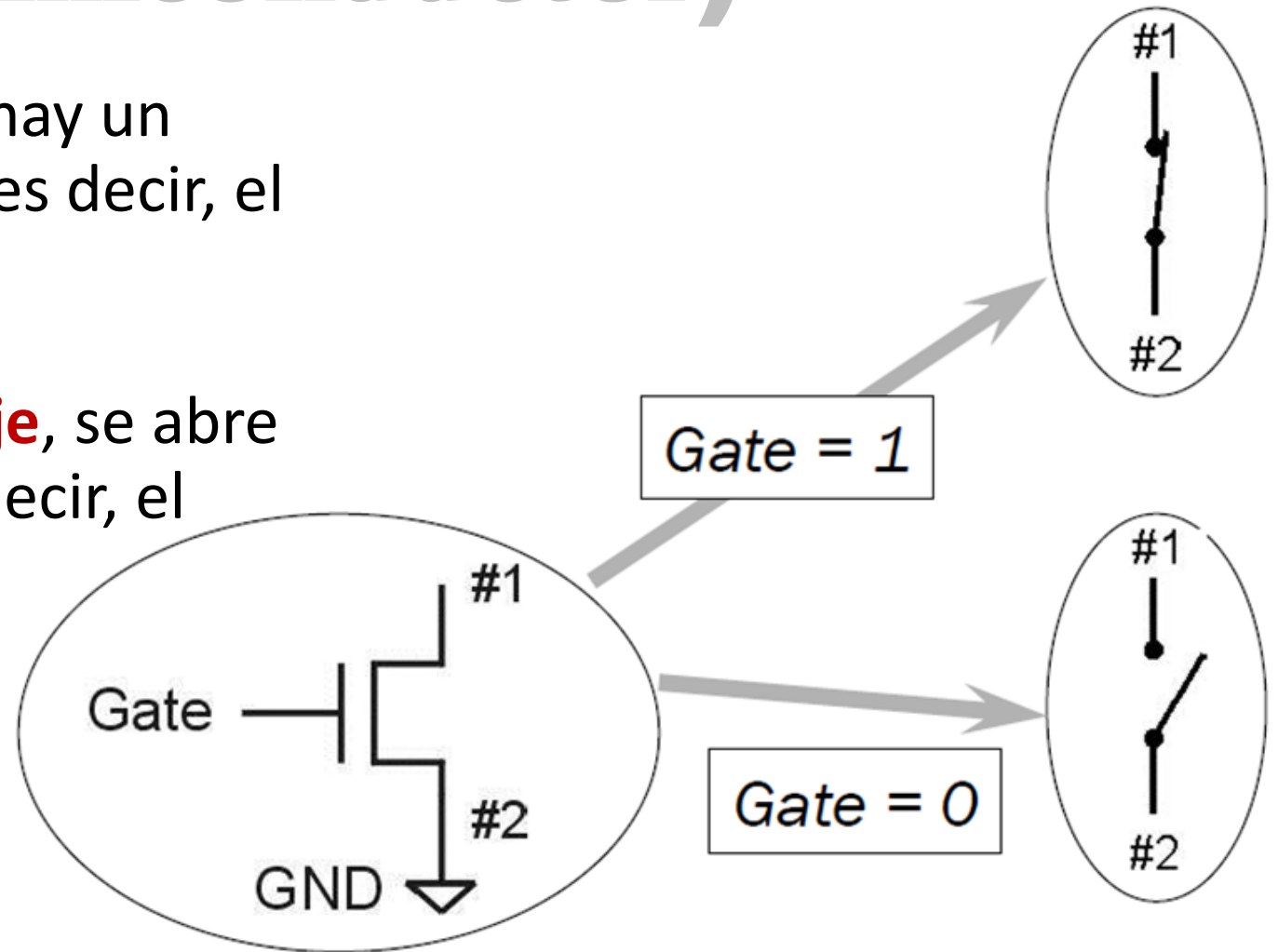
- **Switch cerrado:**

- Corto circuito (de no existir el foco)
- La corriente fluye
- La luz está encendida
- V_{out} es +0V

Transistores MOS Tipo-N

(metal oxide semiconductor)

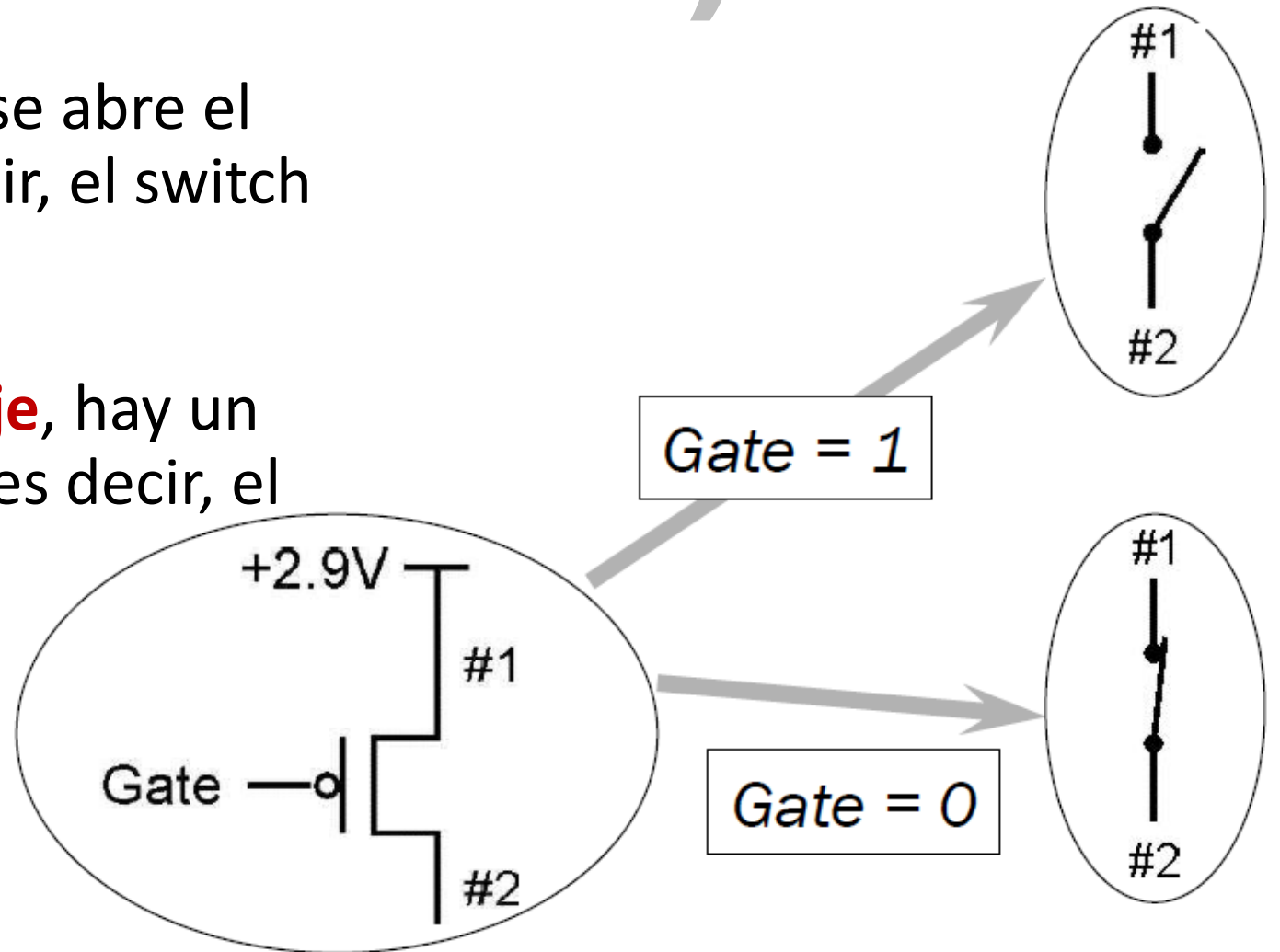
- Cuando **GATE tiene voltaje**, hay un corto circuito entre #1 y #2 (es decir, el switch está **cerrado**)
- Cuando **GATE no tiene voltaje**, se abre el circuito entre #1 y #2 (es decir, el switch está **abierto**)



Transistores MOS Tipo-P

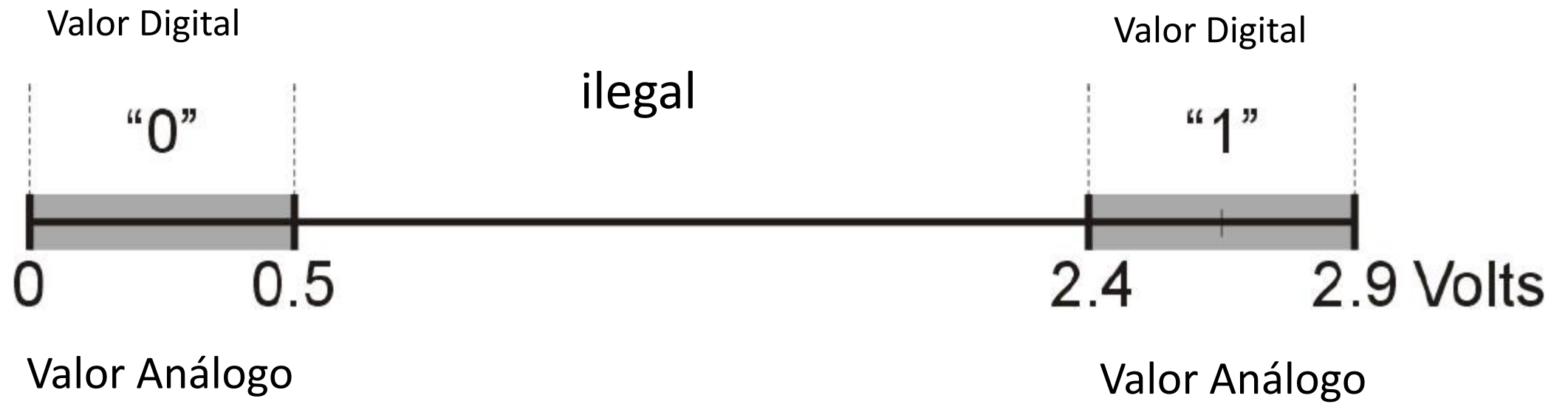
(metal oxide semiconductor)

- Cuando **GATE tiene voltaje**, se abre el circuito entre #1 y #2 (es decir, el switch está **abierto**)
- Cuando **GATE no tiene voltaje**, hay un corto circuito entre #1 y #2 (es decir, el switch está **cerrado**)

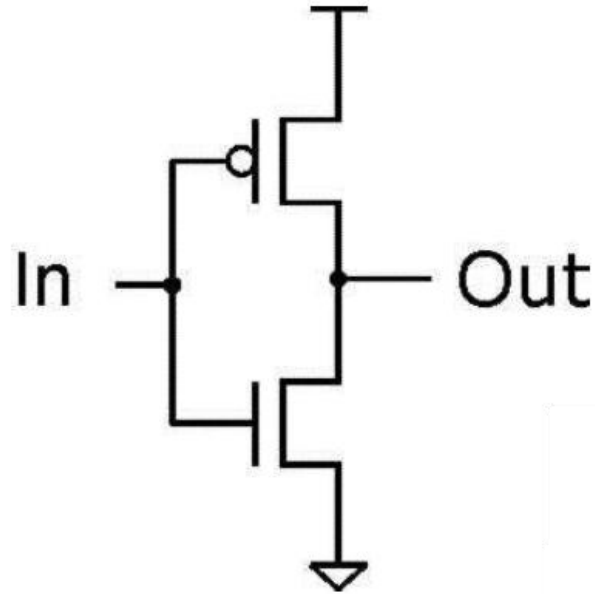


Compuertas Lógicas

- El comportamiento tipo switch de los transistores se utiliza para implementar las **funciones lógicas** AND, OR y NOT.

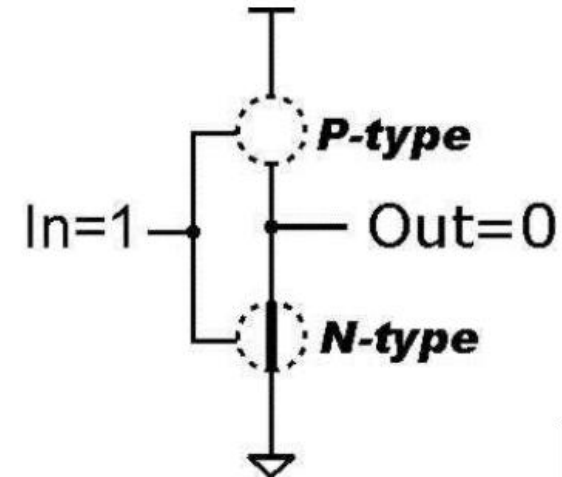
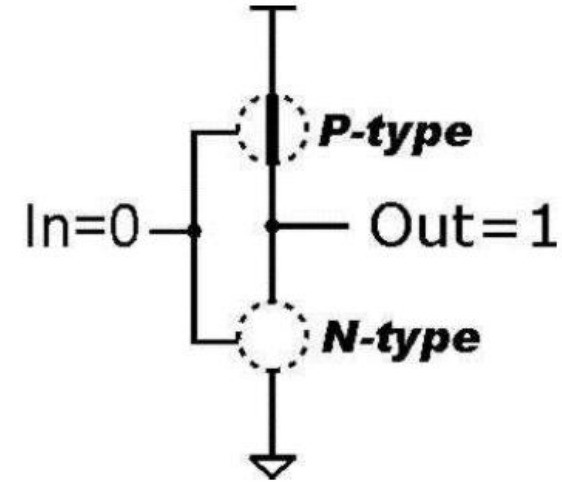


Compuerta NOT

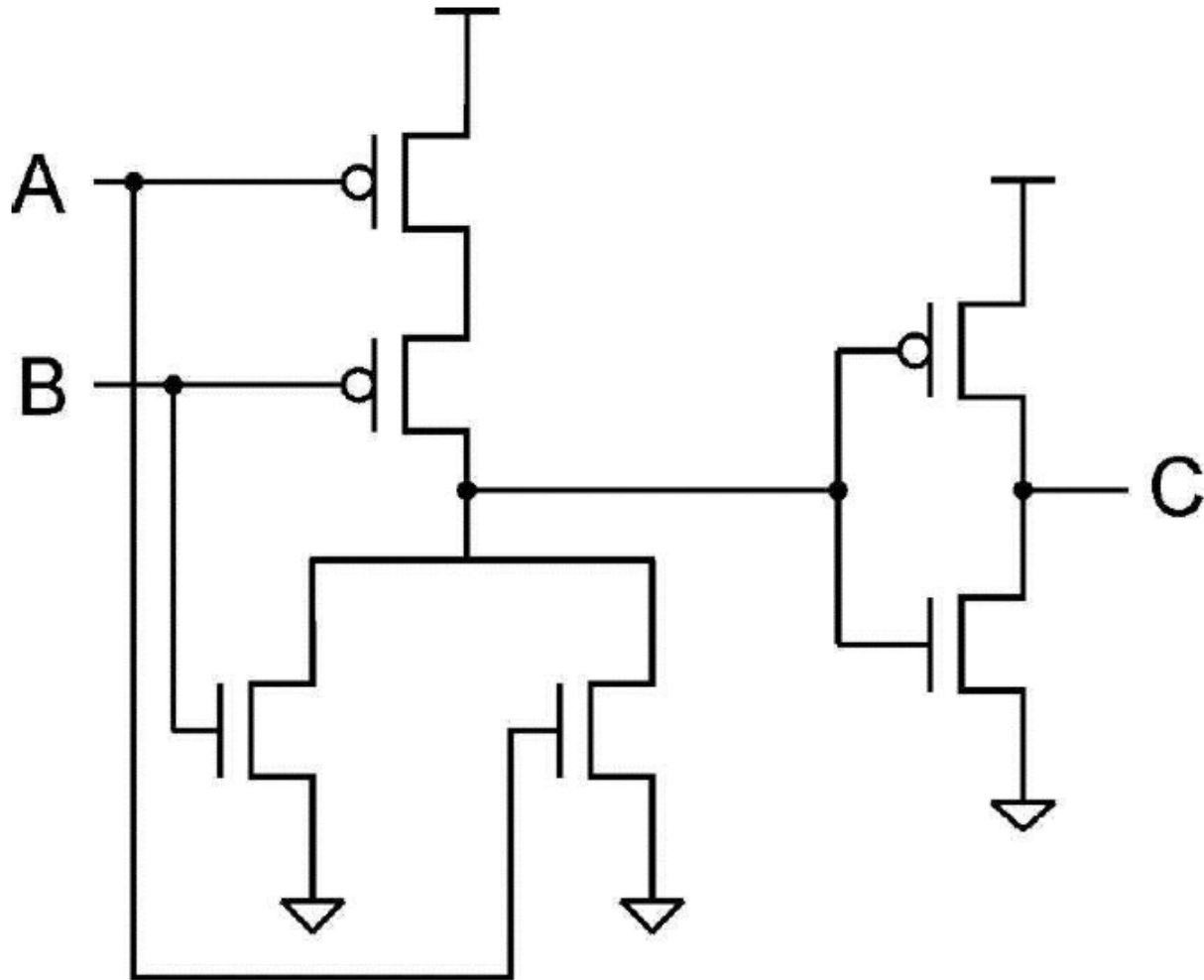


In	Out
0 V	2.9 V
2.9 V	0 V

In	Out
0	1
1	0

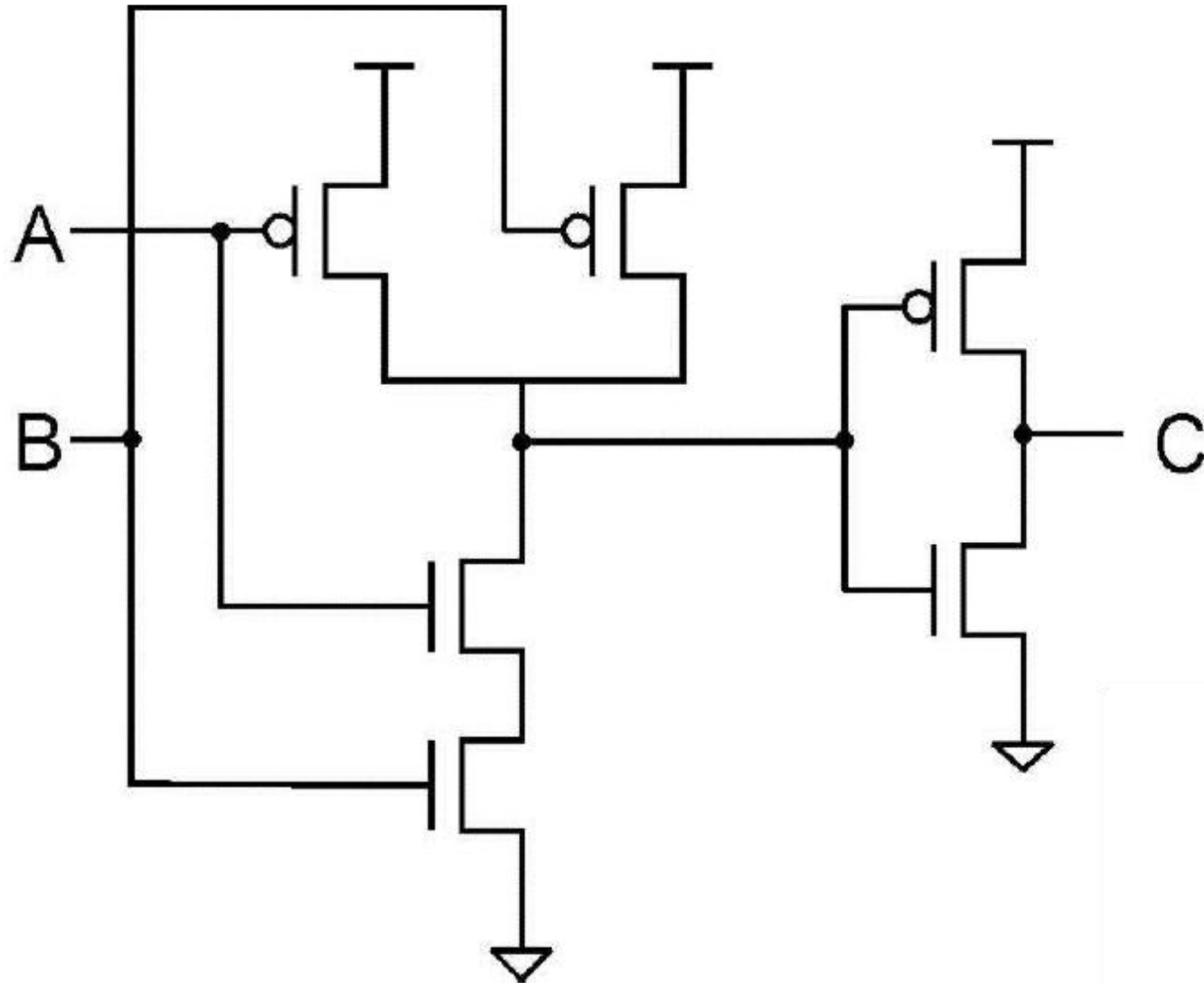


Compuerta OR



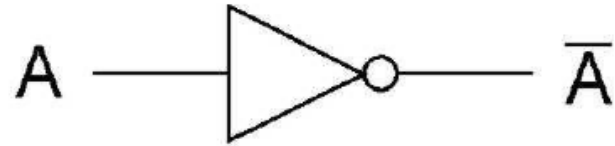
A	B	C
0	0	0
0	1	1
1	0	1
1	1	1

Compuerta AND

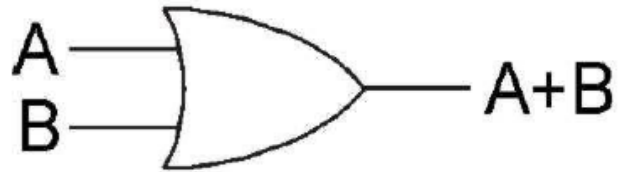


A	B	C
0	0	0
0	1	0
1	0	0
1	1	1

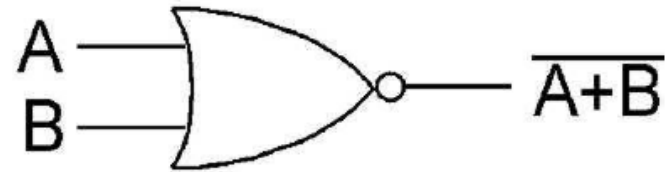
Compuertas Lógicas



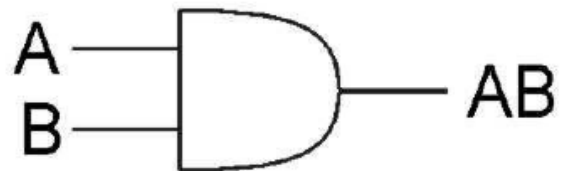
NOT



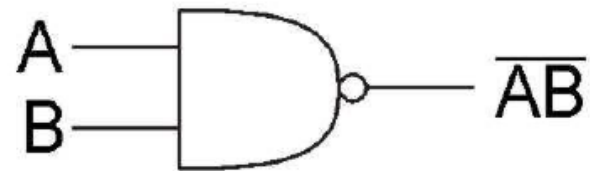
OR



NOR



AND



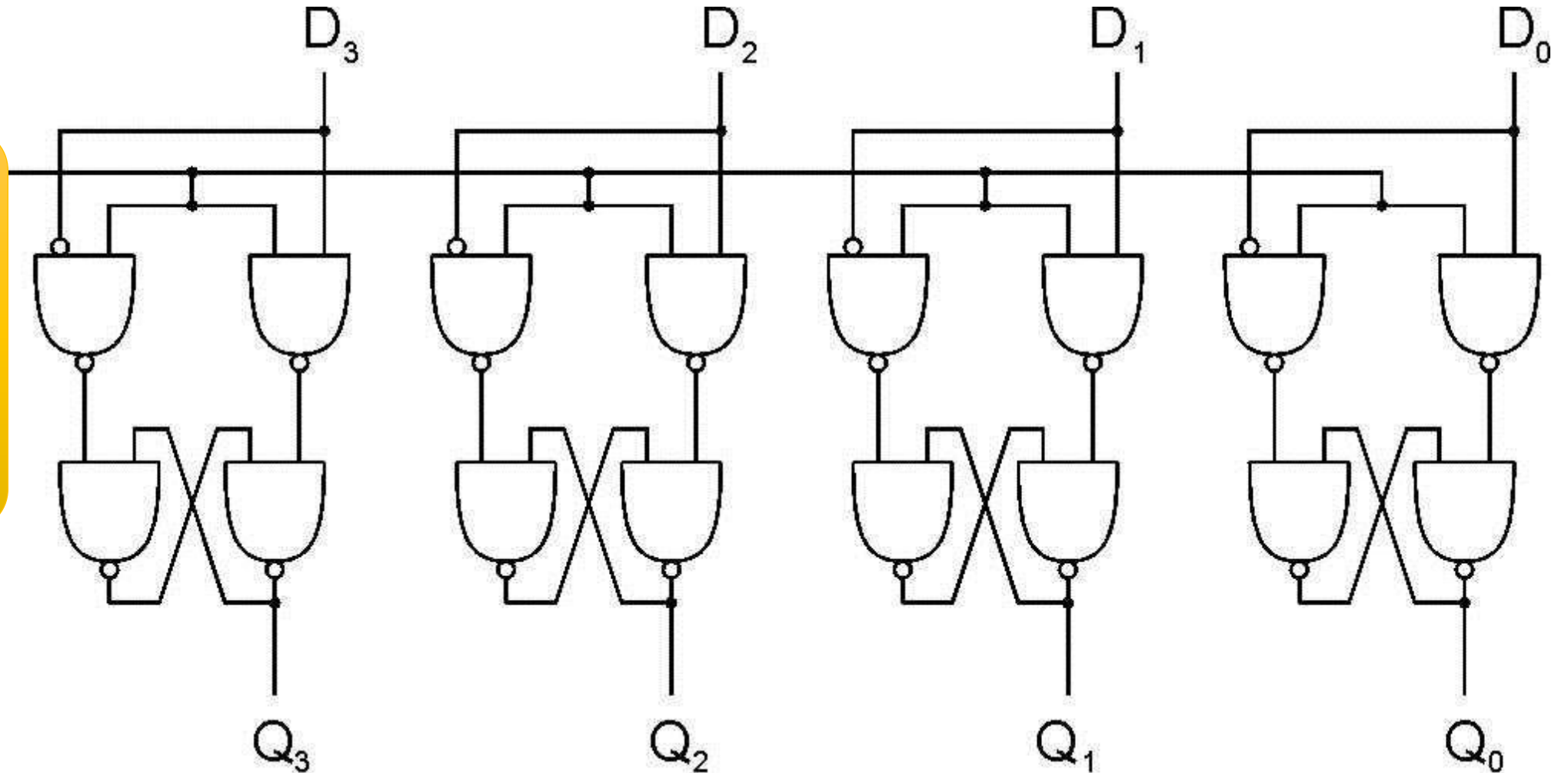
NAND

Registros (Base de la Memoria)

HE

Habilita
Escritura

1 o 0



Representando valores multi-bit

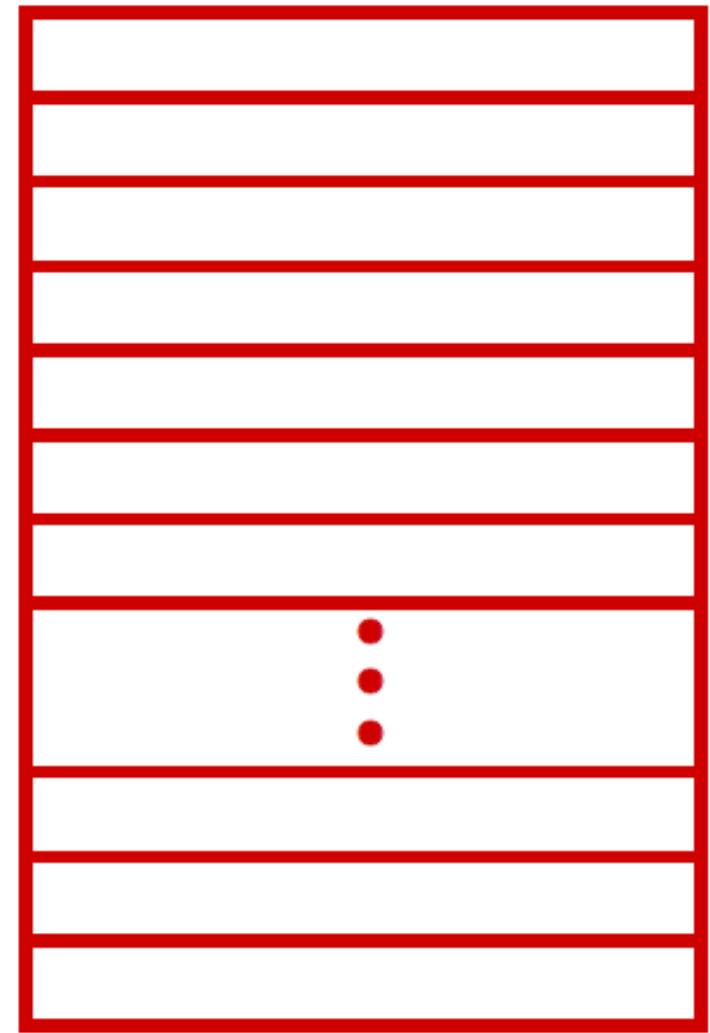
$A =$ ¹⁵ 0101001 ⁰ 11010101

$A[14:9] =$ 101001

$A[2:0] =$ 101

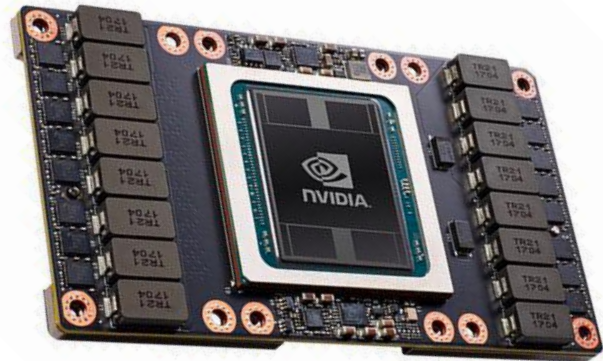
Memoria

$k = 2^n$
locations



m bits

¿Qué hace un procesador?



Programa desde la perspectiva del procesador

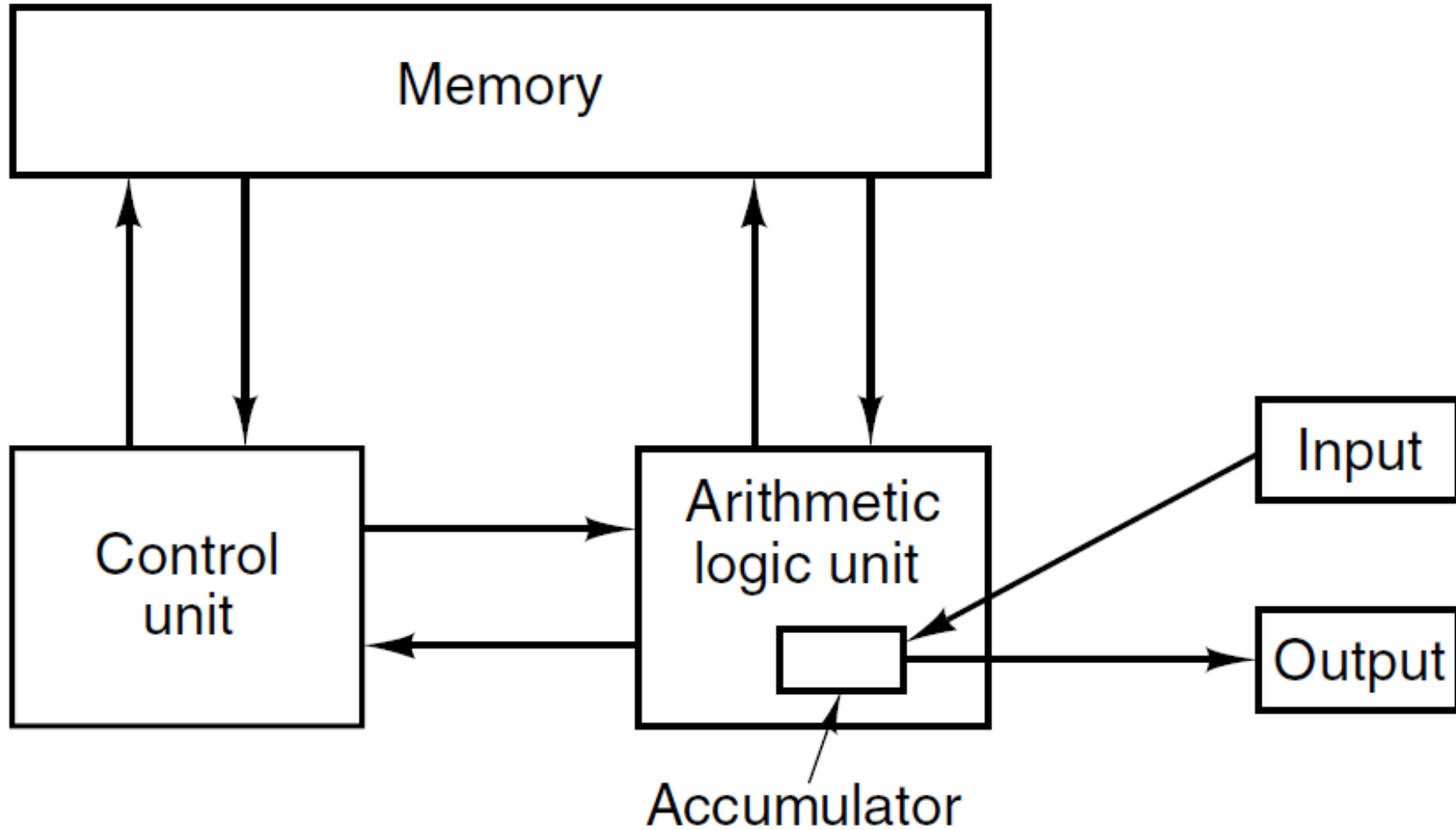
Julia

@code_native 2*3

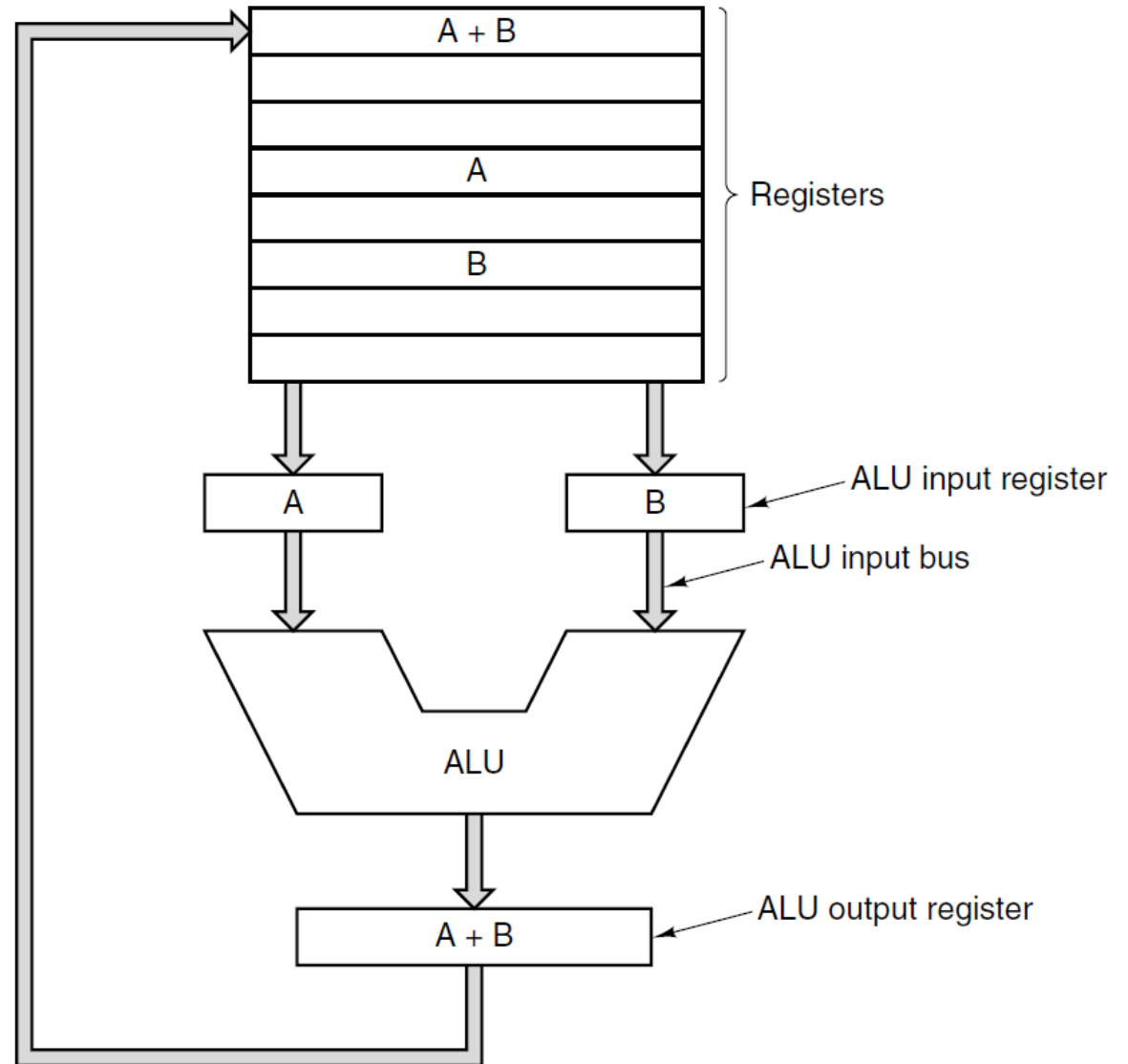
Código
compilado

```
pushq    %rbp
movq     %rsp, %rbp
movq     %rcx, %rax
imulq    %rdx, %rax
popq     %rbp
retq
nopl     (%rax)
```

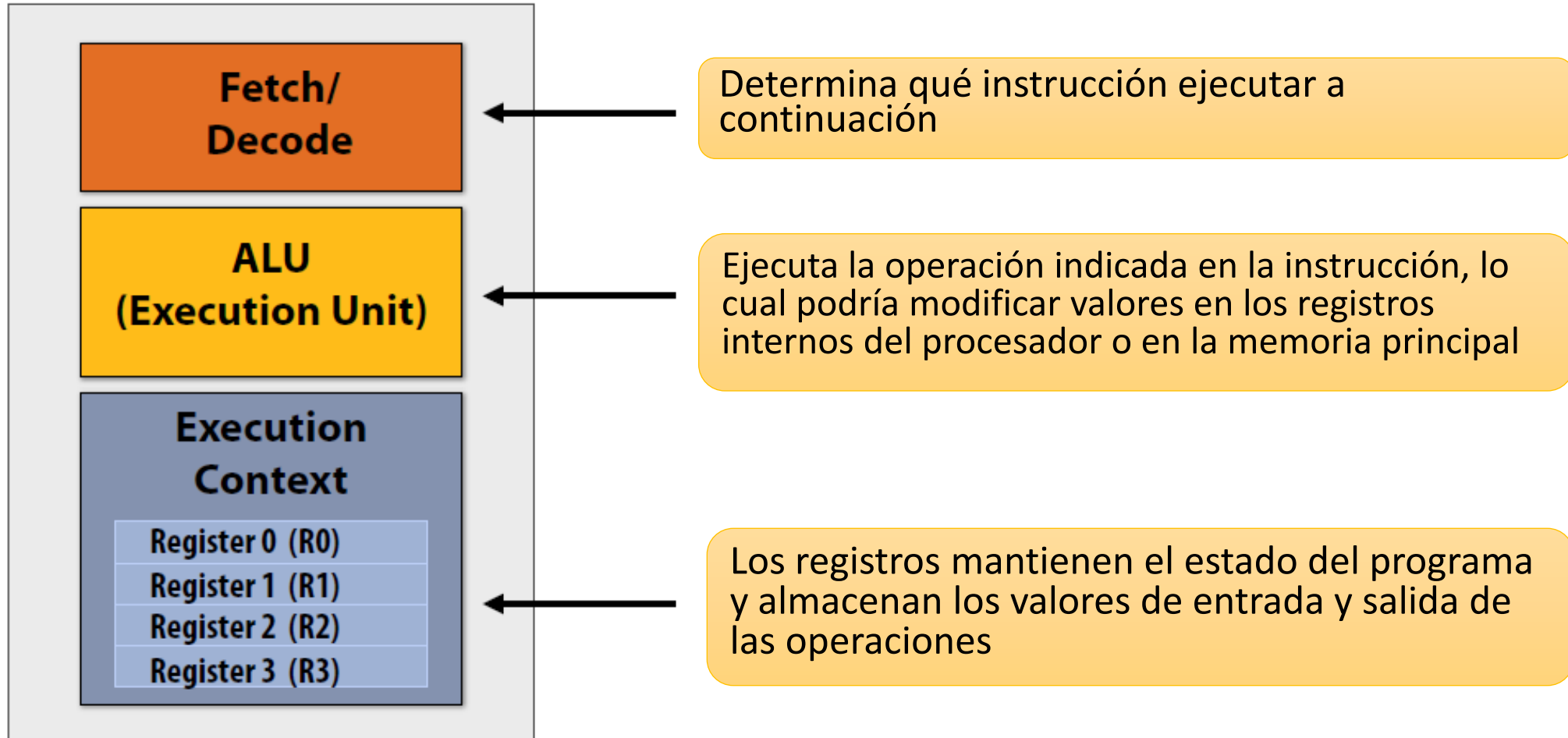
Arquitectura Von Neumann



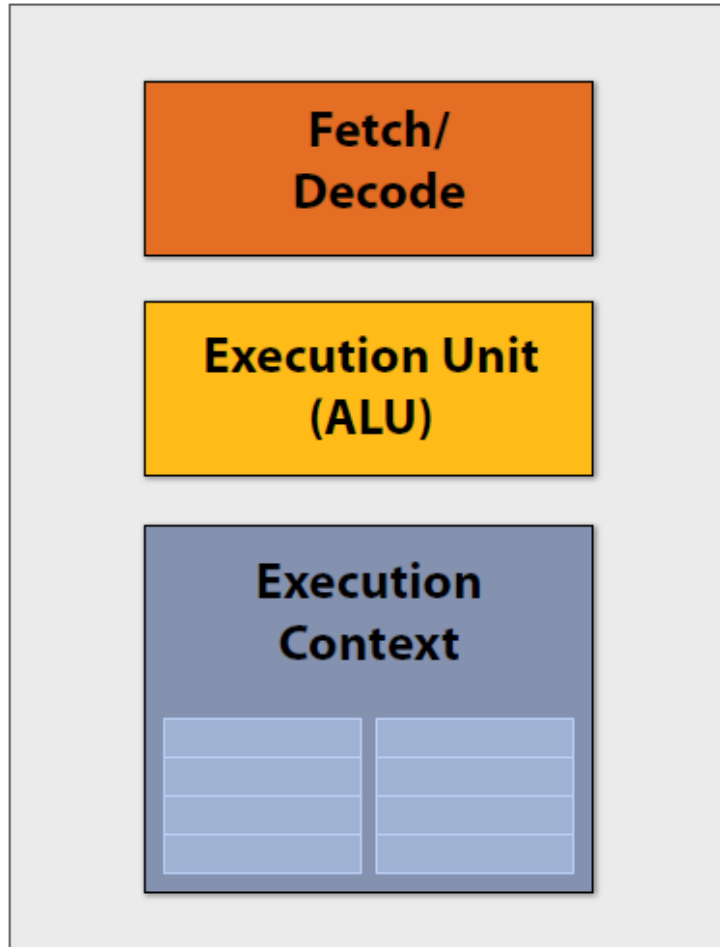
Camino de datos en Arquitectura Von Neumann



Qué hace un procesador



Ejecución de Programa



`x[i]`



```
ld  r0, addr[r1]
mul r1, r0, r0
mul r1, r1, r0
...
...
...
...
...
...
st  addr[r2], r0
```



`result[i]`

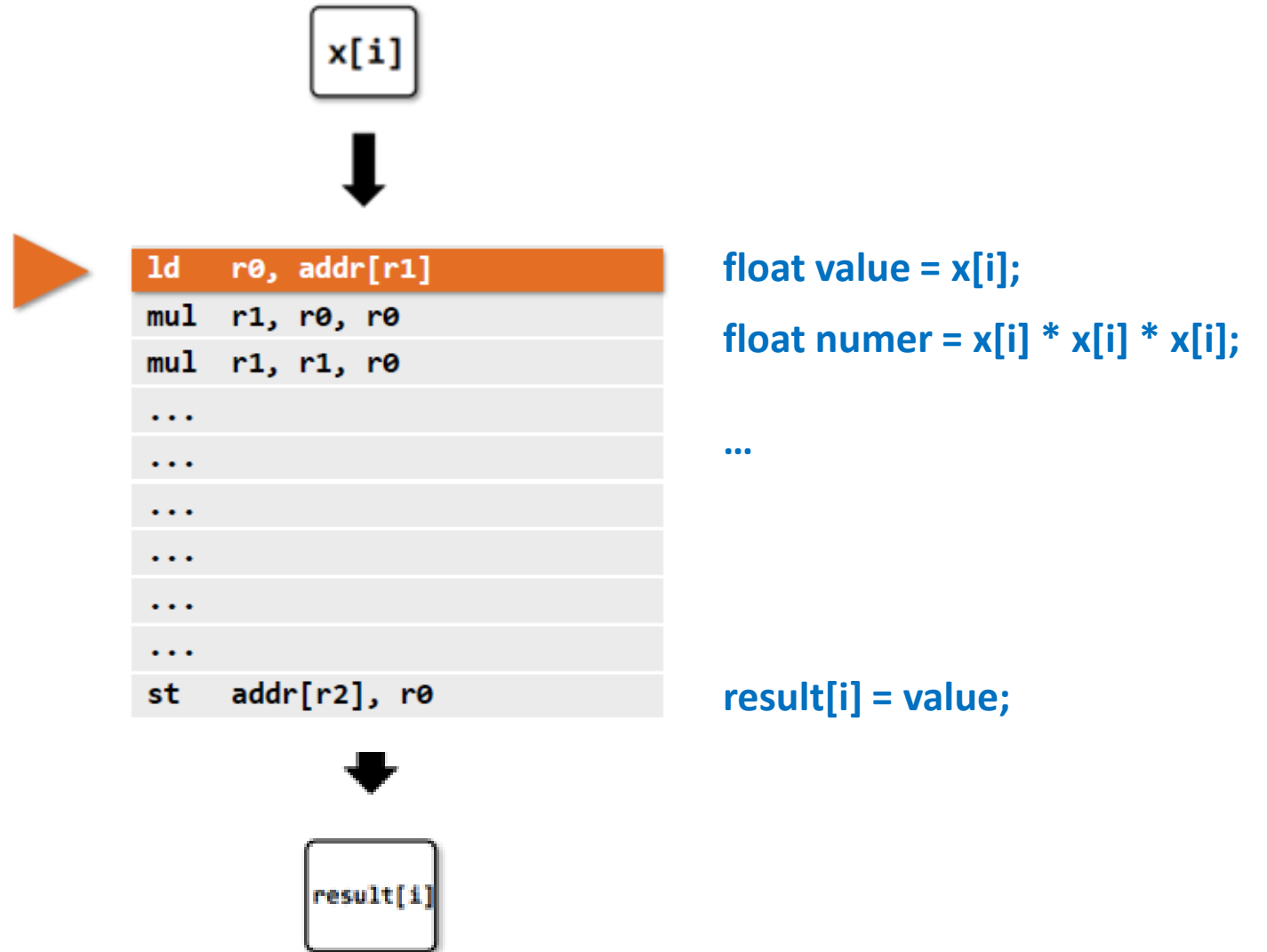
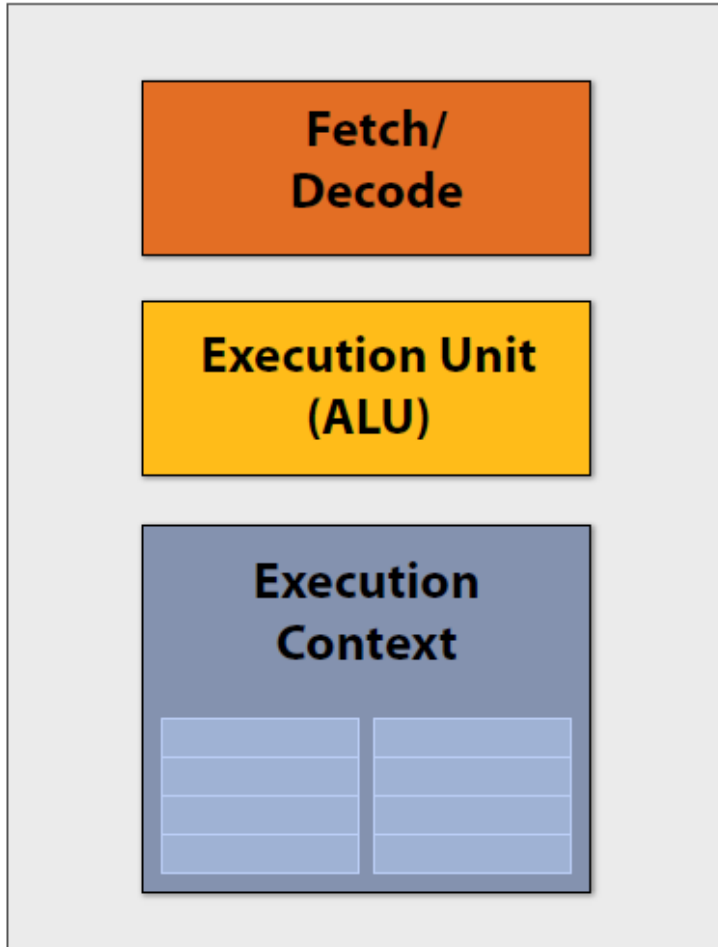
`float value = x[i];`

`float numer = x[i] * x[i] * x[i];`

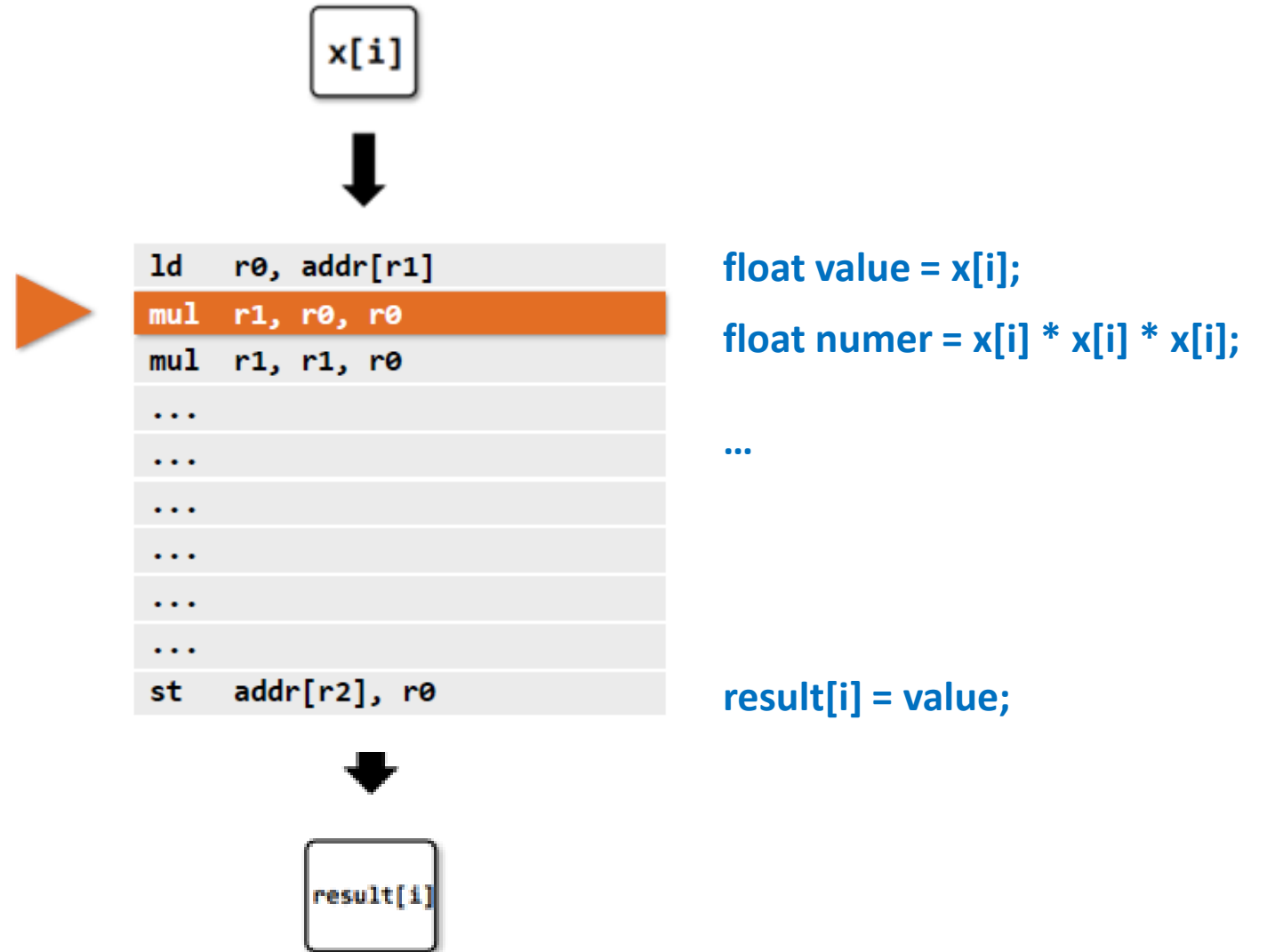
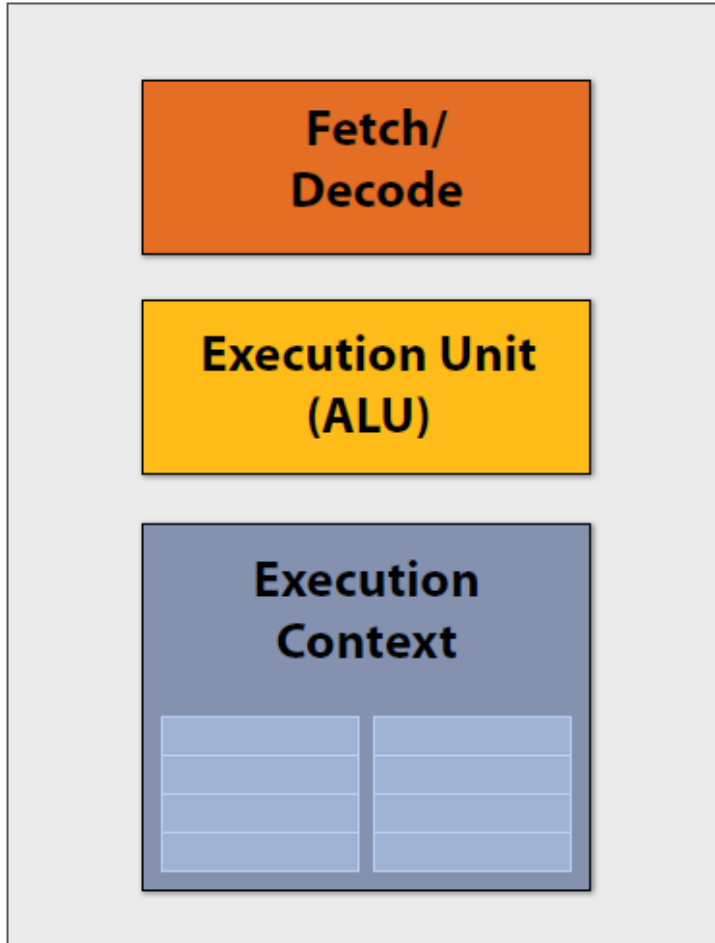
`...`

`result[i] = value;`

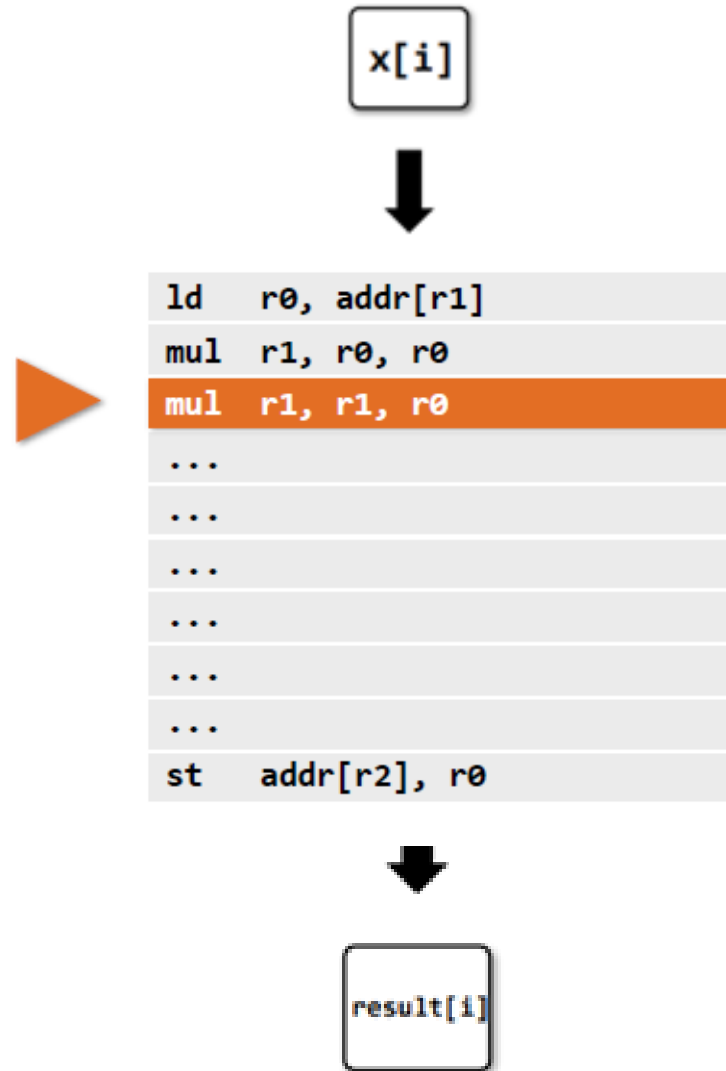
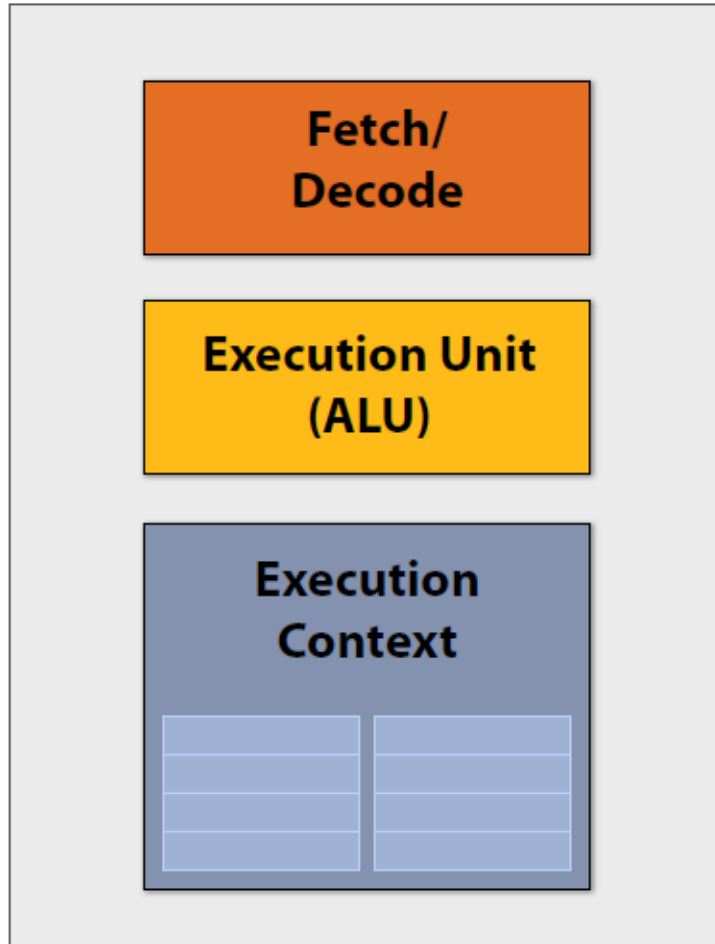
Ejecución de Programa



Ejecución de Programa



Ejecución de Programa



`float value = x[i];`

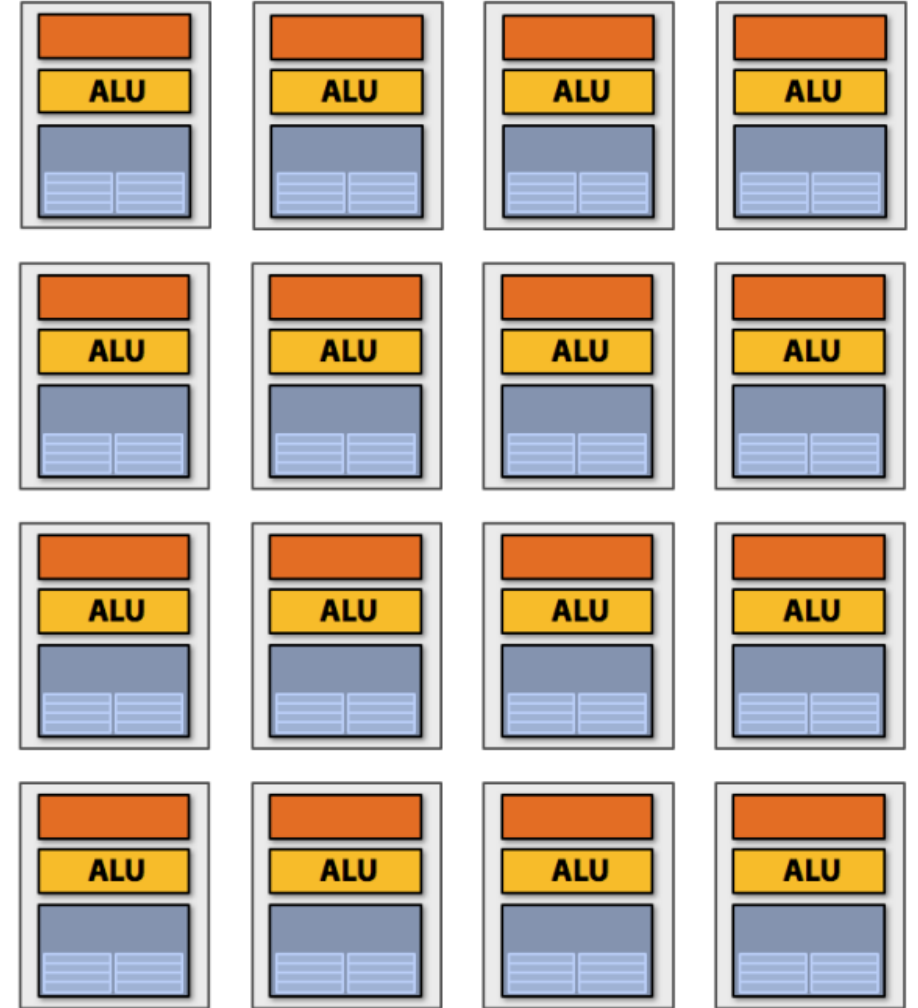
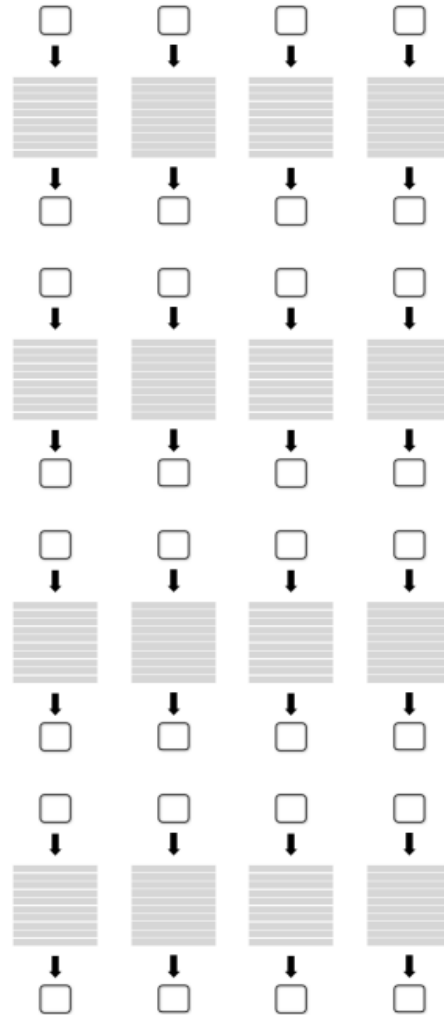
`float numer = x[i] * x[i] * x[i];`

`...`

`result[i] = value;`

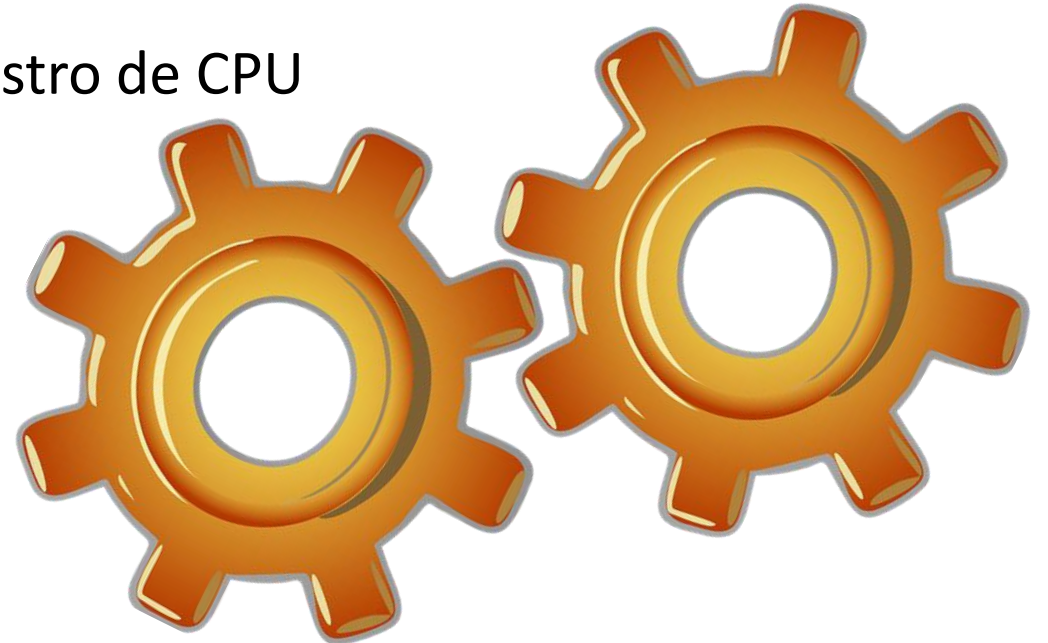
Ejecución de Programa

Si tuviéramos
16 cores, tendríamos
16 flujos de
instrucción
independientes

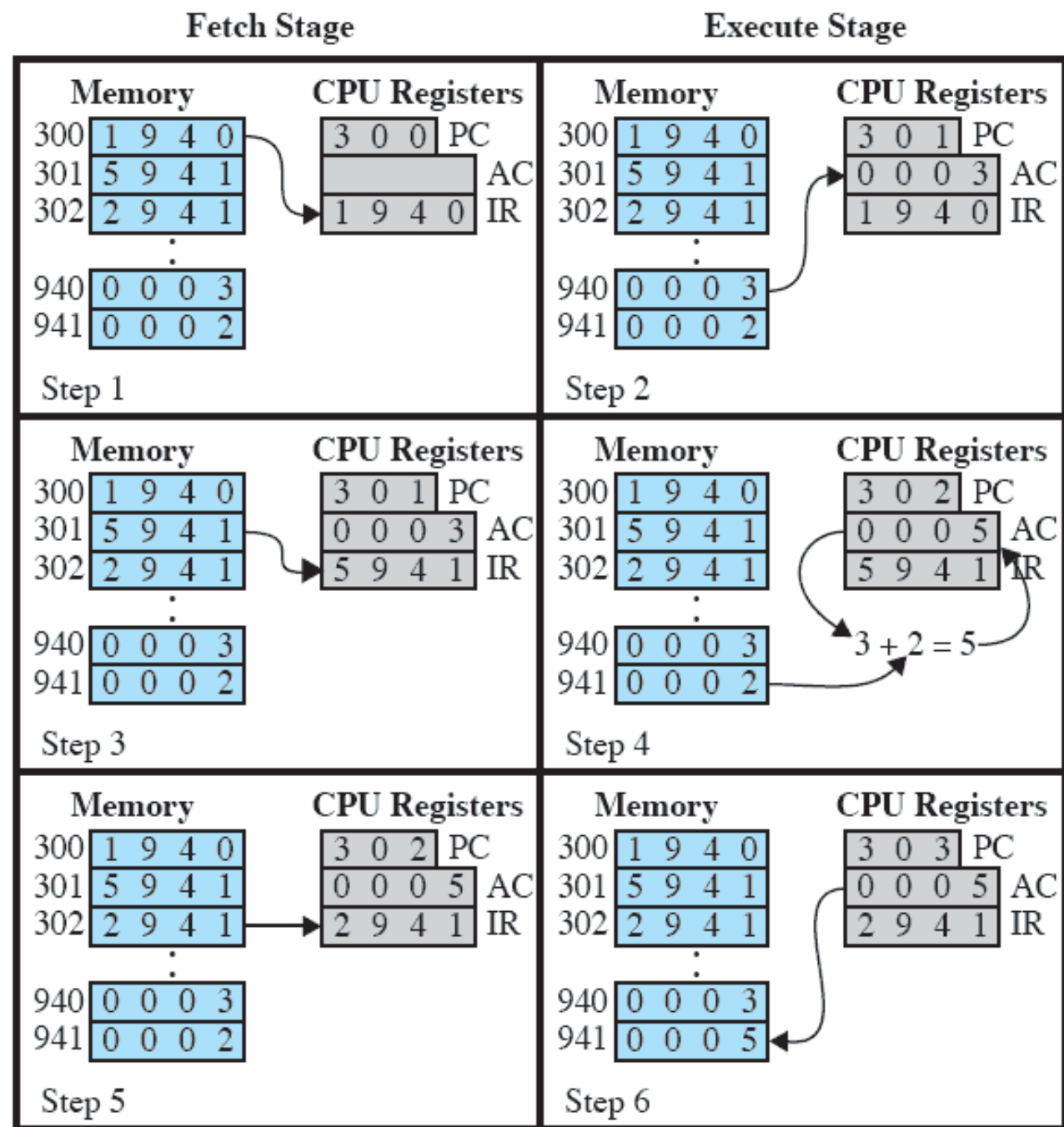
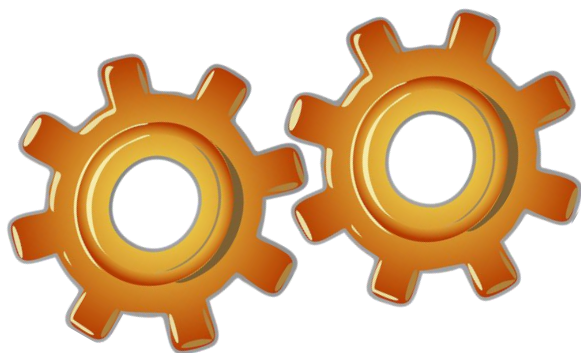


Ejecución de Instrucciones

1. Toma la **siguiente instrucción** de memoria y ponla en **IR** (Instruction Register)
2. Cambia el **PC** (program counter) para apuntar **a la siguiente instrucción**
3. Determinar **el tipo** de instrucción tomada
4. Si la instrucción usa **un word** en memoria **determina dónde está.**
5. **Trae el word**, si se requiere, dentro de un registro de CPU
6. **Ejecuta** la instrucción
7. Ve al paso 1

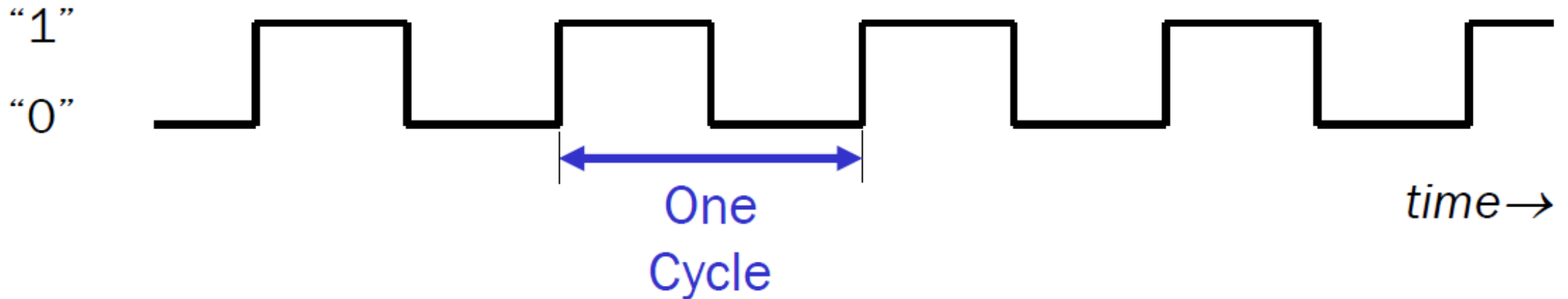


Ejecución de Instrucciones



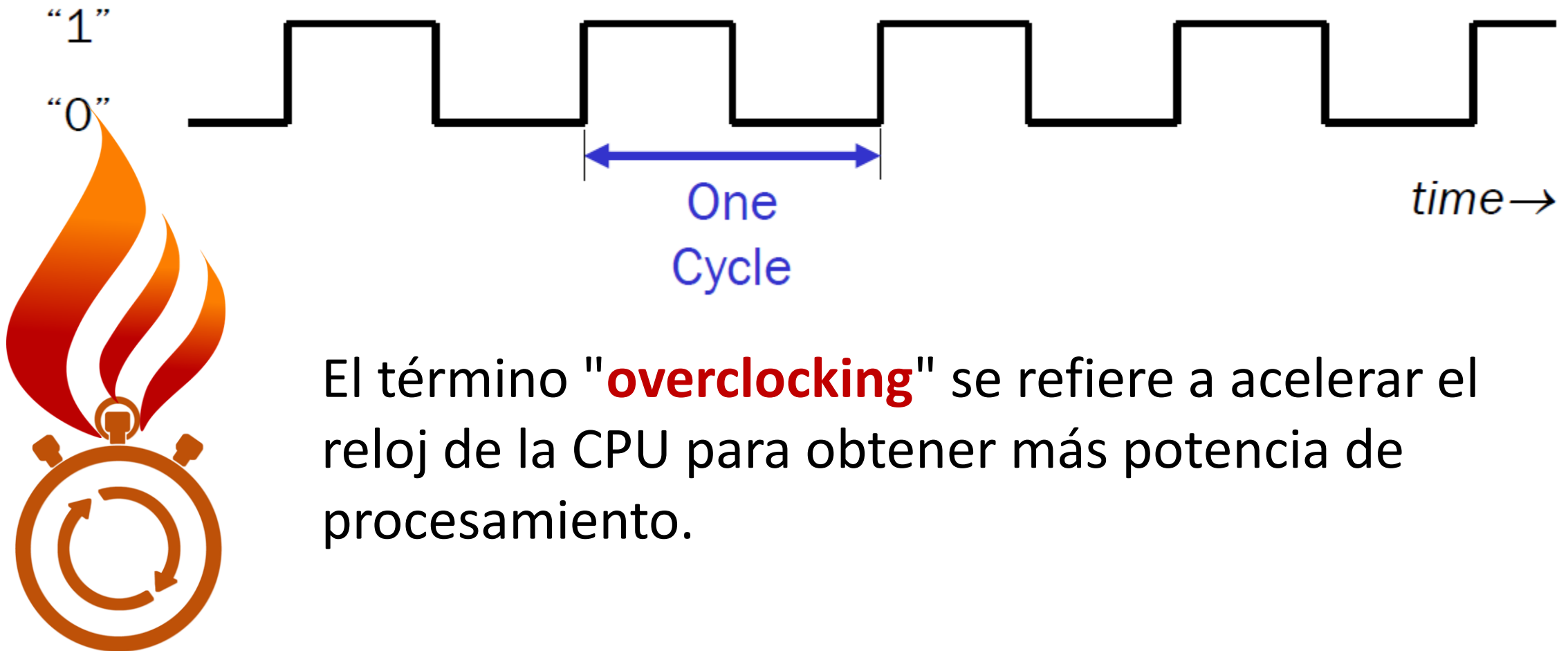
¿Qué es un Circuito de Reloj?

- Un circuito de reloj **dispara la transición** de un estado al **siguiente**
- Velocidad del reloj: **número de ciclos que ejecuta el CPU/s** GHz (gigahercios)



Ejemplo: 3.2 GHz ejecuta 3200 millones de ciclos por segundo

¿Qué es un Circuito de Reloj?



El término "**overclocking**" se refiere a acelerar el reloj de la CPU para obtener más potencia de procesamiento.

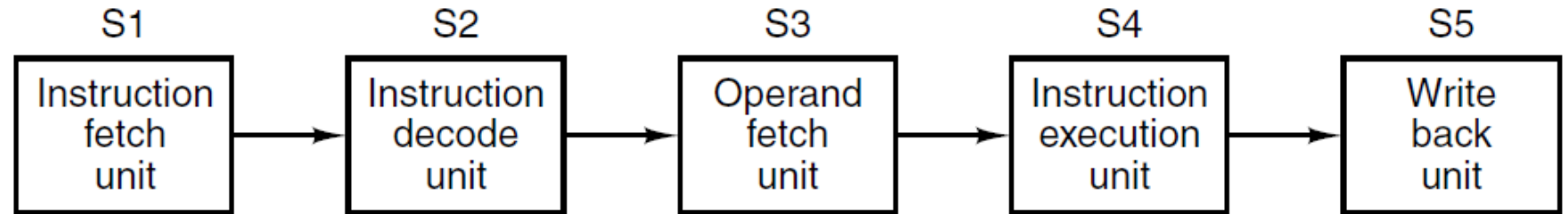
Otra razón de la mejora en el rendimiento del procesador **(Hasta hace 15 años)**

- Paralelismo a nivel instrucción



Paralelismo a nivel instrucción

Pipeline



(a)

A: add a, b

B: mul c, d

C: div e, f

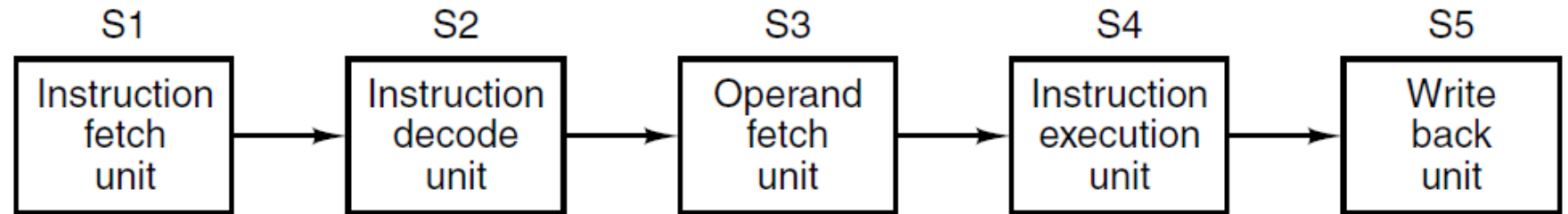
D: add g, h

E: mul i, j

F: div k, l

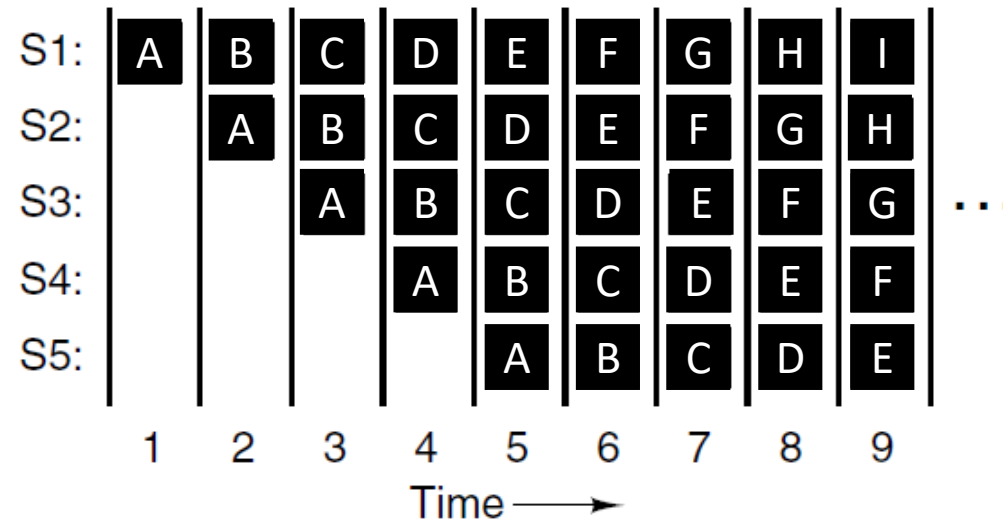
Paralelismo a nivel instrucción

Pipeline



(a)

A: add a, b
B: mul c, d
C: div e, f
D: add g, h
E: mul i, j
F: div k, l

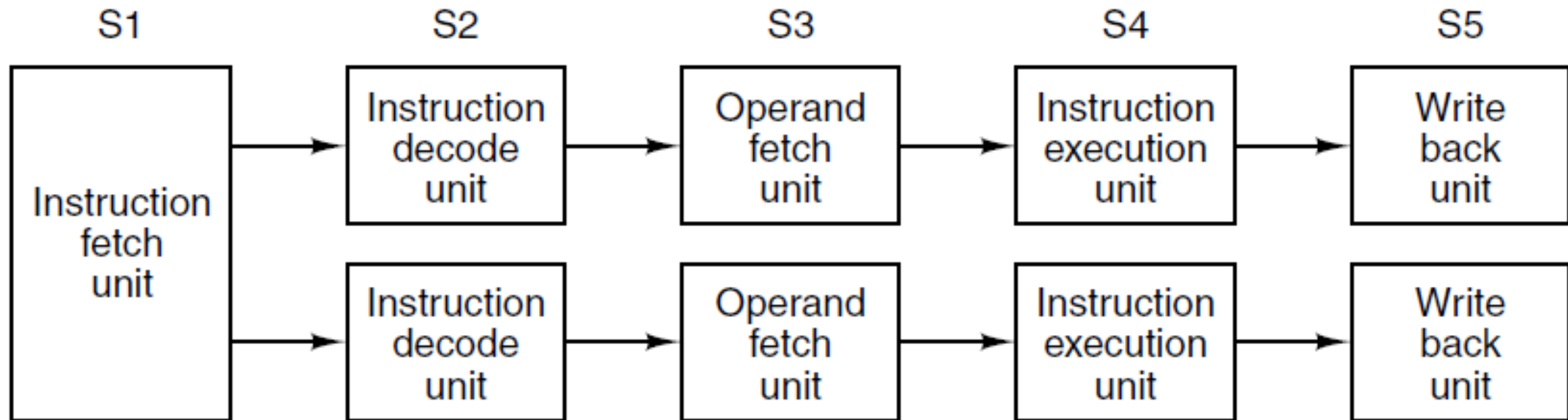


(b)

(a) A five-stage pipeline. (b) The state of each stage as a function of time. Nine clock cycles are illustrated.

Paralelismo a nivel instrucción

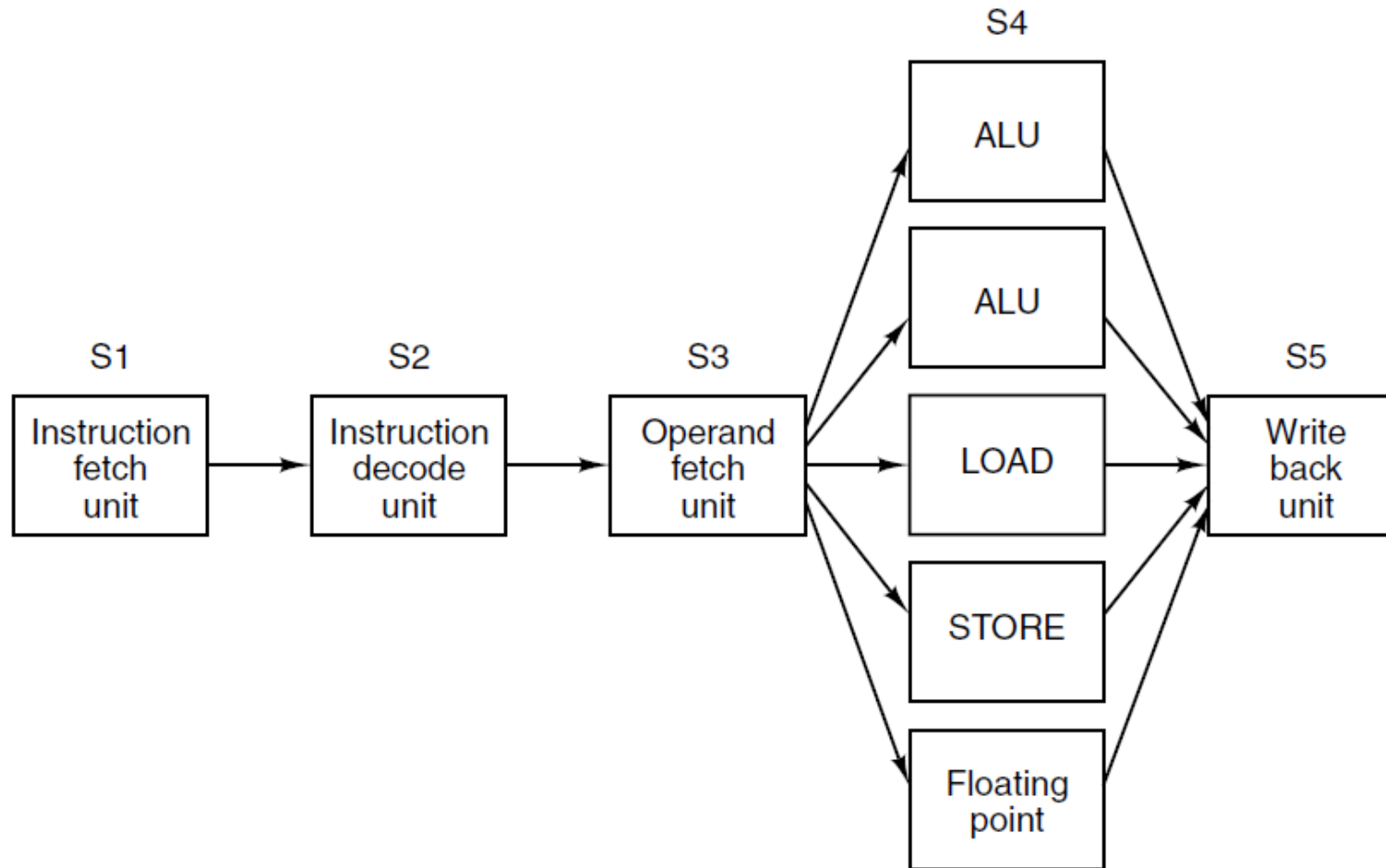
Arquitecturas Superescalares (antiguas)



Dual five-stage pipelines with a common instruction fetch unit.

Paralelismo a nivel instrucción

Arquitecturas Superscalares (nuevas)



· A superscalar processor with five functional units.

Ejemplo de Ejecución:

Una instrucción a la vez

Considera el siguiente programa

$$a = x * x + y * y + z * z$$

Asume que R0 = x, R1 = y, R2 = z

```
1 mul R0, R0, R0
2 mul R1, R1, R1
3 mul R2, R2, R2
4 add R0, R0, R1
5 add R3, R0, R2
```

R3 almacena el valor del programa en la variable a

Ejemplo de Ejecución:

Una instrucción a la vez

Considera el siguiente programa

$$a = x^*x + y^*y + z^*z$$

Asume que $R0 = x$, $R1 = y$, $R2 = z$

```
1 mul R0, R0, R0
2 mul R1, R1, R1
3 mul R2, R2, R2
4 add R0, R0, R1
5 add R3, R0, R2
```

R3 almacena el valor del programa en la variable α

Voluntario 1

Voluntario 2

Ejemplo de Ejecución:

Una instrucción a la vez

Considera el siguiente programa

$$a = x * x + y * y + z * z$$

Asume que R0 = x, R1 = y, R2 = z

```
1 mul R0, R0, R0
2 mul R1, R1, R1
3 mul R2, R2, R2
4 add R0, R0, R1
5 add R3, R0, R2
```

R3 almacena el valor del programa en la variable **a**

Voluntario 1

mul R0, R0, R0

mul R2, R2, R2

add R3, R0, R2

Voluntario 2

mul R1, R1, R1

add R0, R0, R1

Ejemplo de Ejecución:

Una instrucción a la vez

Considera el siguiente programa

$$a = x^*x + y^*y + z^*z$$

Asume que $R0 = x$, $R1 = y$, $R2 = z$

```
1 mul R0, R0, R0
2 mul R1, R1, R1
3 mul R2, R2, R2
4 add R0, R0, R1
5 add R3, R0, R2
```

R3 almacena el valor del programa en la variable *a*

Vol 1

Vol 2

Vol 3

Ejemplo de Ejecución:

Una instrucción a la vez

Considera el siguiente programa

$$a = x * x + y * y + z * z$$

Asume que R0 = x, R1 = y, R2 = z

```
1 mul R0, R0, R0
2 mul R1, R1, R1
3 mul R2, R2, R2
4 add R0, R0, R1
5 add R3, R0, R2
```

R3 almacena el valor del programa en la variable *a*

Vol 1

Vol 2

Vol 3

mul R0, R0, R0

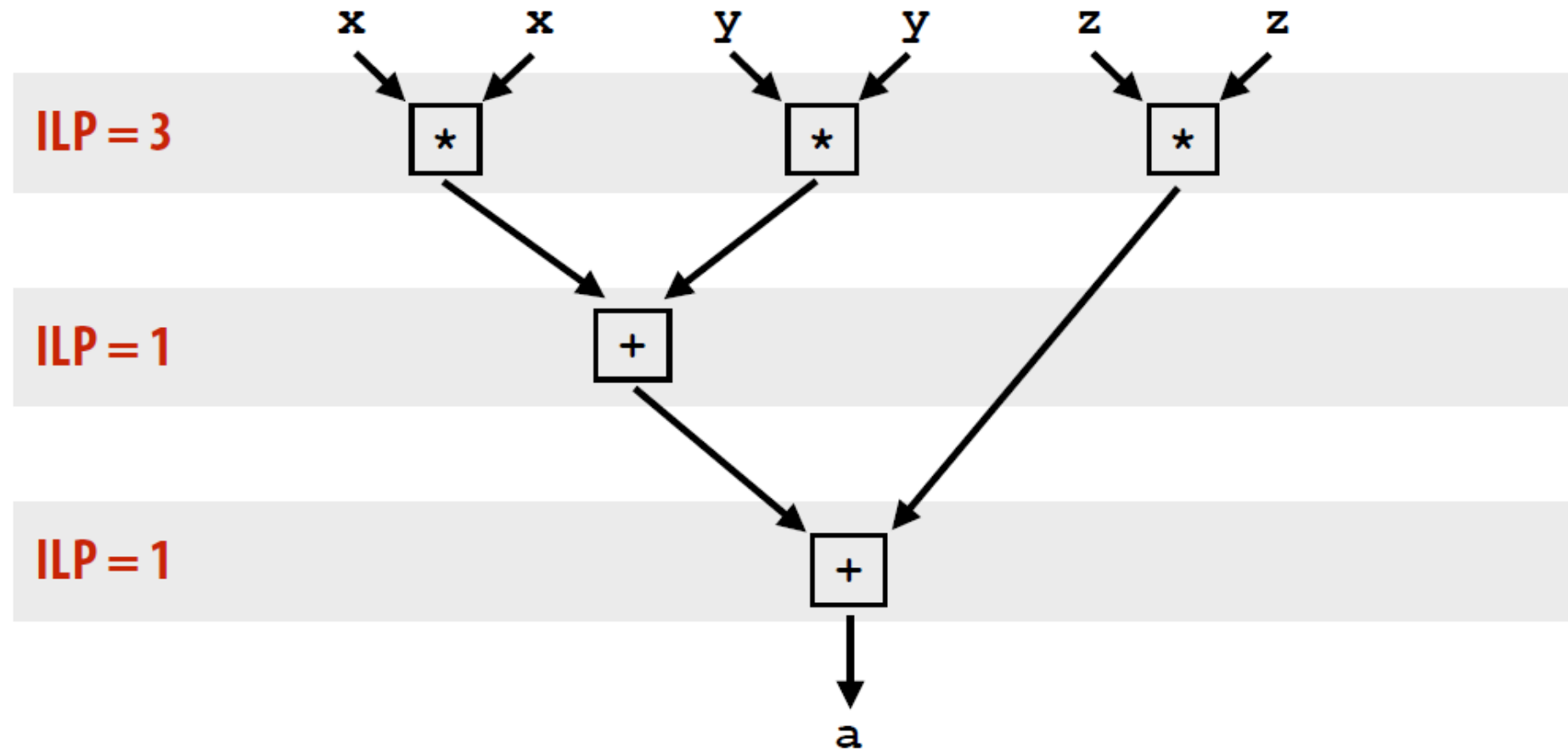
mul R1, R1, R1

mul R2, R2, R2

add R0, R0, R1

add R3, R0, R2

Ejemplo de paralelismo a nivel instrucción – 3 pipelines



Un ejemplo más complejo

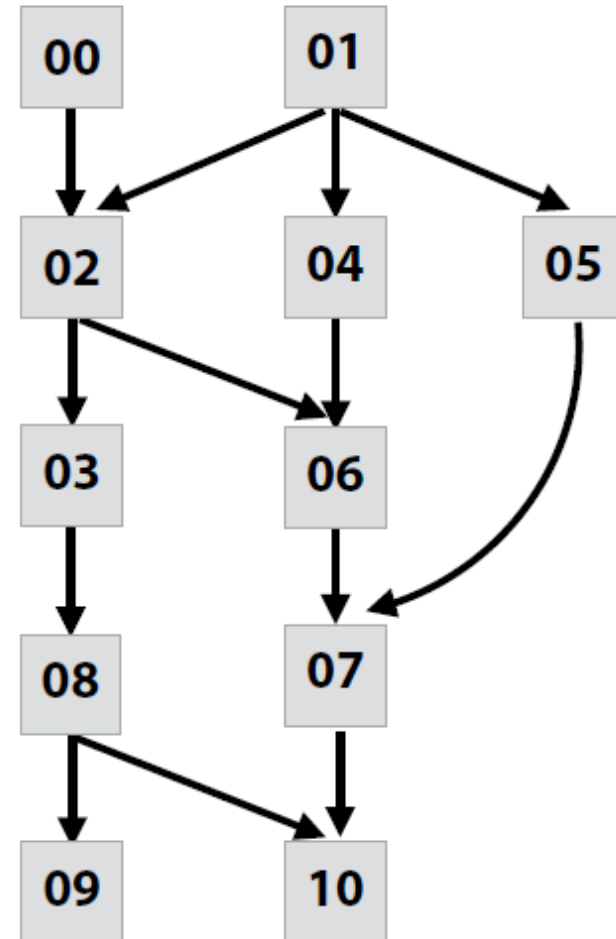
Grafo de Dependencias

PC	Instruction
00	a = 2
01	b = 4
02	tmp2 = a + b
03	tmp3 = tmp2 + a
04	tmp4 = b + b
05	tmp5 = b * b
06	tmp6 = tmp2 + tmp4
07	tmp7 = tmp5 + tmp6
08	if (tmp3 > 7)
09	print tmp3
	else
10	print tmp7

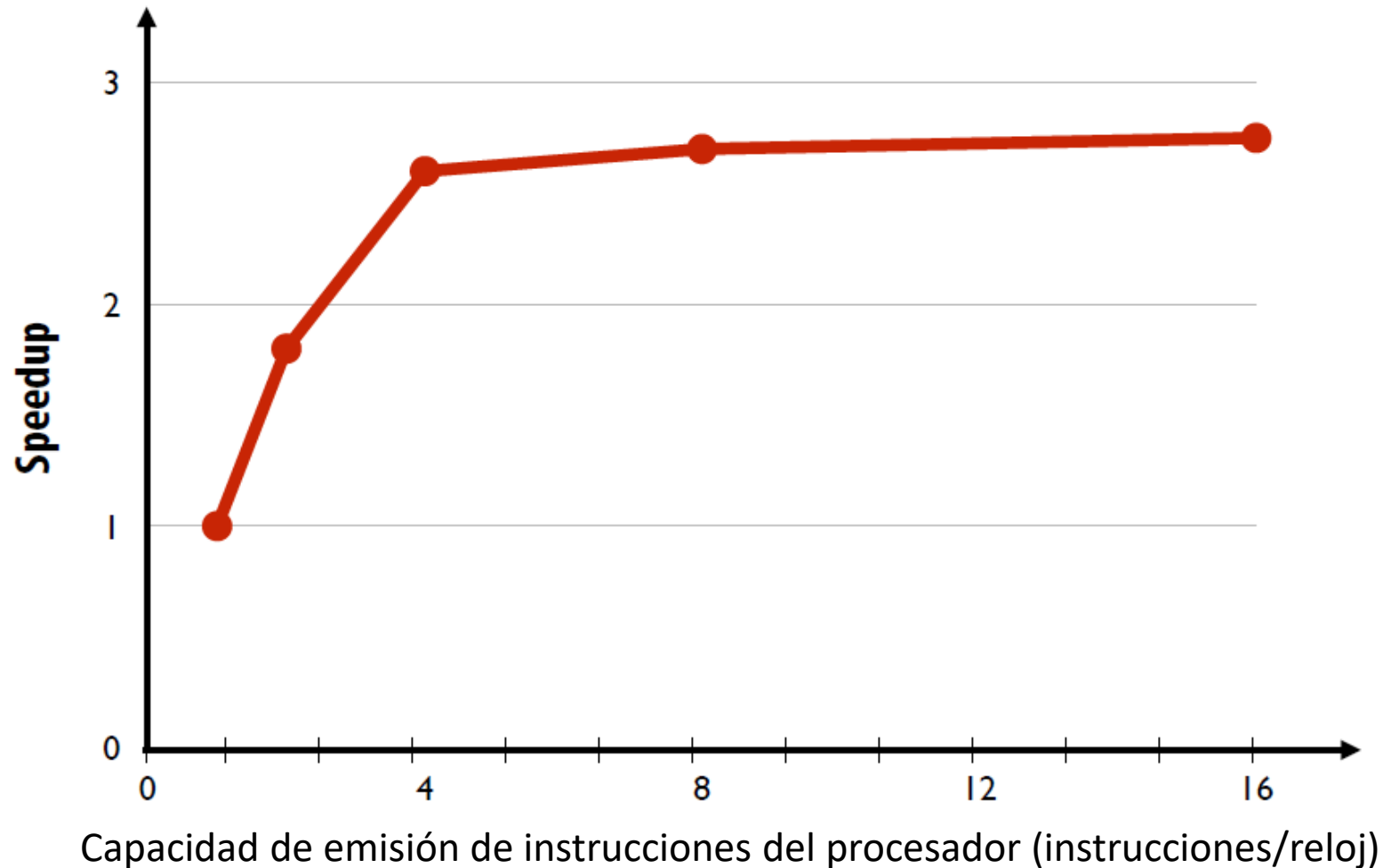
Un ejemplo más complejo

PC	Instruction
00	a = 2
01	b = 4
02	tmp2 = a + b
03	tmp3 = tmp2 + a
04	tmp4 = b + b
05	tmp5 = b * b
06	tmp6 = tmp2 + tmp4
07	tmp7 = tmp5 + tmp6
08	if (tmp3 > 7)
09	print tmp3
	else
10	print tmp7

Grafo de Dependencias



El rendimiento de la ejecución **superescalar** decrece

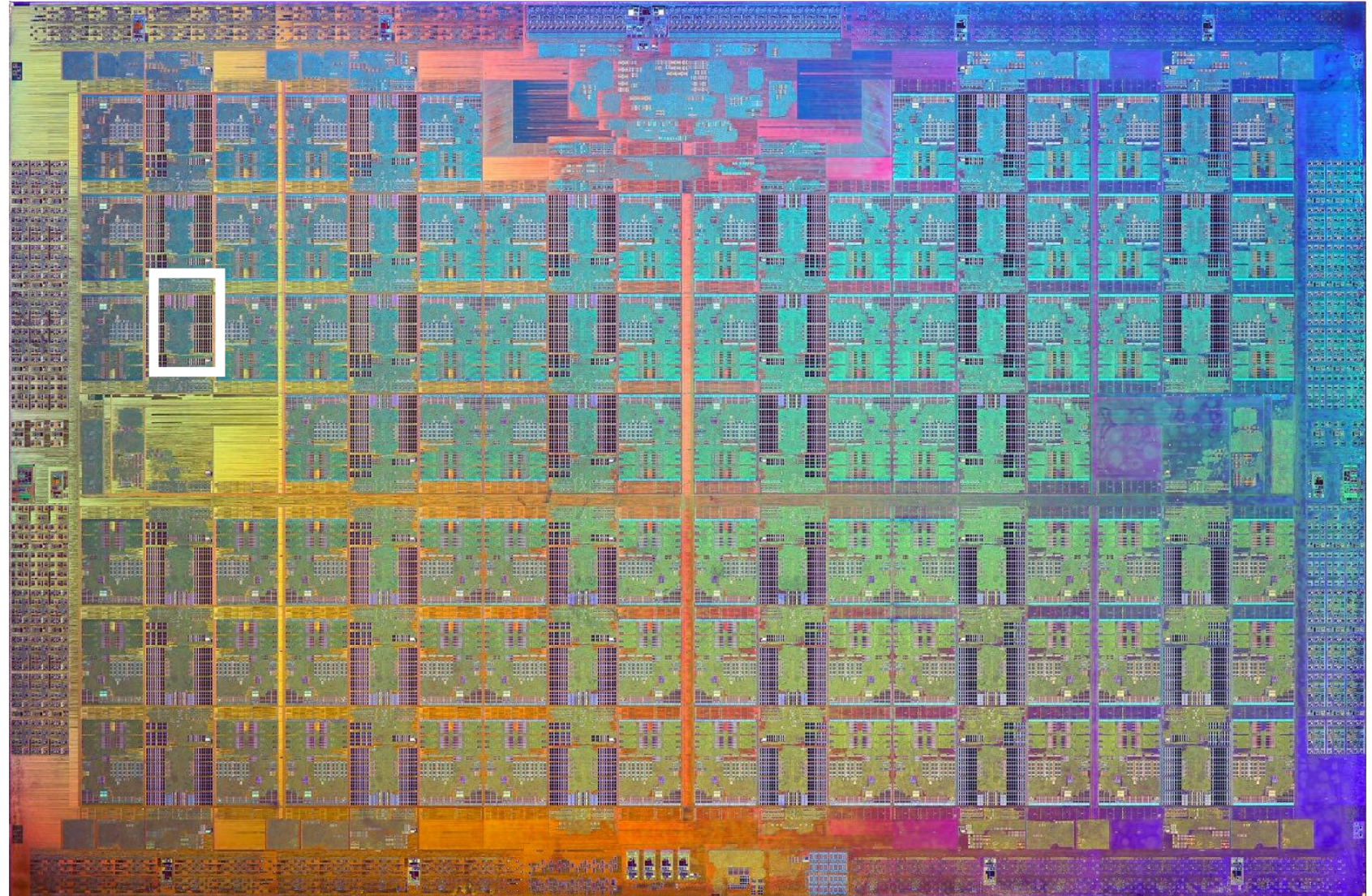




Ejemplos de Arquitecturas Paralelas

Intel Xeon Phi 7290 (2016)

72 cores (1.5 Ghz)

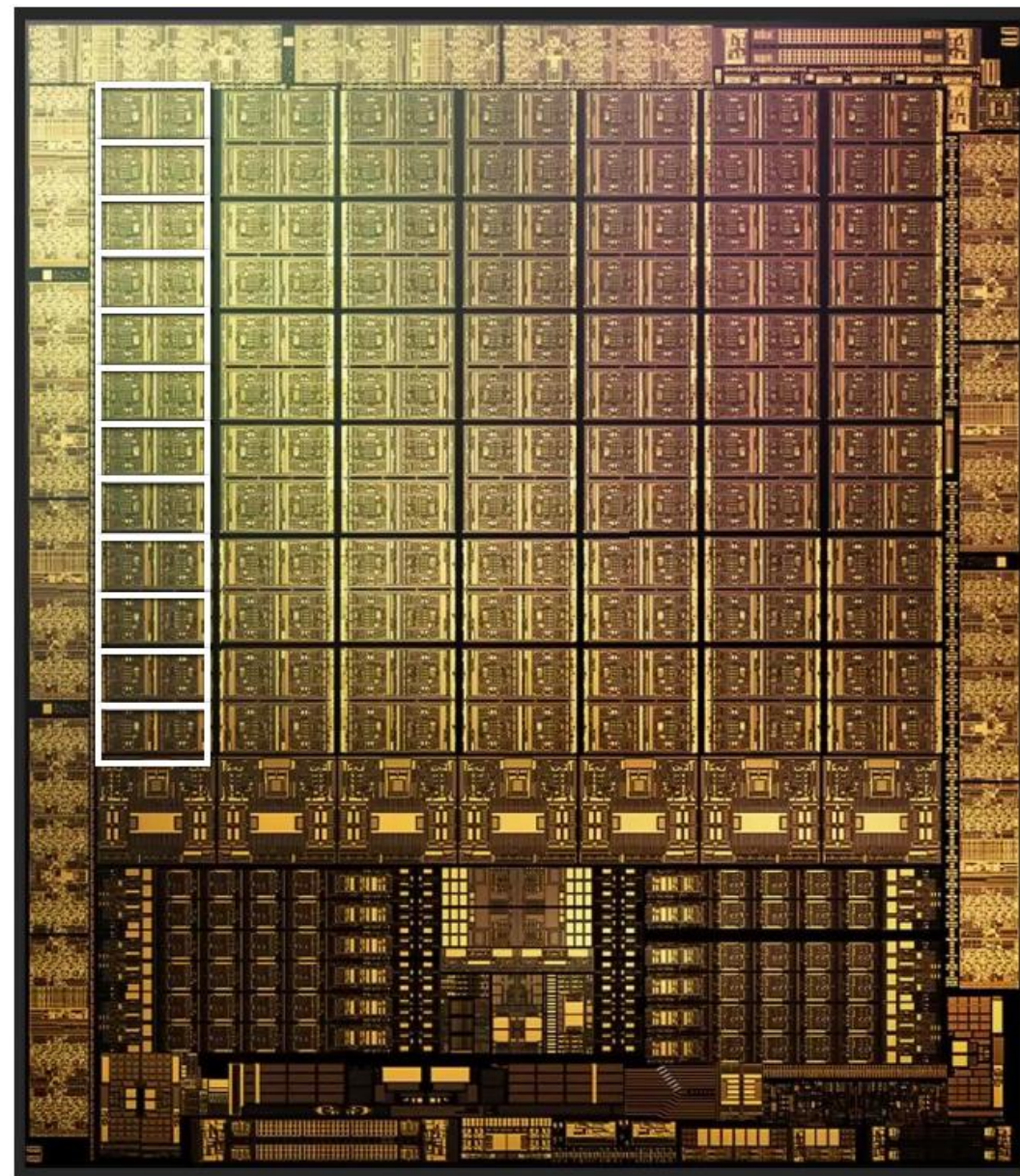


NVIDIA Ampere GA102 GPU GeForce RTX 3080 (2020)

10,752 CUDA Cores

84 second-generation RT Cores

336 third-generation Tensor Cores



Supercómputo

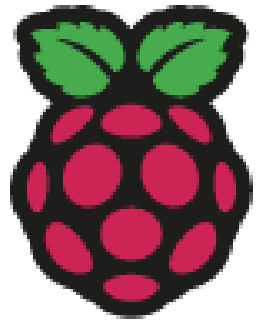
Oak Ridge National Laboratory: Summit

9,216 x 22-core IBM Power9 CPUs **+** **27,648 NVIDIA Volta GPUs**



Procesamiento paralelo móvil

Raspberry Pi 3 Quad-core ARM A53 CPU



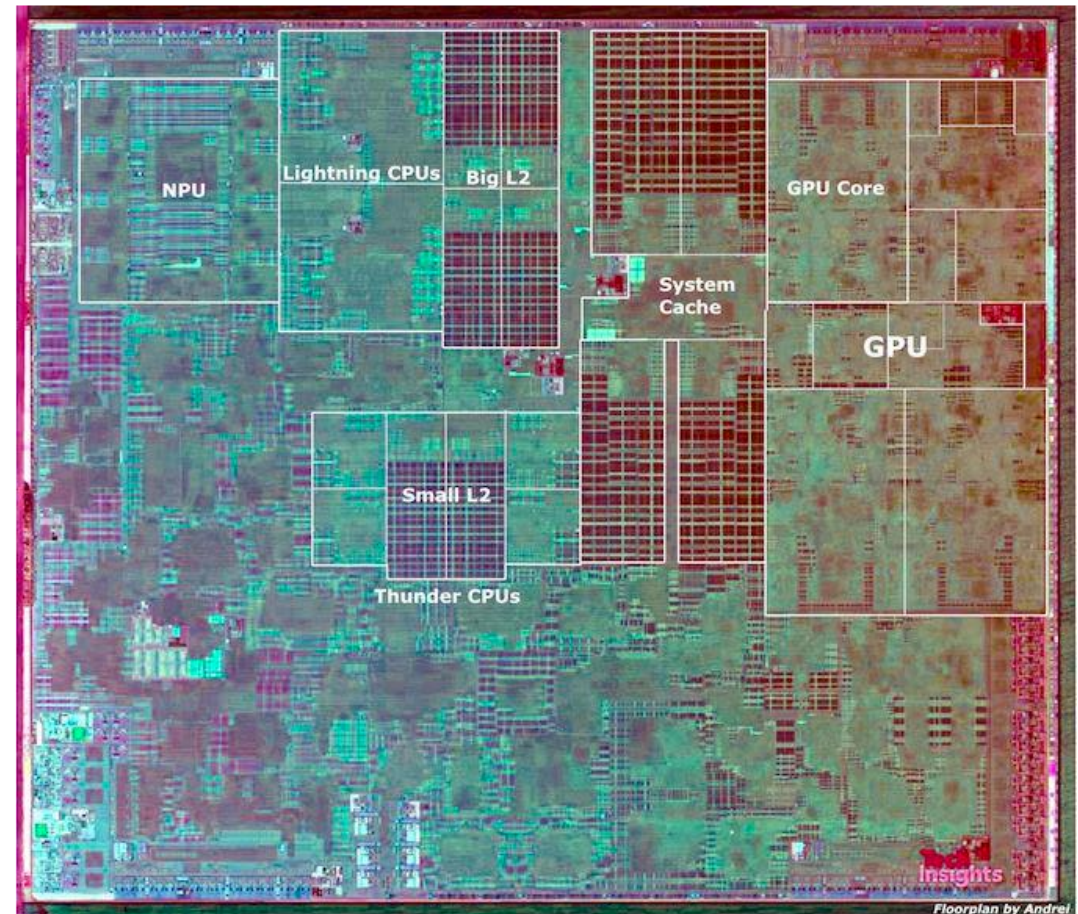
Raspberry Pi
Foundation



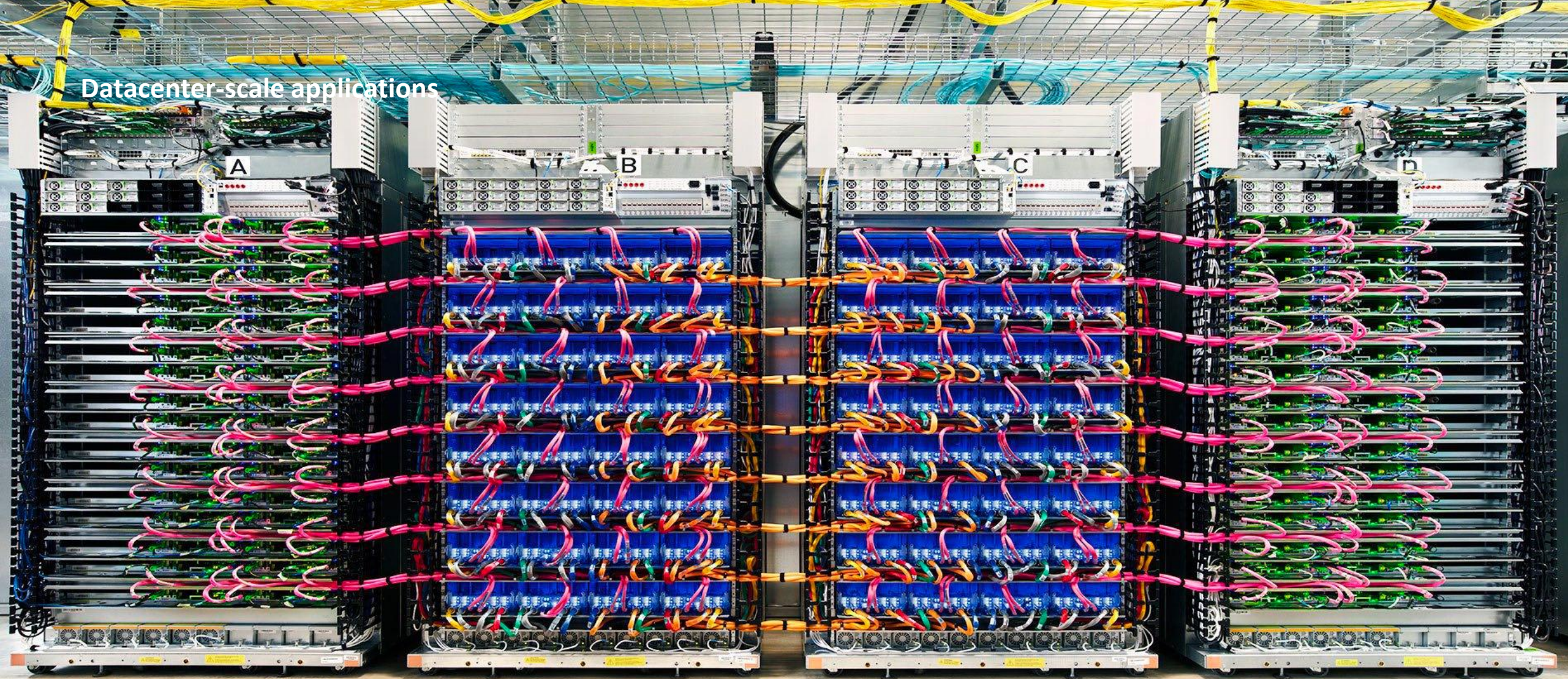
Procesamiento paralelo móvil

Apple A13 Bionic (in iPhone 11)

2 “big” CPU cores +
4 “small” CPU cores +
Apple-designed multi-core GPU +
Image processor +
Neural Engine for DNN acceleration +
Motion processor



Datacenter-scale applications



Google TPU pods

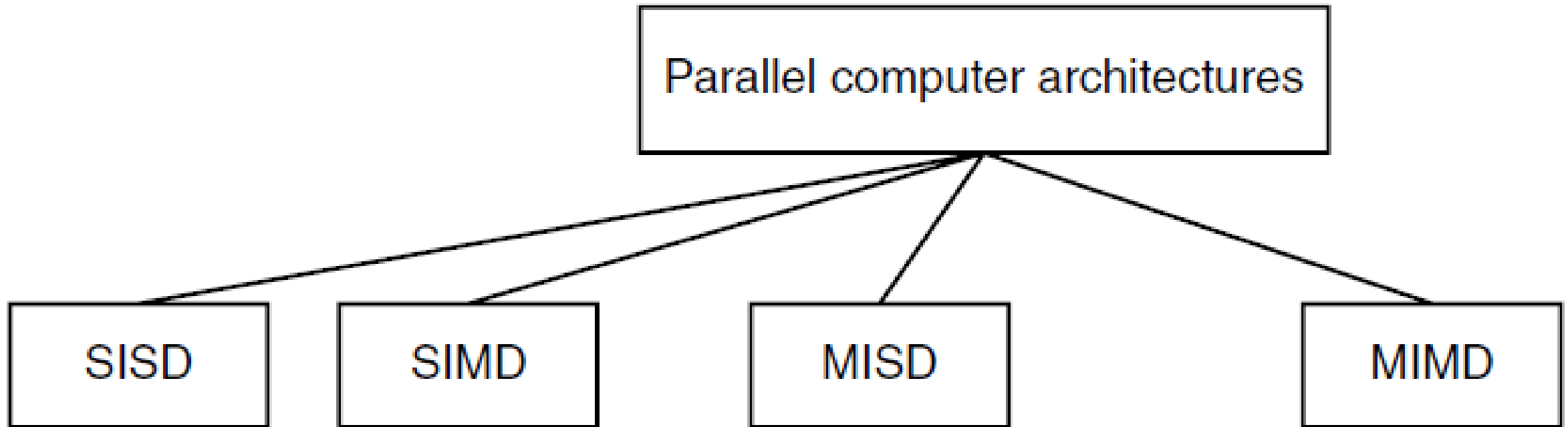
TPU = Tensor Processing Unit



Supercomputadoras

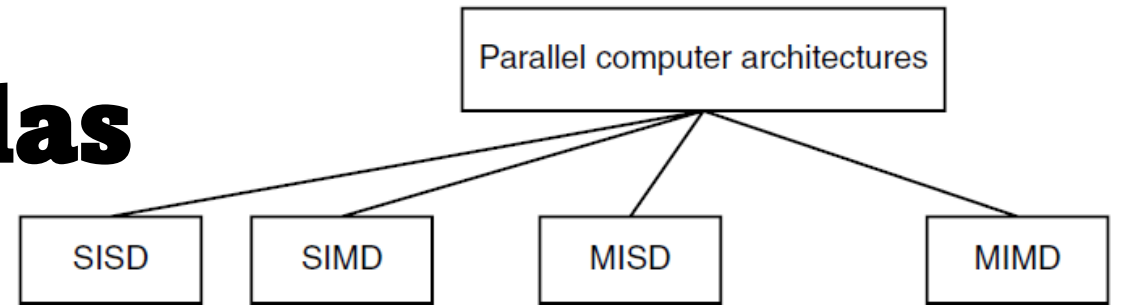
top500.org

Taxonomía de Computadoras Paralelas de Flynn



Von Neumann

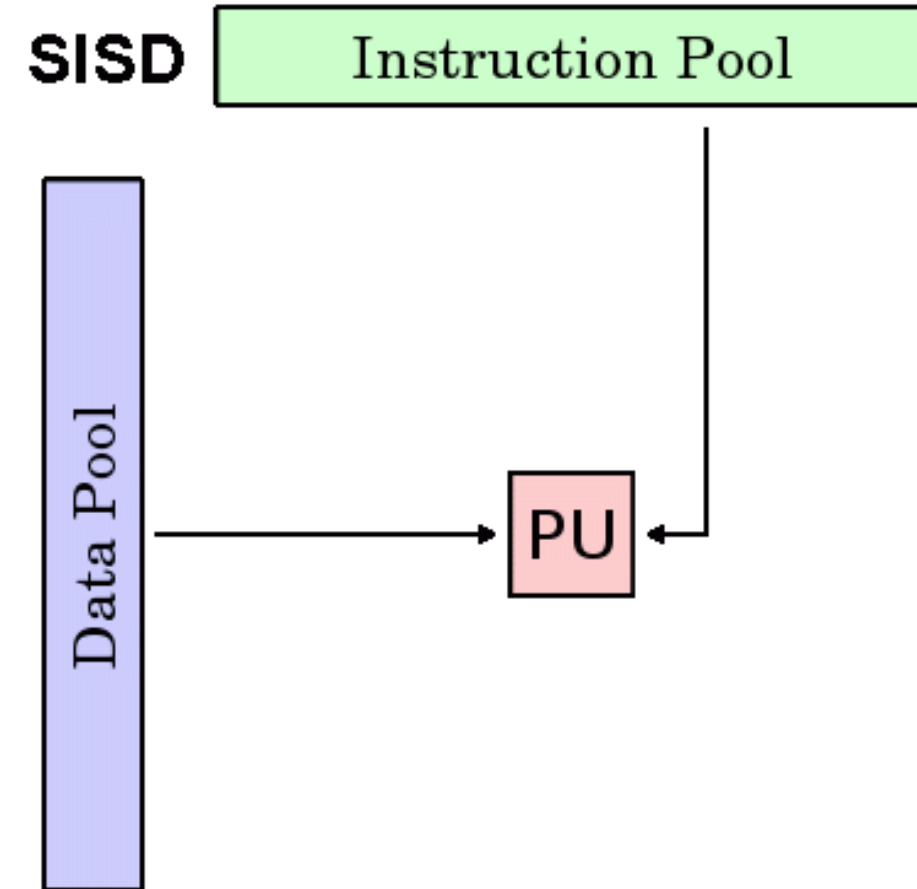
Computadoras Paralelas



- **SISD**: Single Instruction stream, Single Data stream
- **SIMD**: Single Instruction stream, Multiple Data streams
- **MISD**: Multiple Instruction streams, Single Data stream
- **MIMD**: Multiple Instruction streams, Multiple Data streams

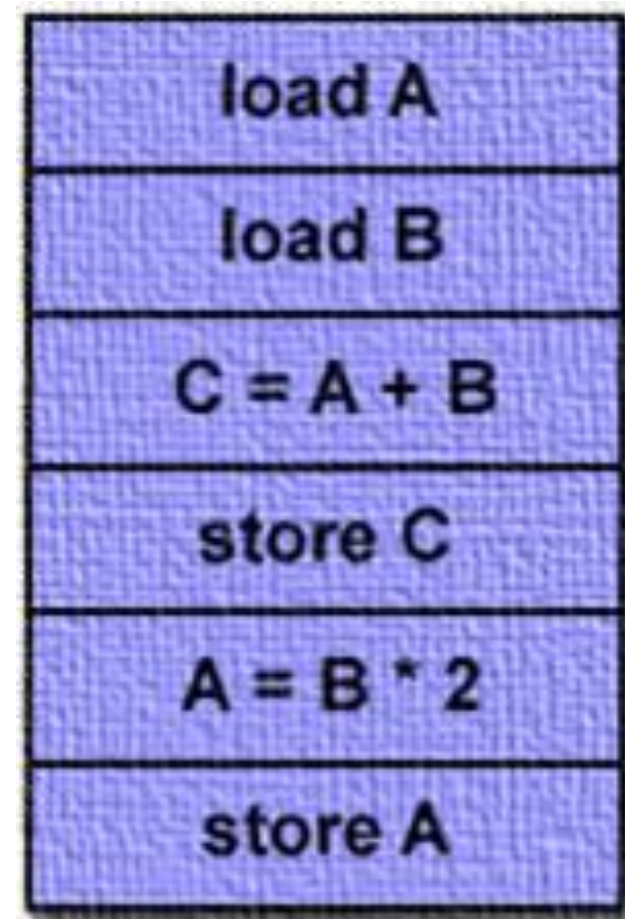
Single Instruction stream, Single Data stream

- Una computadora **serial**
- Ejecución determinista
- **Instrucción única**: la CPU solo actúa sobre un flujo de instrucciones durante cualquier ciclo de reloj
- **Dato único**: solo se utiliza un flujo de datos como entrada durante cualquier ciclo de reloj
- Este es el tipo de computadora más antiguo



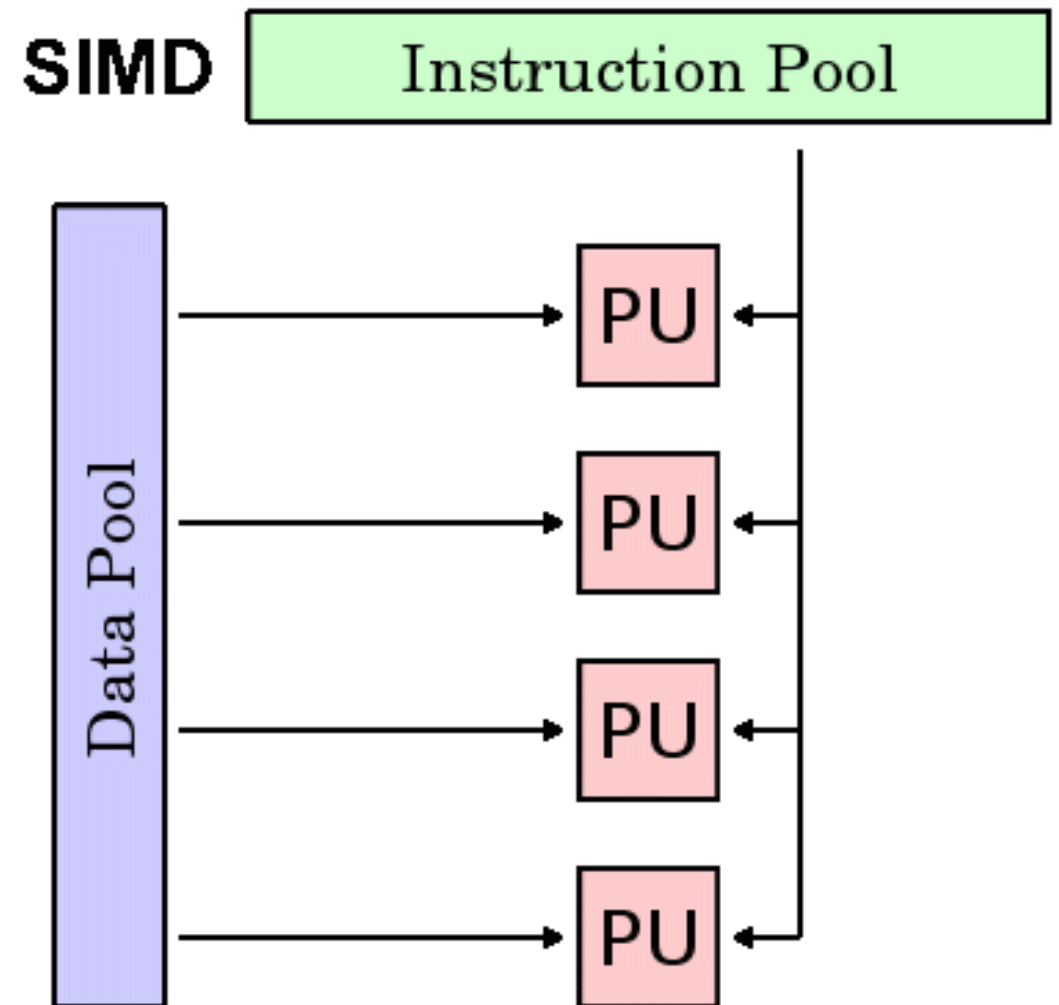
Single Instruction stream, Single Data stream

- Una computadora **serial**
- Ejecución determinista
- **Instrucción única**: la CPU solo actúa sobre un flujo de instrucciones durante cualquier ciclo de reloj
- **Dato único**: solo se utiliza un flujo de datos como entrada durante cualquier ciclo de reloj
- Este es el tipo de computadora más antiguo



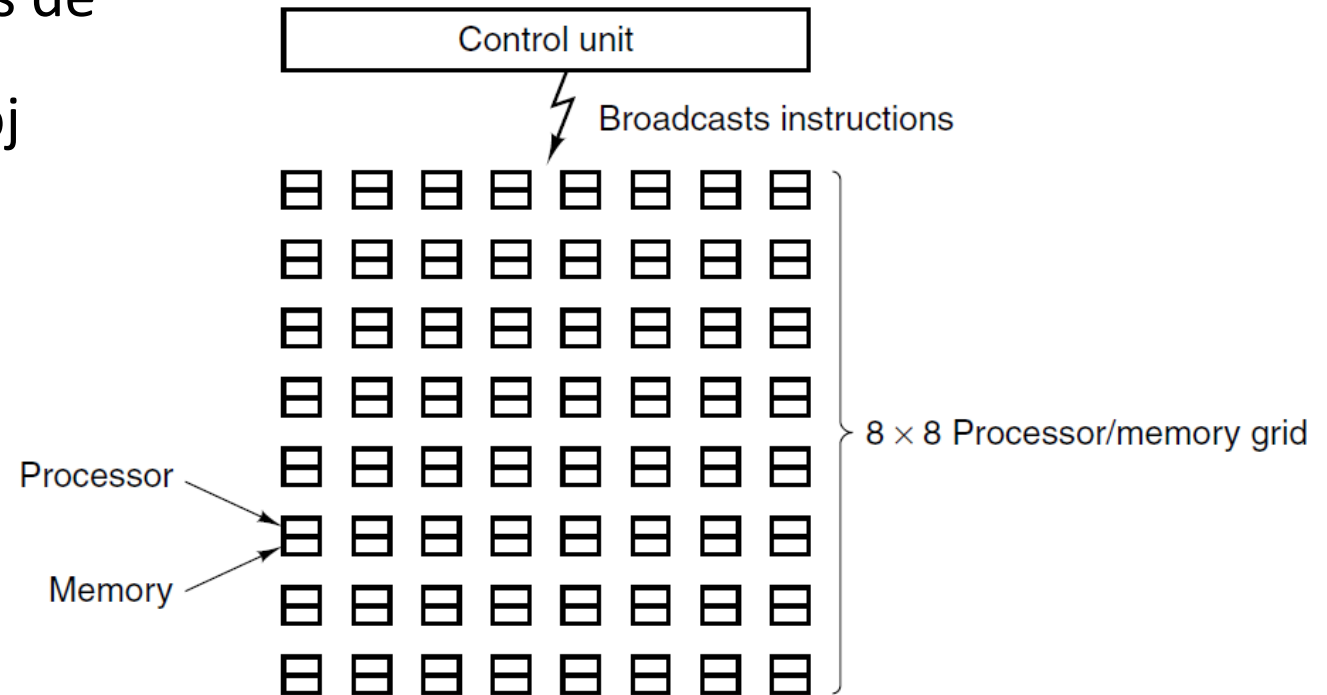
Single Instruction stream, Multiple Data streams

- **Instrucción única**: todas las unidades de procesamiento ejecutan la misma instrucción en cualquier ciclo de reloj dado
- **Datos múltiples**: cada unidad de procesamiento puede operar en un elemento de datos diferente
- Ejecución síncrona y determinista
- Se utiliza para procesamiento matricial y de gráficos/imágenes

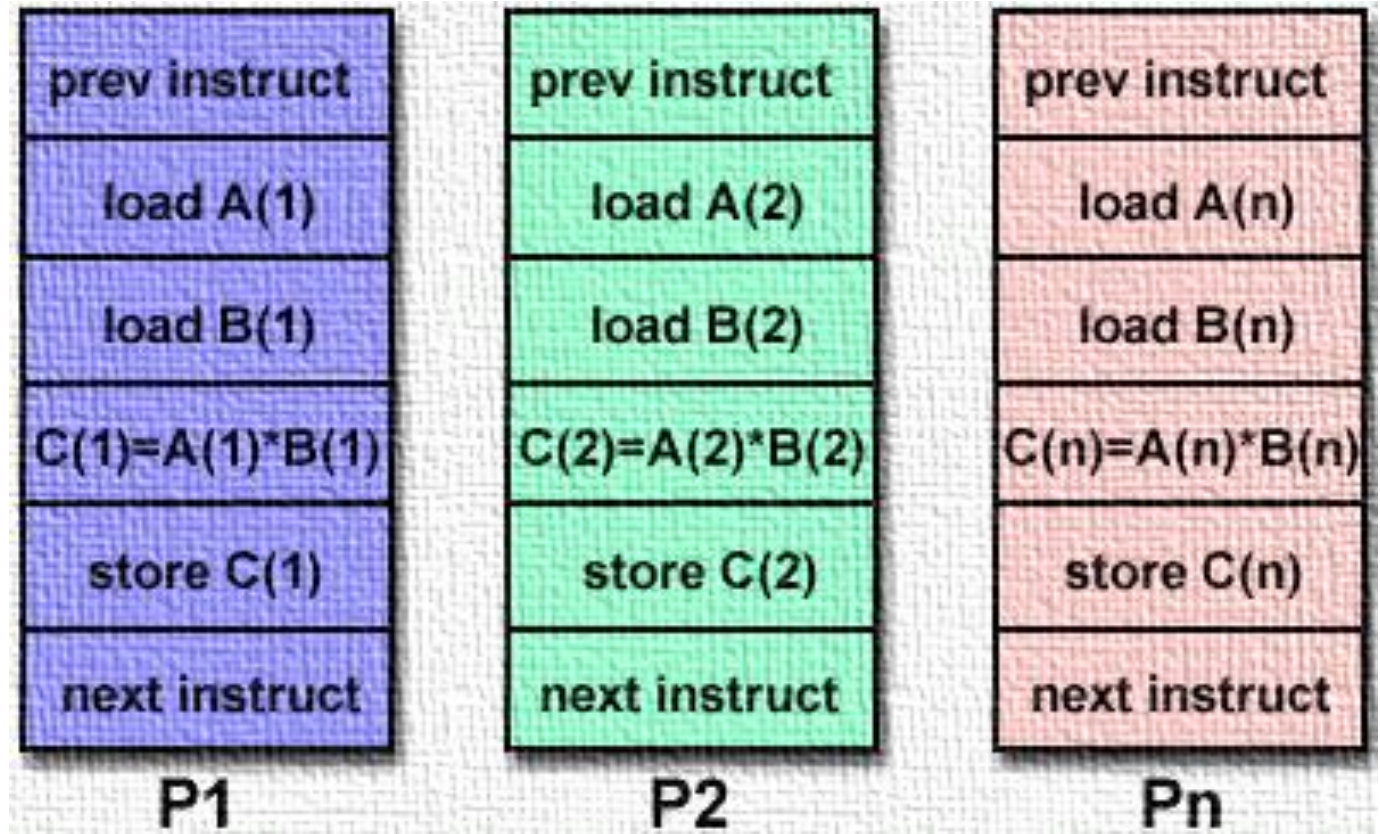
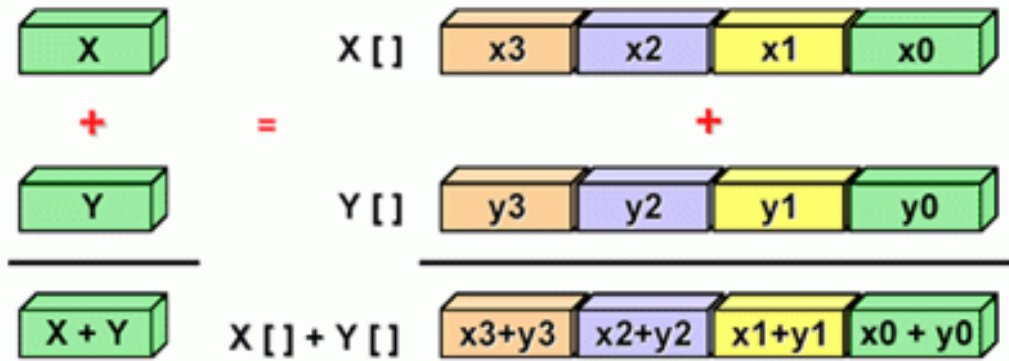


Single Instruction stream, Multiple Data streams

- **Instrucción única**: todas las unidades de procesamiento ejecutan la misma instrucción en cualquier ciclo de reloj dado
- **Datos múltiples**: cada unidad de procesamiento puede operar en un elemento de datos diferente
- Ejecución síncrona y determinista
- Se utiliza para procesamiento matricial y de gráficos/imágenes

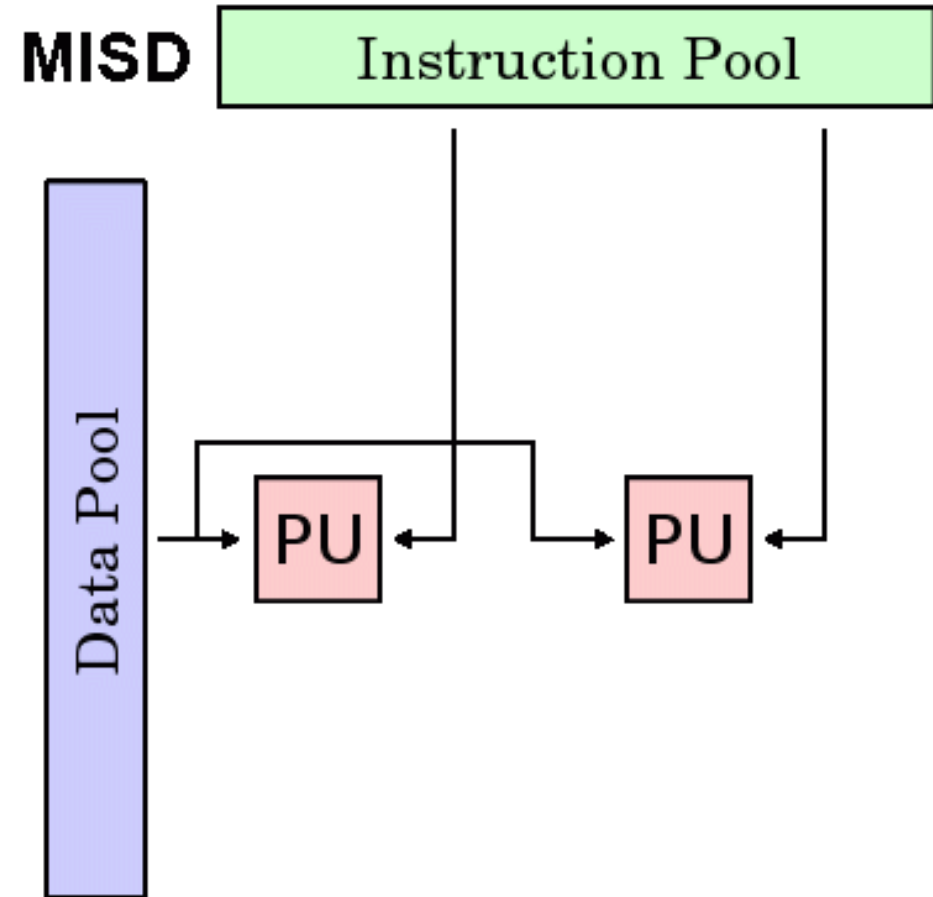


Single Instruction stream, Multiple Data streams



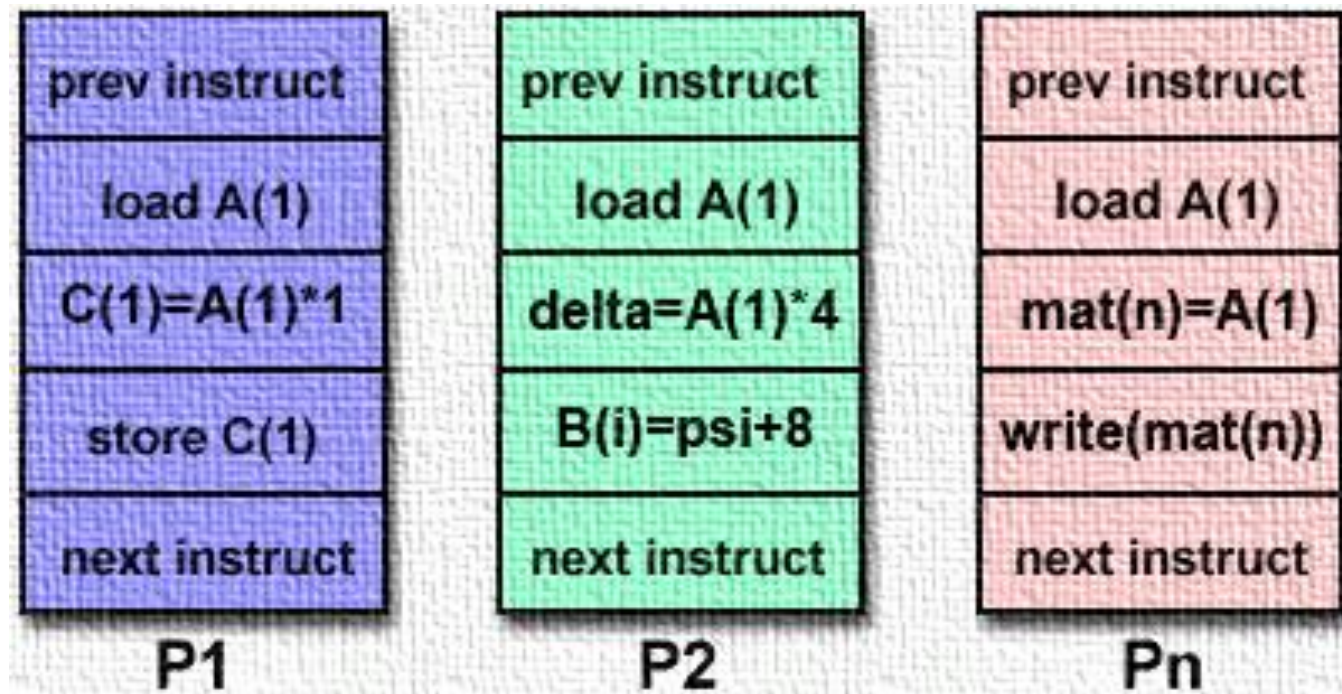
Multiple Instruction streams, Single Data stream

- **Instrucciones múltiples:** cada unidad de procesamiento opera con los datos de forma independiente a través de flujos de instrucciones separados.
- **Dato único:** un único flujo de datos se alimenta a múltiples unidades de procesamiento.
- Pocos ejemplos reales de esta clase de computadora paralela.
- Algunos usos:
 - Aplicar múltiples filtros a una misma imagen,
 - Usar múltiples algoritmos de criptografía para descifrar un solo mensaje.



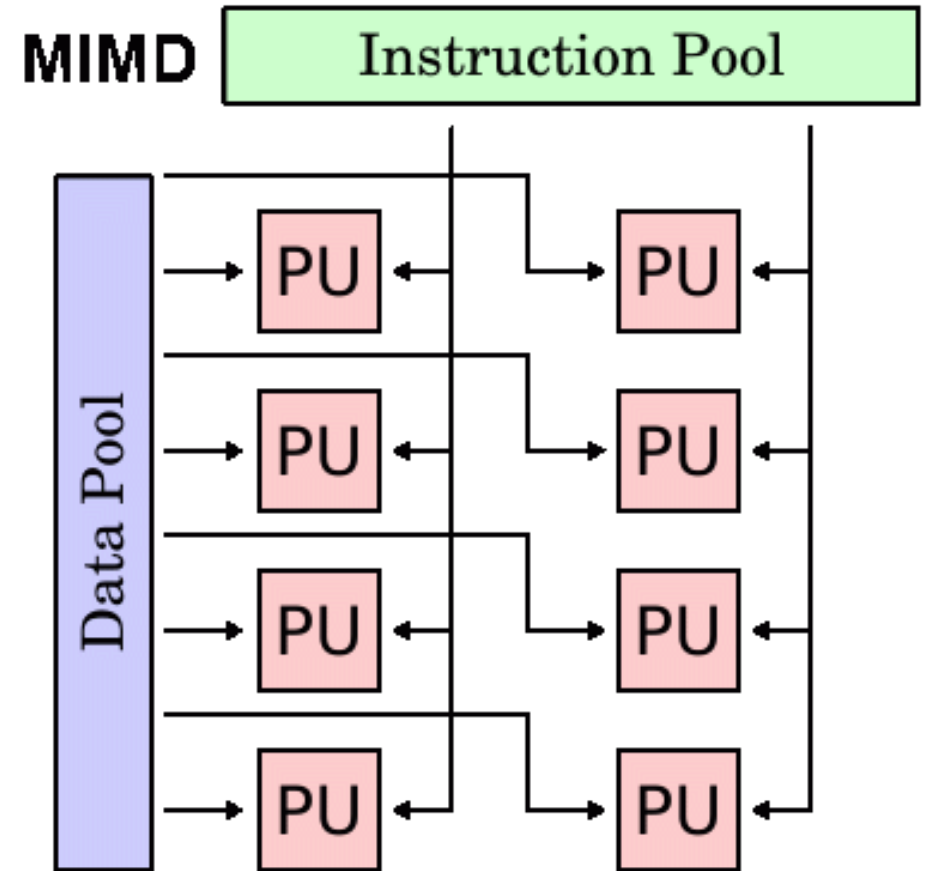
Multiple Instruction streams, Single Data stream

- **Instrucciones múltiples:** cada unidad de procesamiento opera con los datos de forma independiente a través de flujos de instrucciones separados.
- **Dato único:** un único flujo de datos se alimenta a múltiples unidades de procesamiento.
- Pocos ejemplos reales de esta clase de computadora paralela.
- Algunos usos:
 - Aplicar múltiples filtros a una misma imagen,
 - Usar múltiples algoritmos de criptografía para descifrar un solo mensaje.



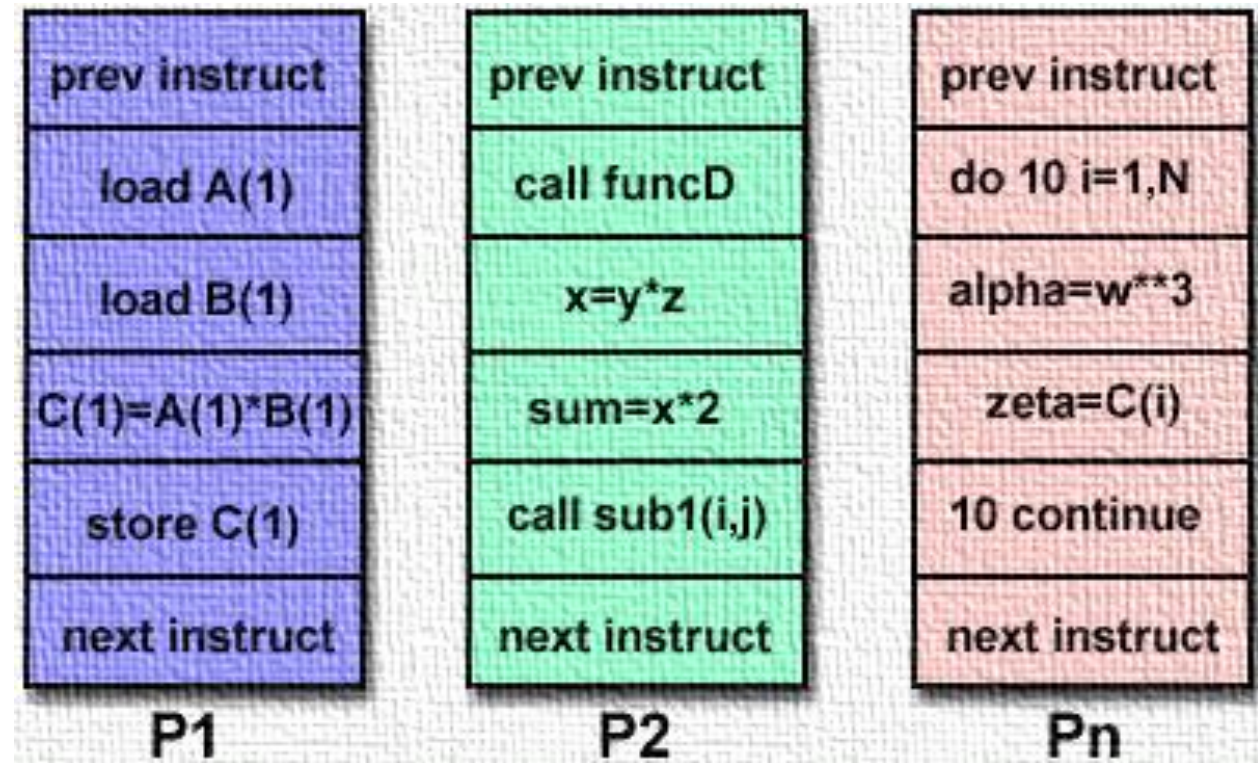
Multiple Instruction streams, Multiple Data streams

- **Instrucciones múltiples:** cada procesador puede estar ejecutando un flujo de instrucciones diferente
- **Datos múltiples:** cada procesador puede estar trabajando con un flujo de datos diferente
- La ejecución puede ser síncrona o asíncrona, determinista o no determinista
- Actualmente el tipo más común
- Muchas arquitecturas MIMD también incluyen subcomponentes de ejecución SIMD



Multiple Instruction streams, Multiple Data streams

- **Instrucciones múltiples:** cada procesador puede estar ejecutando un flujo de instrucciones diferente
- **Datos múltiples:** cada procesador puede estar trabajando con un flujo de datos diferente
- La ejecución puede ser síncrona o asíncrona, determinista o no determinista
- Actualmente el tipo más común
- Muchas arquitecturas MIMD también incluyen subcomponentes de ejecución SIMD



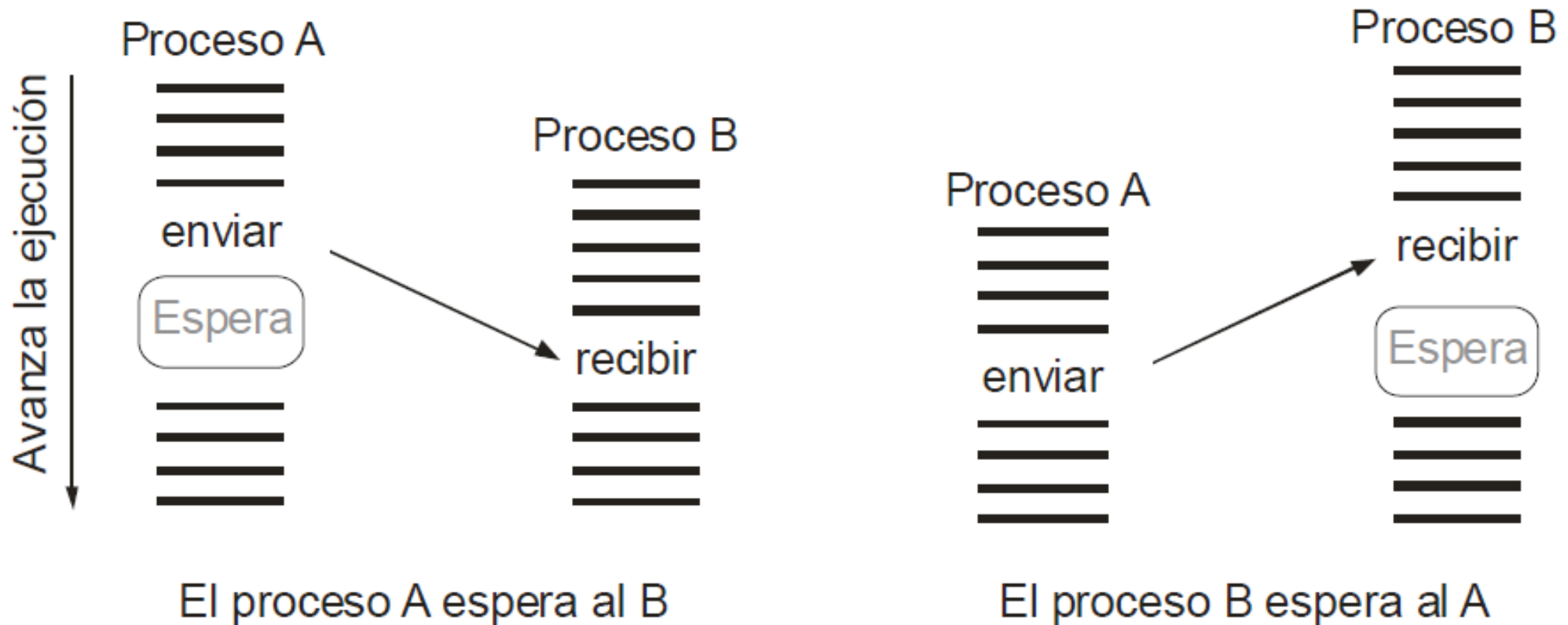


Algunos conceptos importantes relacionados a las Computadoras Paralelas

- **(A)**Sincronía
- **(No)**Determinismo

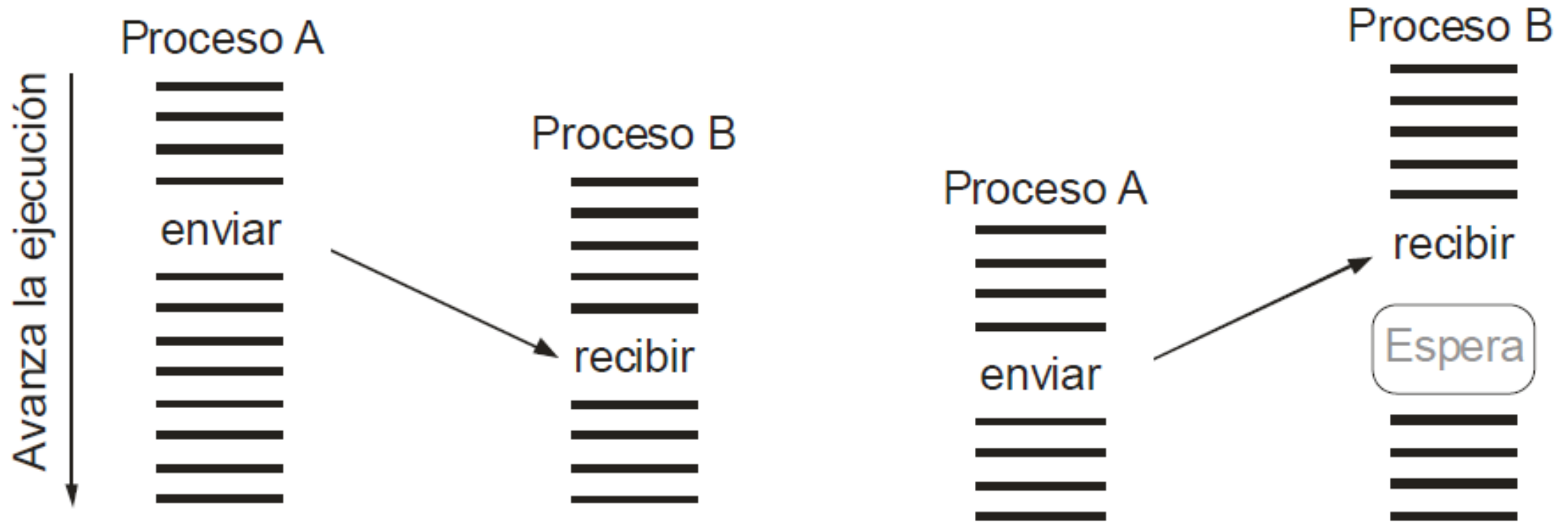
Comunicación síncrona (**bloqueante**)

- Enviar() y Recibir() son operaciones de bloqueo



Comunicación asíncrona (**no bloqueante**)

- Enviar() es no bloqueante
- Recibir() puede ser bloqueante o no bloqueante



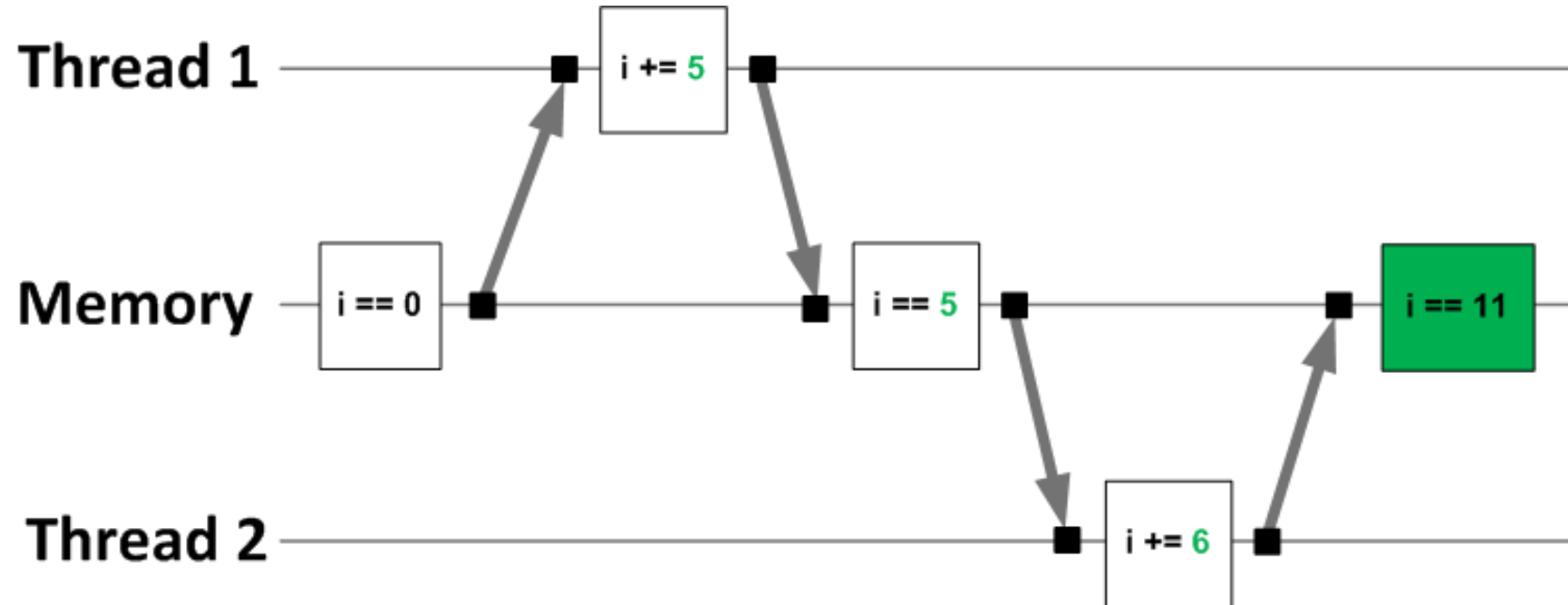
No determinismo



julia

Ejemplo: Condición de Carrera

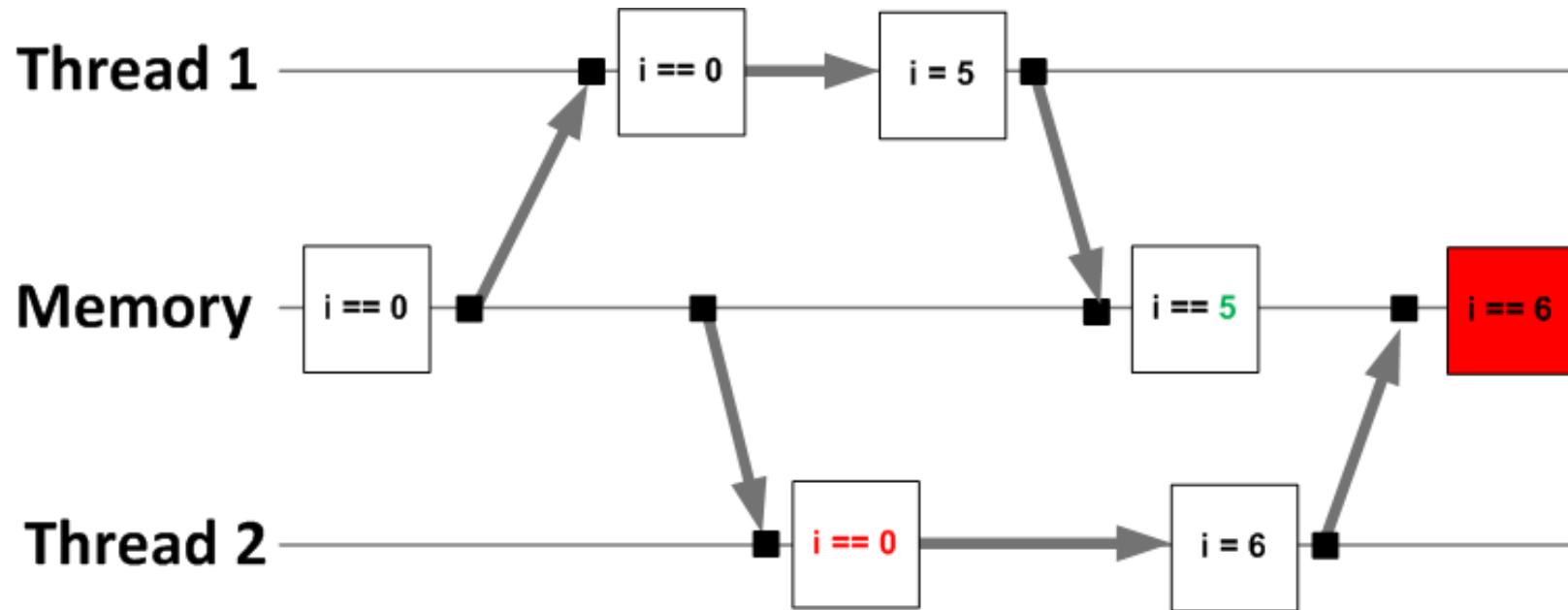
Hilo 1 suma 5 a i



Hilo 2 suma 6 a i

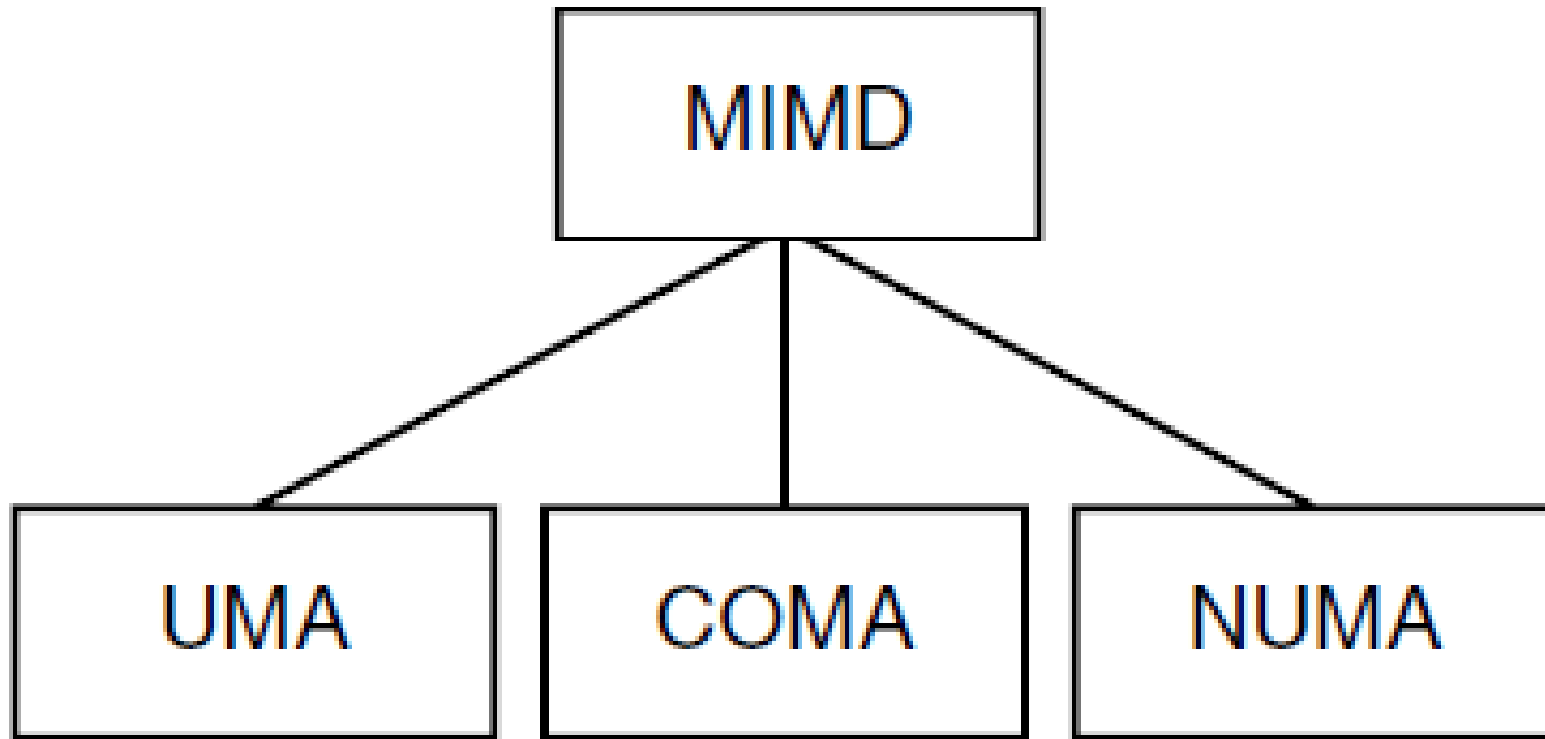
Ejemplo: Condición de Carrera

Hilo 1 suma 5 a i

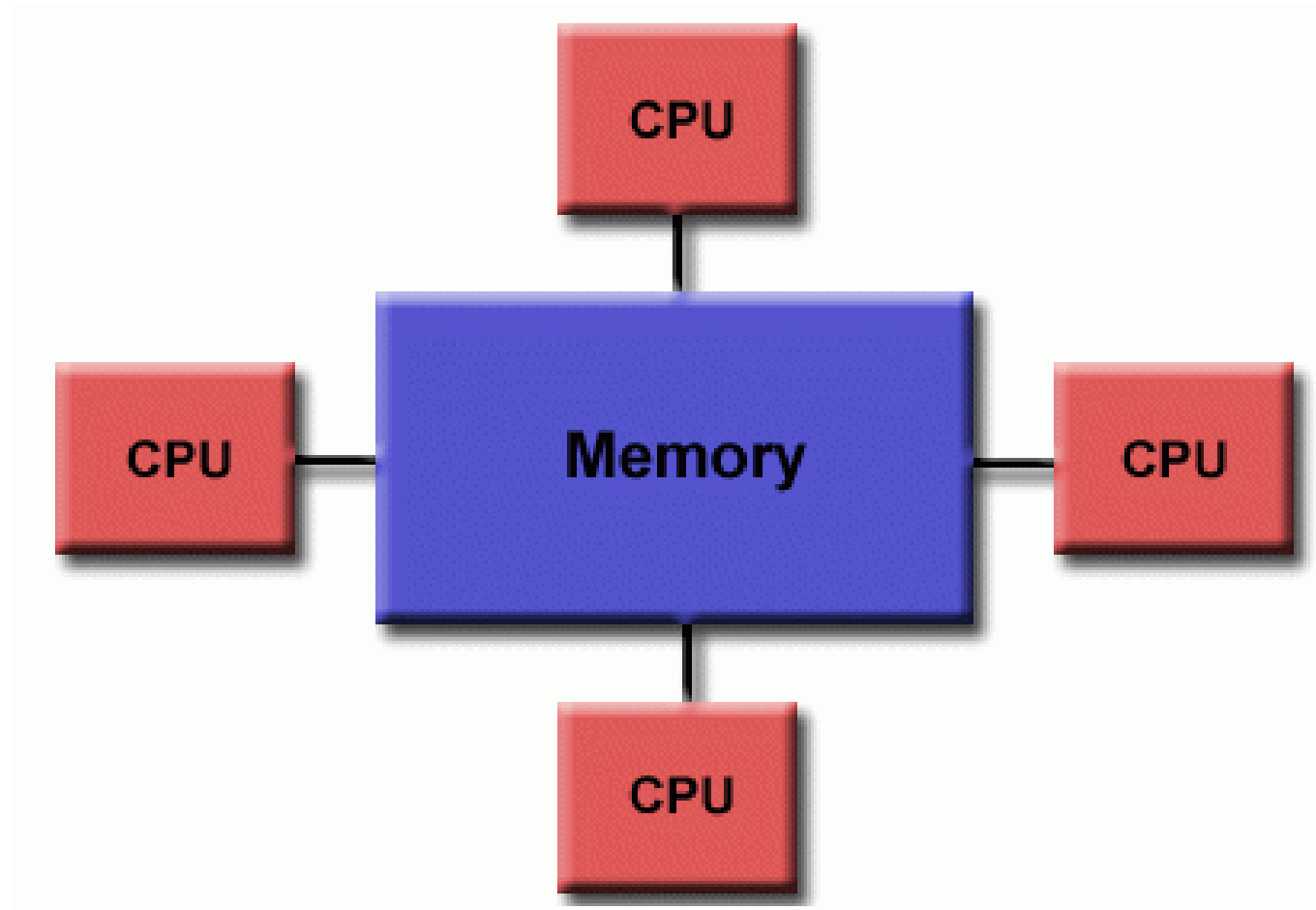


Hilo 2 suma 6 a i

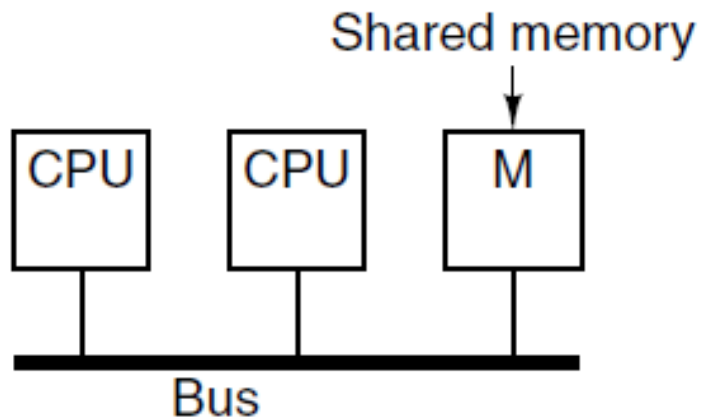
MIMD – Subclasificación en función de la memoria



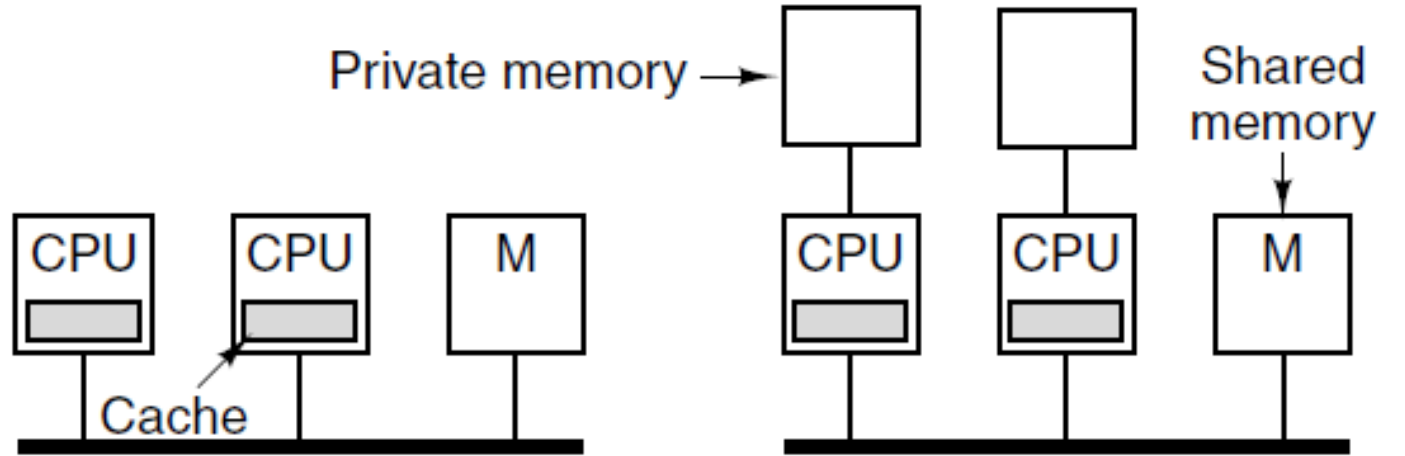
UMA – Uniform Memory Access



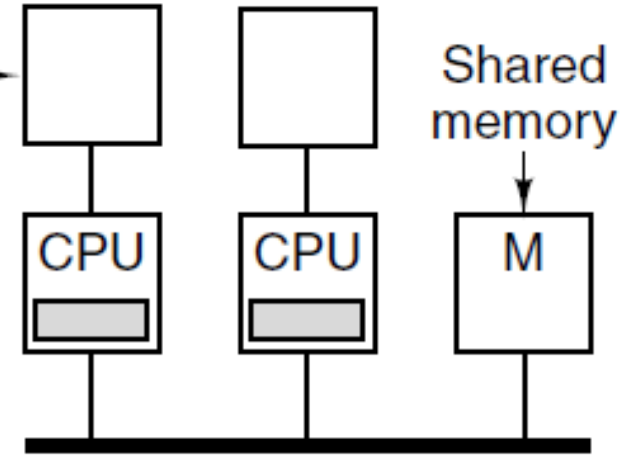
UMA – Uniform Memory Access



(a)

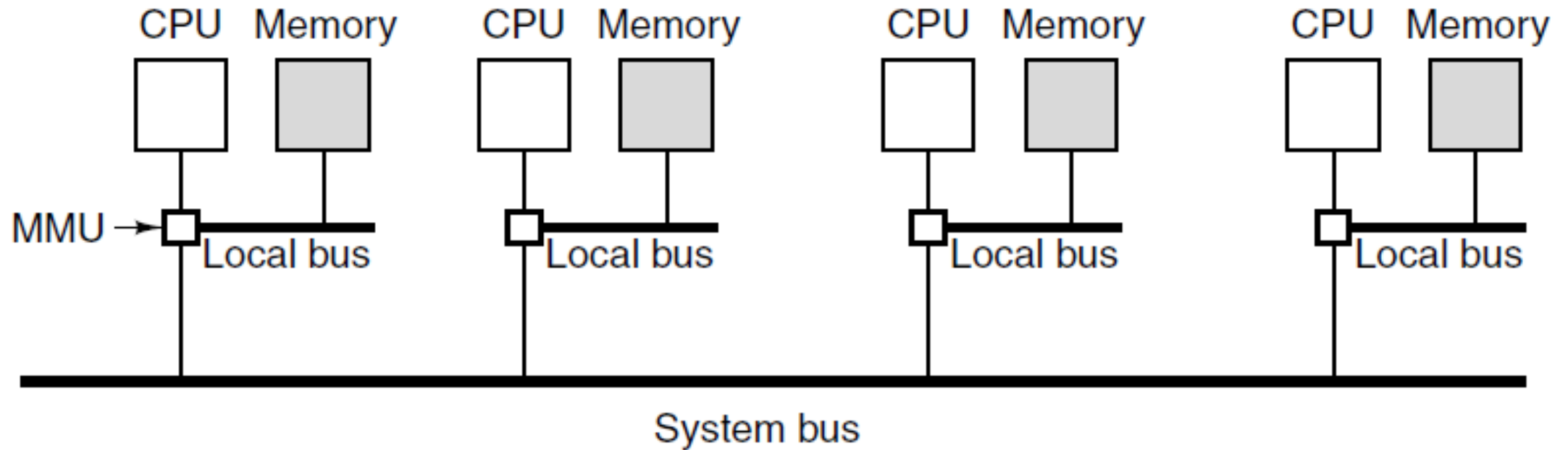


(b)

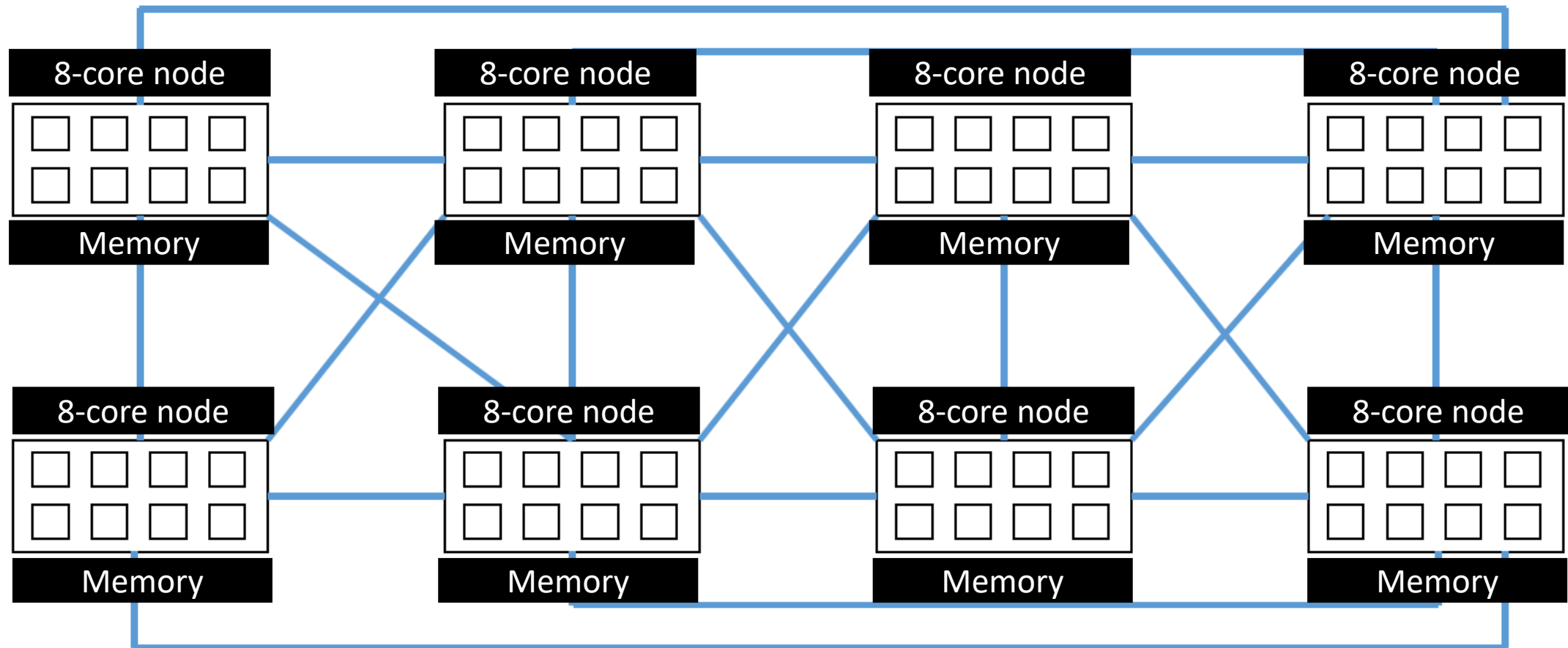


(c)

NUMA – Non-Uniform Memory Access

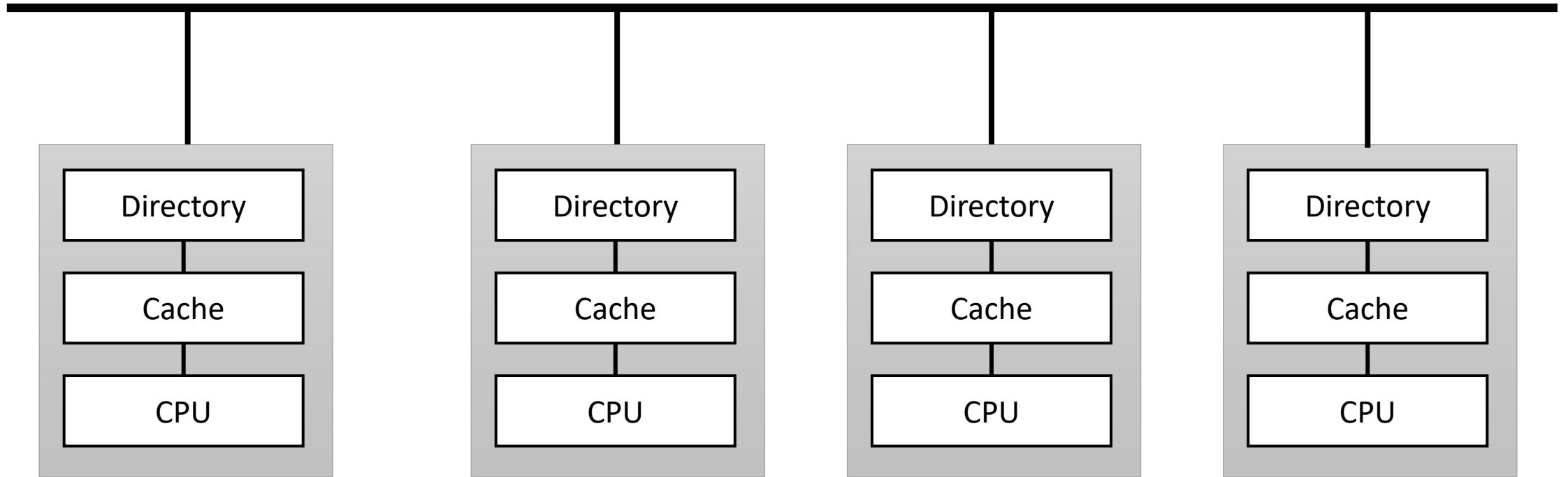


NUMA – Non-Uniform Memory Access

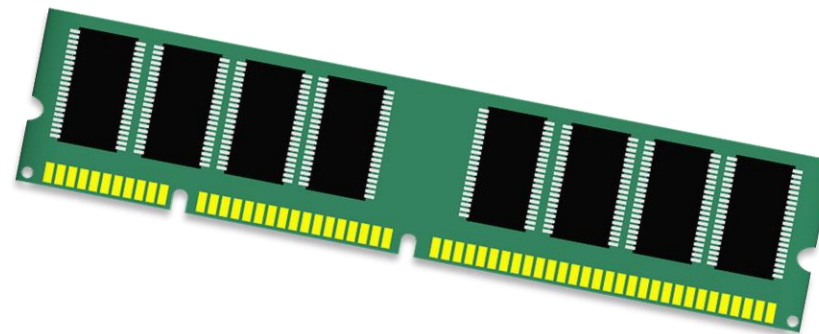


COMA - Cache-Only Memory Architecture

No uniforme y no memoria principal.



Accesso a Memoria



Espacio de direcciones de memoria de un programa

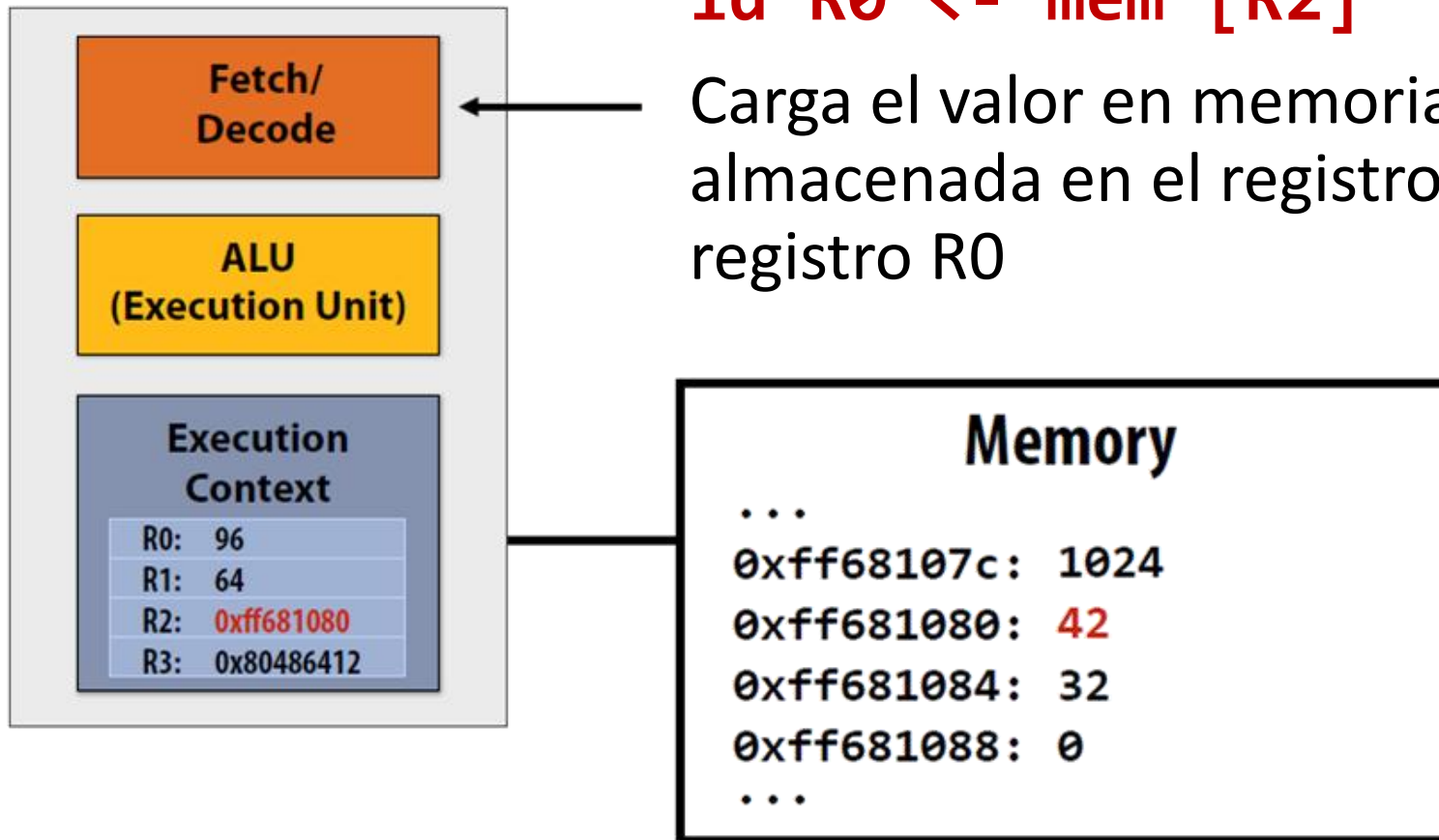
- La memoria de una computadora está organizada como un arreglo de bytes
- Cada byte es identificado por su dirección en memoria (i.e., su posición en el arreglo).

Address	Value
0x0	16
0x1	255
0x2	14
0x3	0
0x4	0
0x5	0
0x6	6
0x7	0
0x8	32
0x9	48
0xA	255
0xB	255
0xC	255
0xD	0
0xE	0
0xF	0
0x10	128
⋮	⋮
0x1F	0

Load: una instrucción para acceder al contenido de la memoria

ld R0 <- mem [R2]

Carga el valor en memoria ubicado en la dirección almacenada en el registro R2 y pon el valor en el registro R0



Latencia de acceso a memoria

- Tiempo que tarda el sistema de memoria en proveer los datos al procesador
- Ejemplo: 100 ciclos de reloj, 100 ns



Solicitud de dato

Latencia

Memory

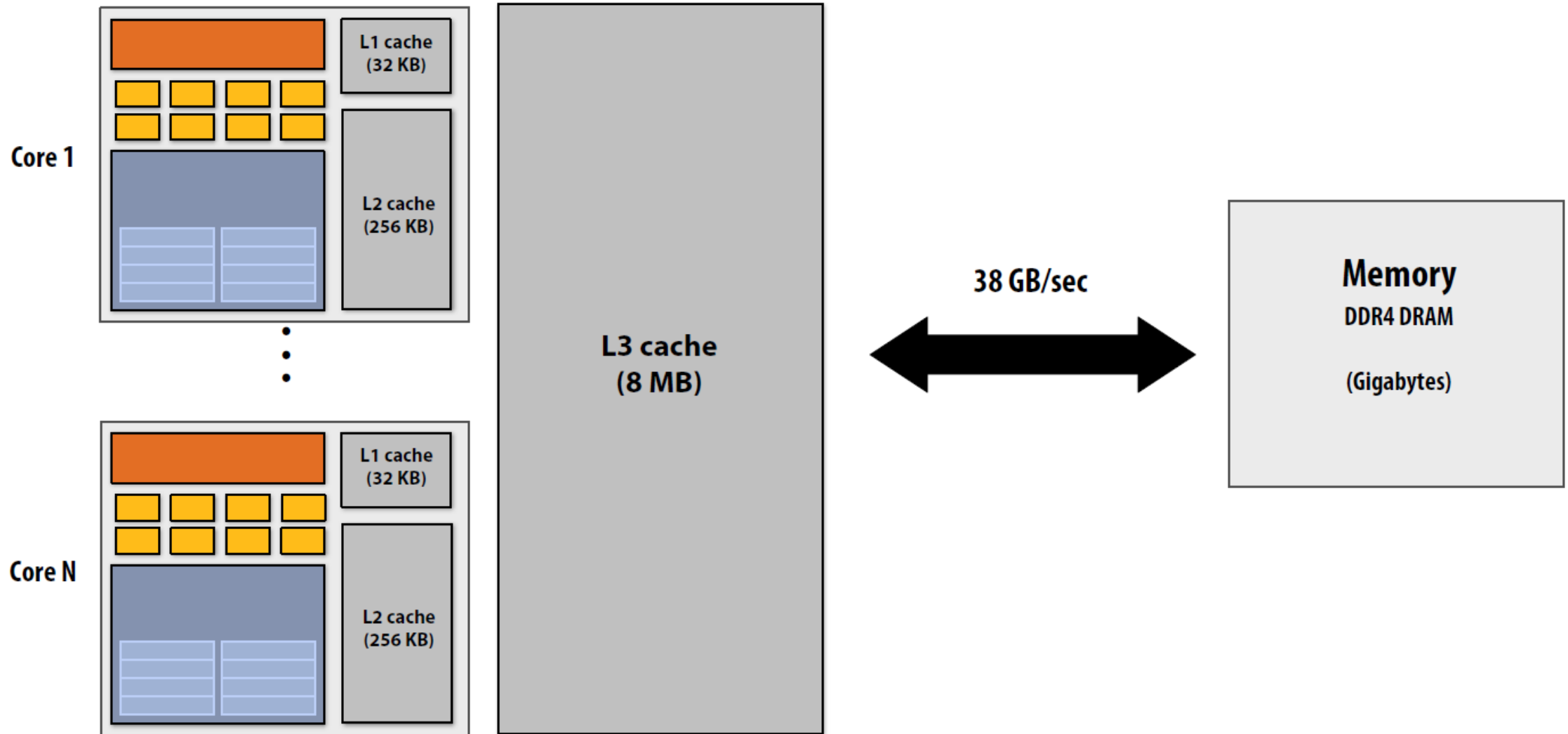
“Paros” del Procesador

- Un procesador se detiene si no puede ejecutar una instrucción debido a una dependencia de orden.
- El acceso a memoria es una fuente importante de “paros”.

```
ld r0 mem[r2]  
ld r1 mem[r3]  
add r0, r0, r1
```

La operación add no se puede ejecutar hasta que se traigan los datos de mem[r2] y mem[r3]

“Paros” del Procesador



Tiempos de acceso

El caso de CPUs Kaby Lake

Latencia (número de ciclos)

Datos en cache L1

4



Datos en cache L2

12



Datos en cache L3

38



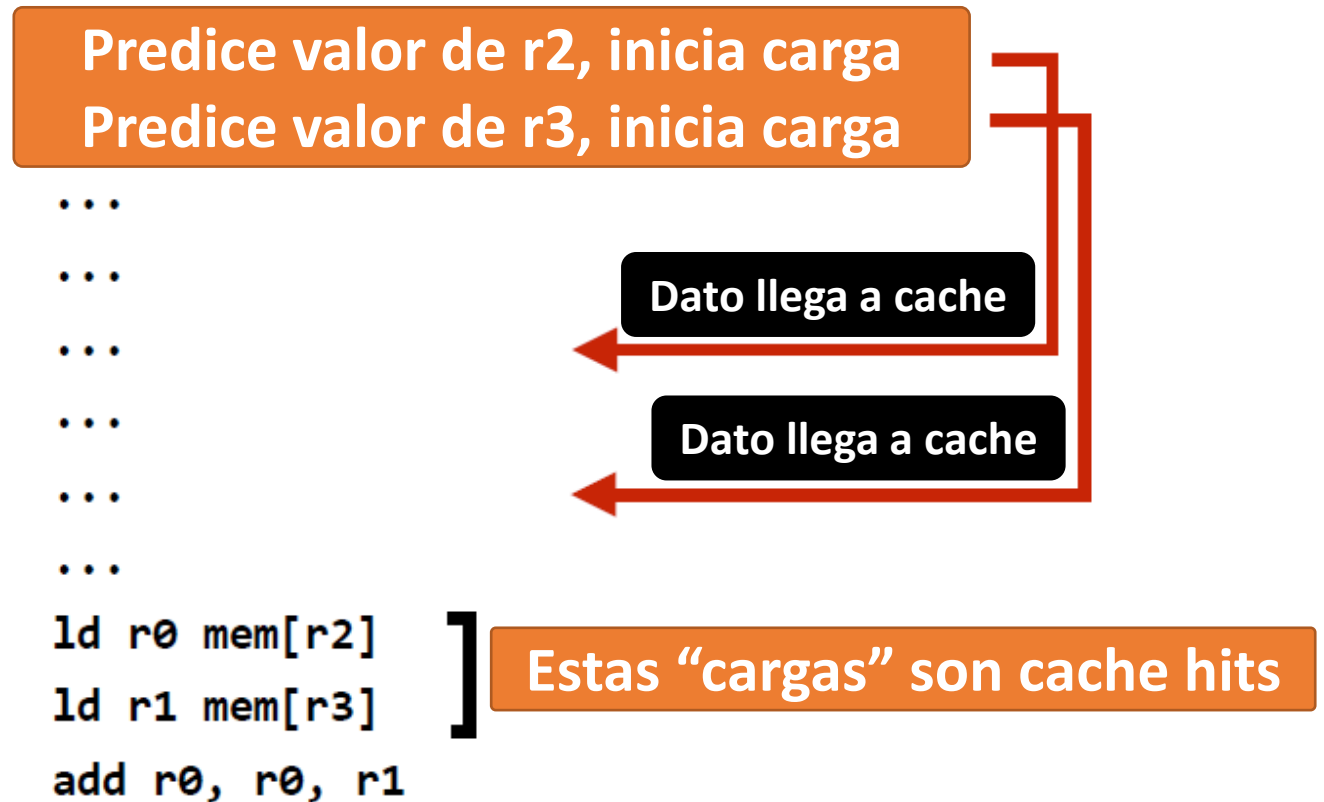
Datos en DRAM (mejor caso)

~248

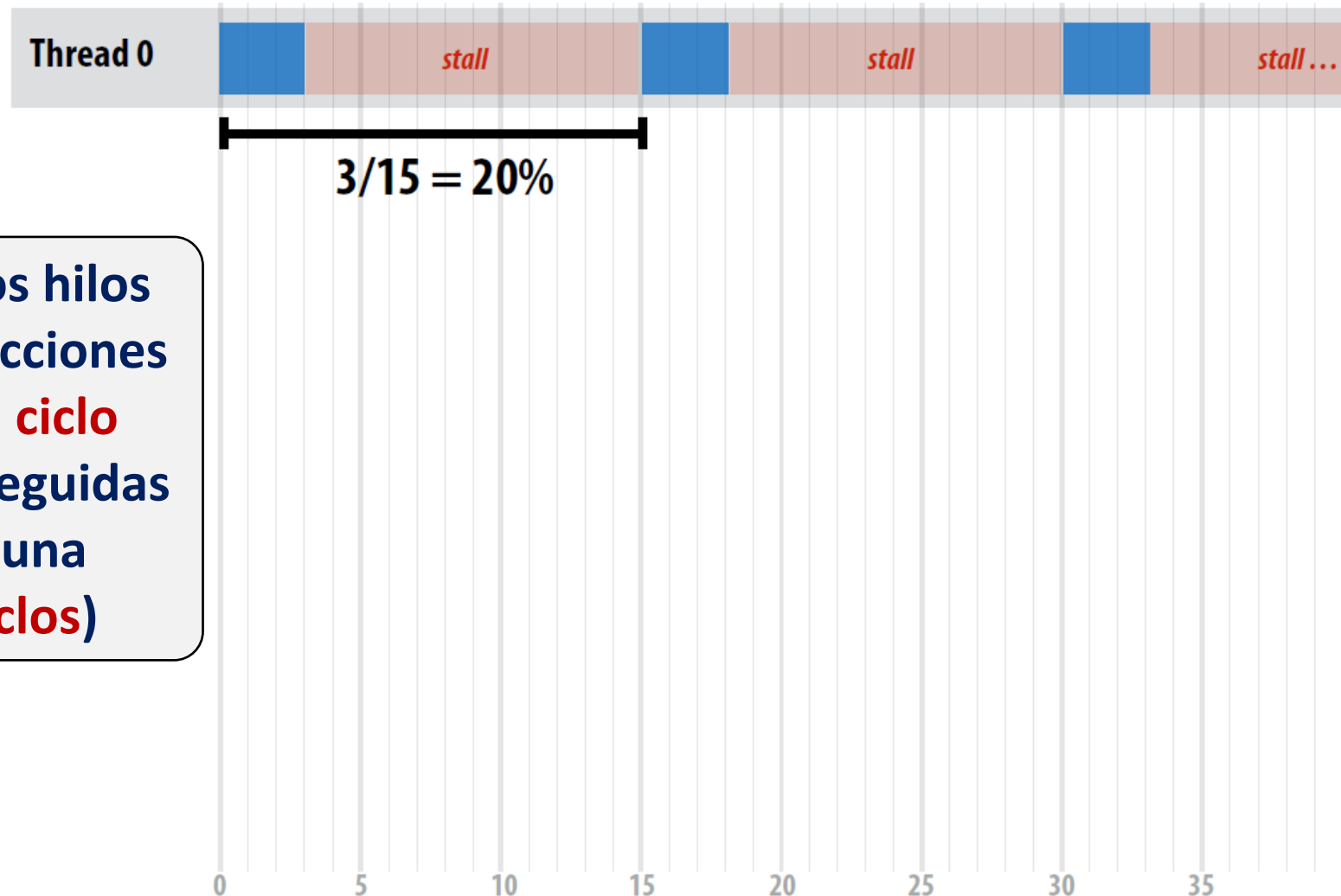


Pre-fetching reduce los “Paros” y esconde la latencia

Se analizan los patrones de acceso a memoria para hacer predicciones dinámicamente

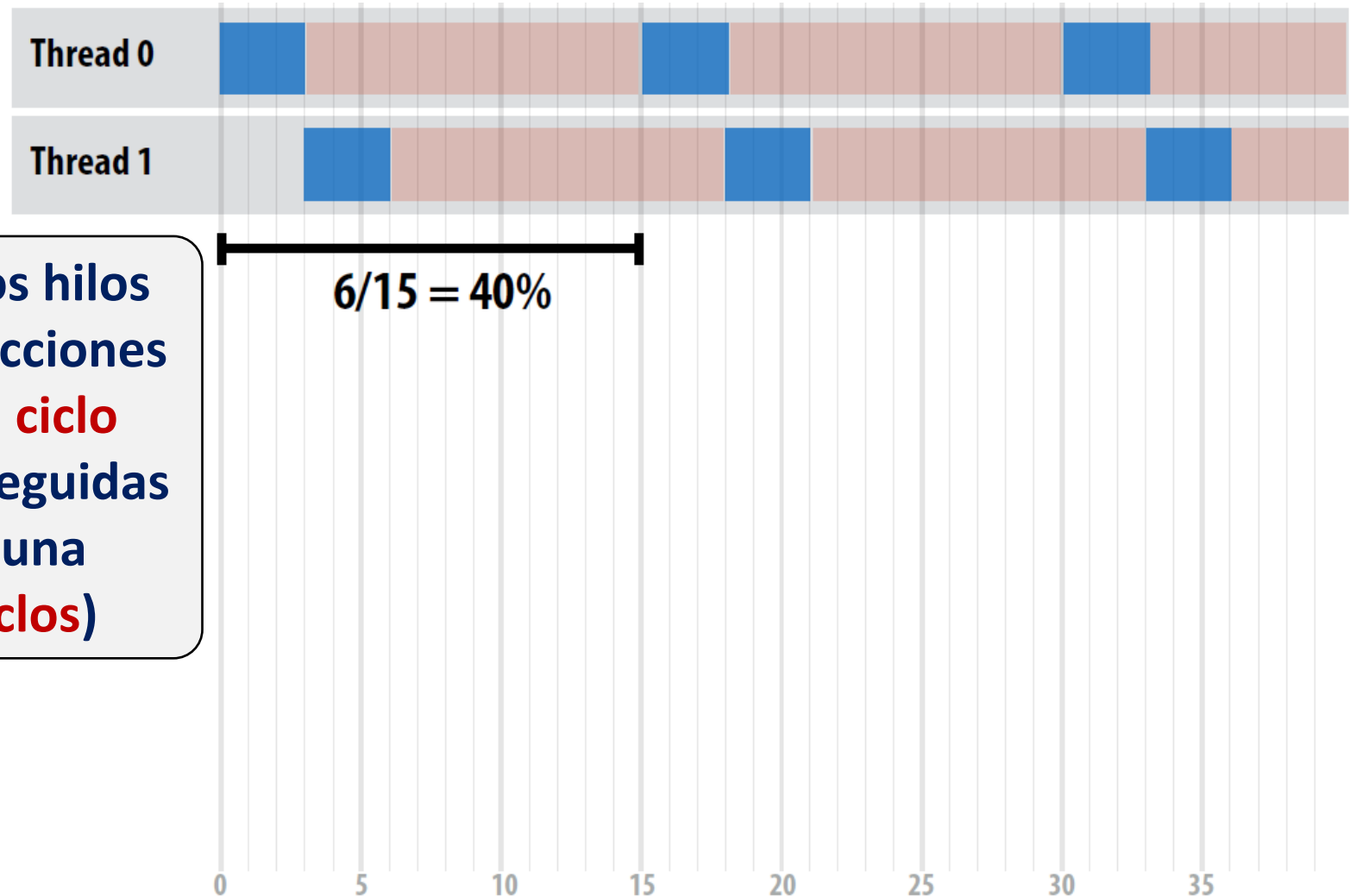


El enfoque multi-hilo reduce los “paros”



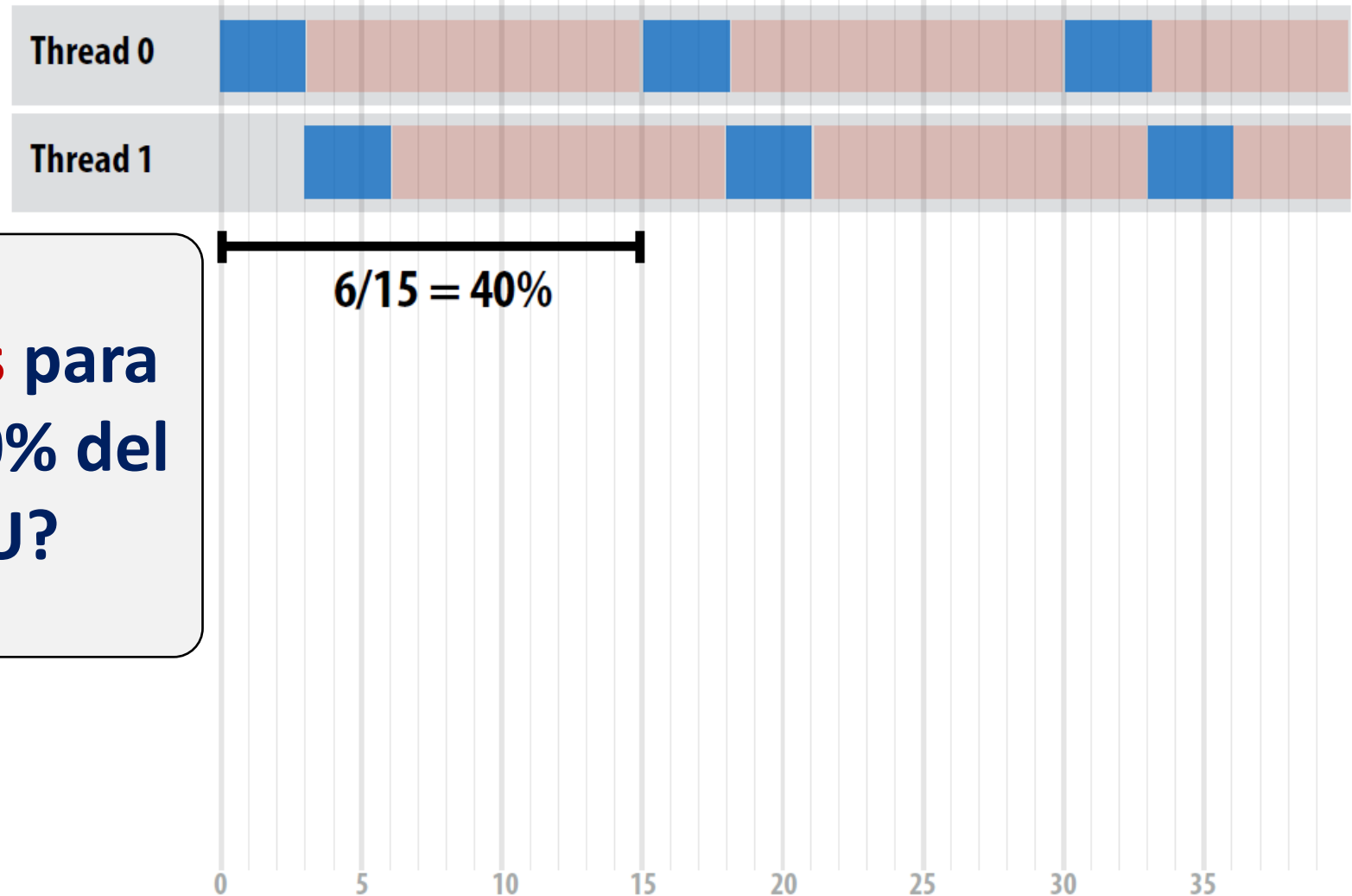
Programa donde los hilos ejecutan **tres** instrucciones aritméticas de un **ciclo** cada una de ellas, seguidas de un **load** (con una latencia de **12 ciclos**)

El enfoque multi-hilo reduce los “paros”



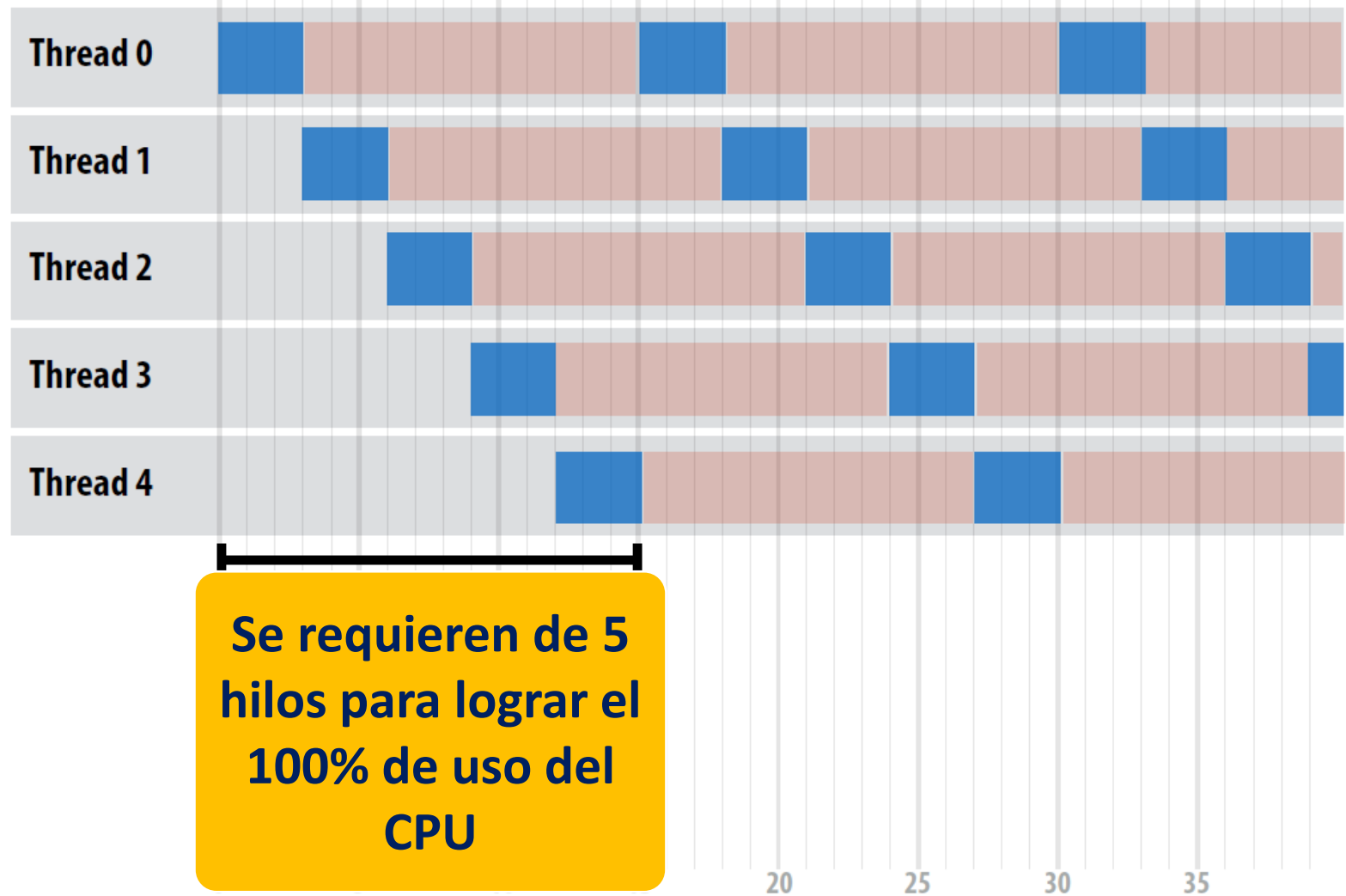
Programa donde los hilos ejecutan **tres** instrucciones aritméticas de un **ciclo** cada una de ellas, seguidas de un **load** (con una latencia de **12 ciclos**)

El enfoque multi-hilo reduce los “paros”

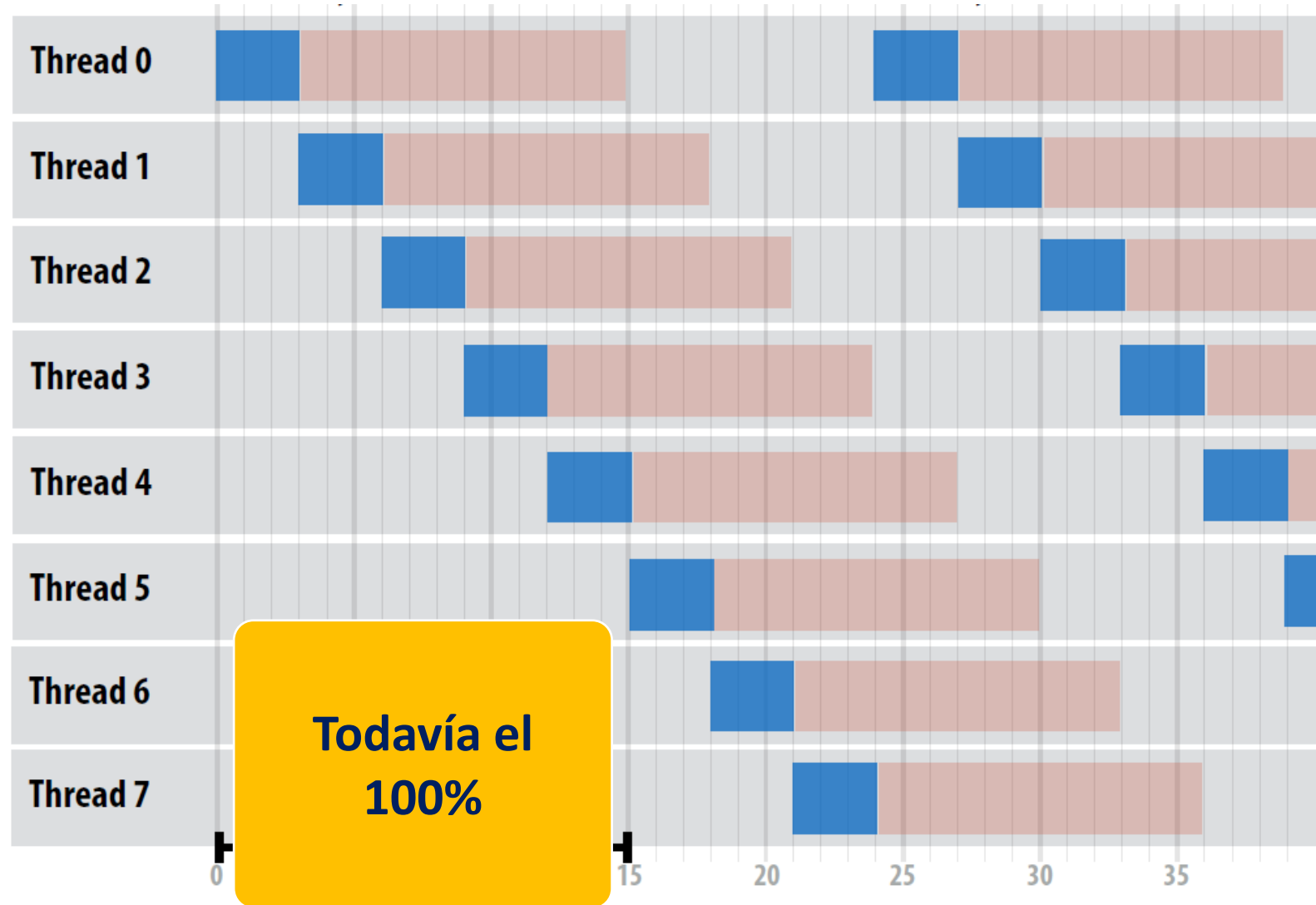


¿Cuántos hilos para alcanzar el 100% del uso del CPU?

El enfoque multi-hilo reduce los “paros”



El enfoque multi-hilo reduce los “paros”



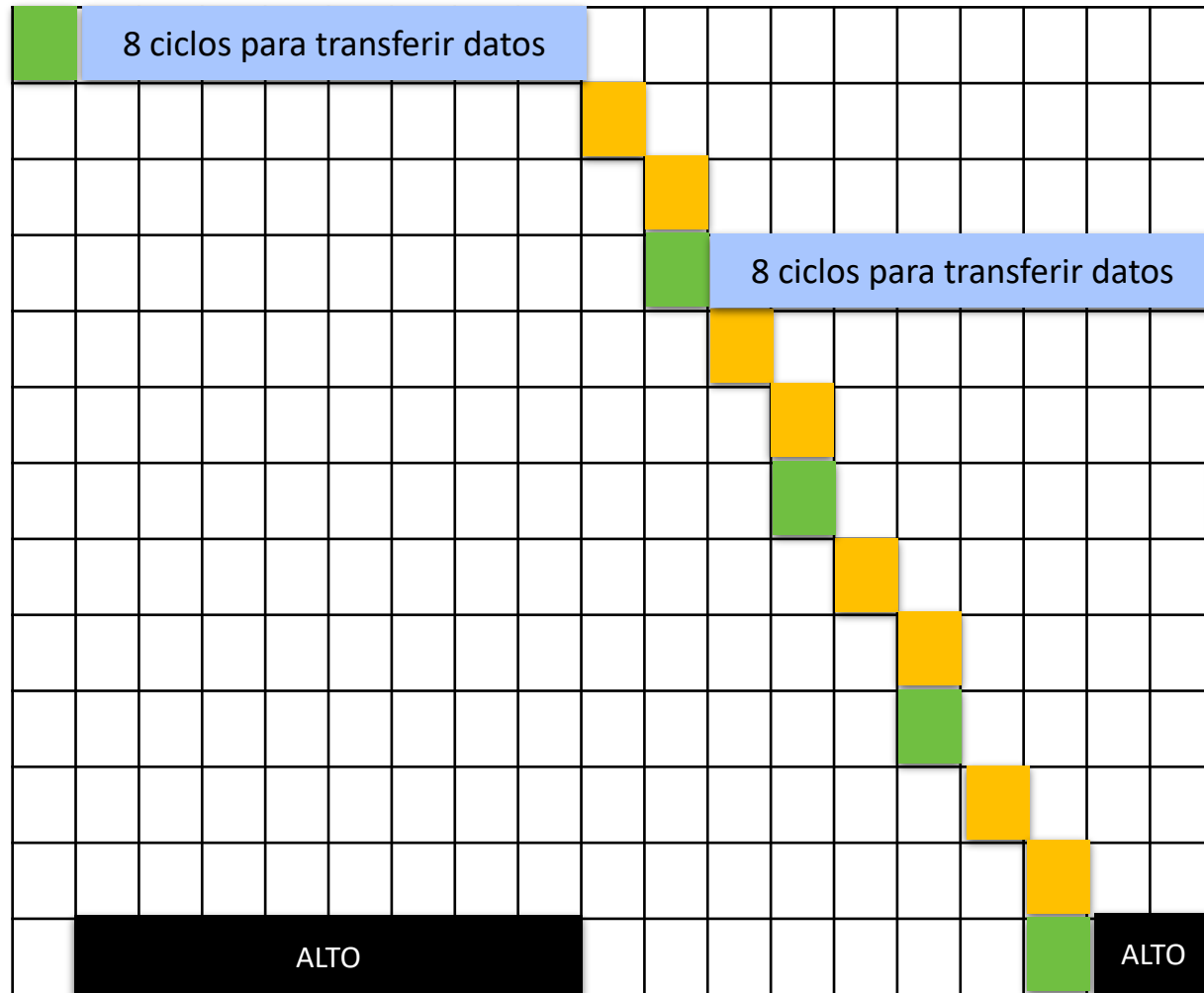


El cómputo paralelo está limitado por el ancho de banda

Los programas deben acceder a memoria no tan frecuentemente para que puedan sacar provecho de las arquitecturas paralelas modernas.

Si es posible, favorece operaciones aritméticas sobre loads/stores

Procesador que realiza un **add** por ciclo
(además de un **load** concurrente)



Transferencia 8 bytes por ciclo

Load
64 bytes

Add

Número de 8 bytes

8 ciclos para transferir datos

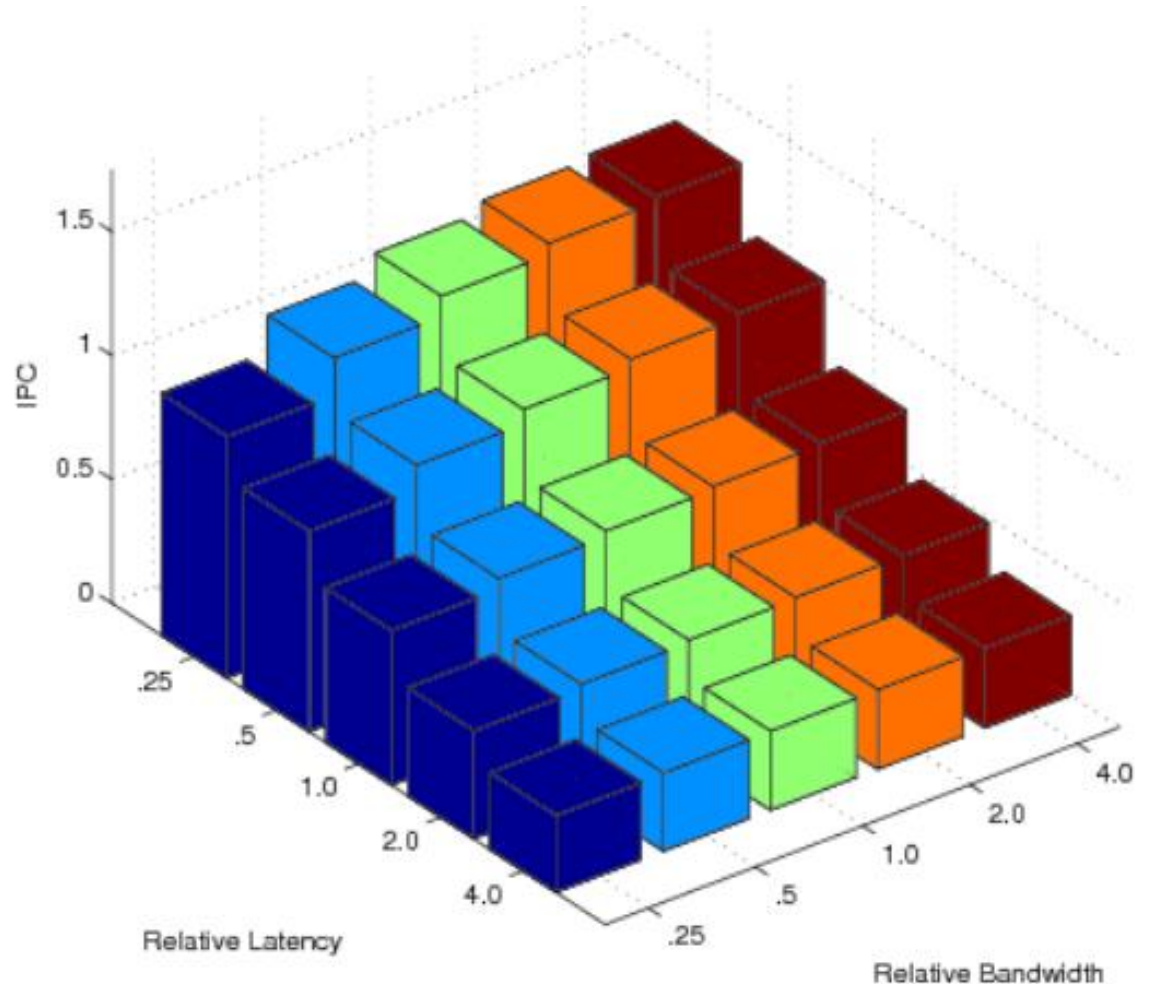
8 ciclos para transferir datos

8 ciclos

Acceso a Memoria: Latencia y Ancho de Banda

Latencia: Tiempo que pasa entre que el procesador solicita un valor de memoria y cuando el primer byte de esa solicitud llega.
Ejemplo: 15 ns

Ancho de banda: Velocidad de transferencia del segundo y todos los bytes subsecuentes de una solicitud de memoria. Ejemplo: 2.5 GB/s



Overhead de sincronización y latencia de acceso a memoria

```
function disminucion(n)
    valor = 10000000
    for i in 1:n
        valor -= 1
    end
end
```

Haz tres versiones de la función *disminución* y mide su tiempo de ejecución con `@time`

- 1) para que se ejecute con *hilos sin usar candados*
- 2) para que se ejecute con *hilos usando candados*
- 3) para que se ejecute con *hilos sin usar candados*, pero reemplazando la instrucción *valor -= 1* con *10000000 - i*

Utiliza un valor de $n = 10000000$

